

PROJET DE FIN D'ÉTUDES
2023-2024

DIPLÔME NATIONAL D'INGÉNIEUR

SPÉCIALITÉ : *NIDS*

***SUJET: Automatisation et sécurisation
du déploiement d'une application de
gestion RH intelligente sur AWS***

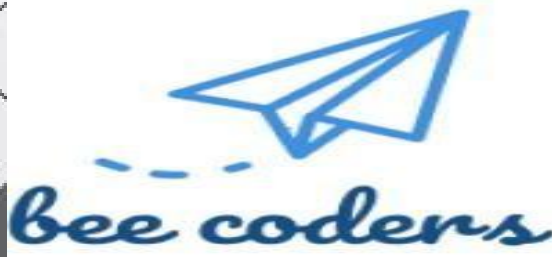
Réalisé par :

Memni Emna

Encadré par :

Encadrant ESPRIT : Mr hassen

Jedidi Encadrant Entreprise : Nefeti AhMED



Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I validate the submission of the student's report :

Nom & Prénom /Name & Surname:Memni Emna

Encadrant Entreprise/ Business site Supervisor

- *Nom & Prénom /Name & Surname: Ahmed naffeti*
Cachet & Signature /Stamp & Signature



Encadrant Académique/Academic Supervisor

- *Nom & Prénom /Name & Surname : Mr hassen Jedidi*
Signature / Signature

Ce formulaire doit être rempli, signé et scanné/This form must be completed, signed, and

scanned.

Dédicace

«Je souhaite exprimer ma reconnaissance sincère envers mes parents pour leur patience et leur soutien constant tout au long de mon cursus académique. Les encouragements qu'ils m'ont prodigués ont été la principale source d'inspiration pour moi. Je tiens également à exprimer ma gratitude envers mes amis qui ont partagé avec moi des instants de bonheur et d'anxiété, contribuant ainsi à rendre cette expérience encore plus mémorable. En conclusion, je tiens à exprimer ma profonde gratitude envers Yasmine, ma meilleure amie, pour sa présence bienveillante et son soutien moral. Ce mémoire de fin d'études est dédié à l'ensemble des lecteurs.

Je tiens à exprimer ma gratitude sincère envers tous mes enseignants, depuis l'école primaire jusqu'à ESPRIT, pour leur soutien moral, leur encadrement et leur suivi attentif».

Remerciement

À l'issue de cette étude, je souhaite exprimer ma gratitude sincère envers M. Hassen Jedidi, mon superviseur à l'École Supérieure Privée des Ingénieurs et de Technologies, pour son dévouement, ses conseils avisés, sa modestie et sa constance dans le suivi. Je tiens à exprimer ma profonde gratitude envers M. Ahmed Naffeti, mon superviseur chez Beecoders, pour sa collaboration et sa modestie exemplaires. Il a toujours été présent pour répondre à mes questions. Je souhaite exprimer ma reconnaissance à l'ensemble de l'équipe de Beecoders, ainsi qu'à l'ensemble du personnel de l'entreprise, pour leur accueil chaleureux. Mes salutations et ma gratitude sont également adressées à l'ensemble de nos enseignants, du primaire à l'université, pour l'ensemble des efforts déployés en faveur de notre éducation tout au long de notre parcours scolaire. En conclusion, je tiens à exprimer ma gratitude envers les membres du jury pour l'honneur qu'ils me font en acceptant d'évaluer mon travail.

Table de matière

Chapitre 1 : Présentation générale du projet

1.1 Introduction	2
1.2 Présentation de l'entreprise d'accueil	3
1.3 Présentation du projet et problématique	3
1.4 Étude préalable du projet	4
1.4.1 Étude de l'existant	4
1.4.2 Critiques de l'existant	4
1.4.3 Solutions proposées	5
1.5 Méthodologie de travail	5
1.5.1 Étude comparative des différentes méthodologies	6
1.5.2 Choix de la méthode	7
1.5.3 Présentation de la méthode Scrum	7
1.5.4 Les intervenants dans Scrum et les outils utilisés	8
1.6 Conclusion	9

Chapitre 2 : Planification et besoins

2.1 Introduction	11
2.2 Identification des besoins	11
2.2.1 Besoins fonctionnels	11
2.2.2 Besoins non-fonctionnels	11
2.3 Analyse des besoins	12
2.3.1 Diagramme de cas d'utilisation	12
2.4 Pilotage de projet avec Scrum	12
2.4.1 Outil de gestion du projet	12
2.5 Planification du projet avec Scrum	13
2.5.1 Planification des sprints	13
2.6 Architecture globale du projet	13
2.7 Environnement de travail	13
2.7.1 Orchestration	13
2.7.2 Intégration et déploiement continu (CI/CD)	14
2.7.3 Sécurité	15
2.7.4 Supervision & Monitoring	17
2.7.5 Base de données	18
2.7.6 Infrastructure as Code – Terraform	19
2.7.7 Cloud Provider – Amazon Web Services (AWS)	19

Chapitre 3 : Release 1 – Mise en place de l'infrastructure

3.1 Sprint 1 : Préparation de l'infrastructure de base	21
3.1.1 Objectif du sprint	21
3.1.2 Configuration de l'infrastructure	21
3.1.3 Création du VPC	21
3.1.4 Déploiement du cluster EKS	22
3.1.5 Configuration de DocumentDB	23
3.1.6 Gestion des permissions IAM	24
3.1.7 Groupes de nœuds managés EKS	25
3.2 Sprint 2 : Déploiement de l'infrastructure et vérification	26
3.2.1 Résultats du déploiement	26
3.2.2 Captures d'écran AWS	26
3.2.3 VPC et sous-réseaux	26
3.2.4 Cluster EKS	26
3.2.5 Cluster DocumentDB	27
3.2.6 IAM Roles et politiques	28
3.2.7 Résultat de terraform apply	28
3.3 Sprint 3 : Déploiement Kubernetes et microservices	29
3.3.1 Mise en œuvre de Kubernetes	29
3.3.2 Objectif	29
3.3.3 Architecture du cluster Kubernetes	29
3.3.4 Composants du cluster	29
3.3.5 Déploiement avec Helm	29
3.3.6 Structure du Chart Helm	29
3.3.7 Fichier Chart.yaml	29
3.3.8 Fichier values.yaml	30
3.3.9 Vérification du bon fonctionnement	31

Chapitre 4 : Release 2 – Automatisation CI/CD et sécurité

4.1 Sprint 1 : Mise en place du pipeline CI/CD	34
4.1.1 Objectif	34
4.1.2 GitHub Actions : workflow CI/CD	35
4.2 Sprint 2 : Sécurité et analyse des vulnérabilités	39
4.2.1 Trivy : scan des images	40
4.2.2 Analyse des dépendances avec OWASP Dependency- check	42
4.2.3 Analyse dynamique avec OWASP ZAP	45
4.3 Sprint 3 : Configuration et Supervision avec ArgoCD	49
4.3.1 Déploiement GitOps avec ArgoCD	49
4.3.2 Surveillance des applications déployées	49

Chapitre 5 : Release 3 - Monitoring avec Grafana et Prometheus

5.1 Sprint 1 : Monitoring avec Grafana et Prometheus	55
5.1.1 Présentation de la stack Prometheus-Grafana	55
5.1.2 Prometheus : Collecte et stockage des métriques	55
5.1.2.1 Principaux composants dPrometheus	55
5.1.3 Grafana : Visualisation et analyse des métriques	55
5.1.4 Déploiement sur Kubernetes	55
5.1.4.1 Utilisation de Helm pour le déploiement	55
5.1.4.2 Configuration de l'Ingress ALB et accès sécurisé via HTTPS	55
5.1.5 Surveillance des microservices et du cluster avec Grafana	57
Conclusion	58

Chapitre 6 : Release 4 Conception et développement

d e l'application JobyGénie

Sprint 1 : Conception et développement des microservices

6.1.1 Introduction	60
6.1.2 Spécifications architecturales	60
6.1.3 Introduction	60
6.1.4 Spécifications architecturales	60
6.1.4.1 Architecture microservices	60
6.1.5 Caractéristiques de l'architecture microservices	60
6.3 Microservices implémentés	63
6.3.1 MS-Auth (Authentification)	63
6.3.1.1 Contexte	63
6.3.1.2 Fonctionnalités	63
6.3.2 MS-Job-And-Application (Offres et candidatures)	65
6.3.2.1 Contexte	65
6.3.2.2 Fonctionnalités	65
6.3.3 MS-CvAnalyzer (Analyse de CV par IA)	69
6.3.3.1 Contexte	69
6.3.3.2 Comparaison des modèles pré-entraînés pour l'analyse et la mise en correspondance de texte	70
6.3.3.3 Captures des interfaces du microservice CvAnalyzer	73
Conclusion	73

<i>CONCLUSIONS AND PERSPECTIVES</i>	74
<i>Bibliographie</i>	75

Table des Figures

Figure 1.1 : Logo Beecoders.....	2
Figure 1.2 : Méthode Agile vs Méthodes Classiques.....	6
Figure 1.3 : Méthode SCRUM.....	7
Figure 2.1 : Architecture Globale du Projet.....	13
Figure 2.2 : Logo de Kubernetes.....	14
Figure 2.3 : Logo de GitHub Actions.....	14
Figure 2.4 : Logo de DockerHub.....	14
Figure 2.5 : Logo d'ArgoCD.....	15
Figure 2.6 : Logo de Helm.....	15
Figure 2.7 : Logo de Trivy.....	15
Figure 2.8 : Logo d'OWASP Dependency-Check.....	16
Figure 2.9 : Logo d'OWASP Zed Attack Proxy (ZAp).....	17
Figure 2.10 : Logo de Grafana et Prometheus.....	17
Figure 2.11 : Logo d'Amazon DocumentDB.....	18
Figure 2.12 : Logo de Terraform.....	18
Figure 2.22 : Logo AWS.....	18
Figure 3.1 : Architecture de Réseau VPC avec Sous-réseaux et Services Associés.....	21
Figure 3.2 : Fichier Terraform (vpc.tf) Définissant le Module VPC.....	22
Figure 3.3 : Configuration Terraform du Cluster Amazon EKS kubernetes-sec-213.....	23
Figure 3.4 : Configuration Terraform pour le Cluster Amazon DocumentDB et ses Règles de Sécurité.....	24
Figure 3.5 : Configuration Terraform des Rôles et Permissions IAM pour le Cluster EKS.....	24
Figure 3.6 : Configuration Terraform pour un Cluster EKS avec Groupes de Nœuds Managés.....	25
Figure 3.7 : VPC et Sous-réseaux Créés dans la Console AWS.....	26
Figure 3.8 : Cluster EKS et Groupes de Nœuds dans la Console AWS.....	27
Figure 3.9 : Cluster DocumentDB dans la Console AWS.....	27
Figure 3.10 : Rôles et Politiques IAM dans la Console AWS.....	28
Figure 3.11 : Sortie de terraform apply Confirmant la Création des Ressources.....	28
Figure 3.12 : Fichier chart.yaml.....	30
Figure 3.13 : Fichier values.yaml.....	31
Figure 3.14 : Vérification des Namespaces.....	31
Figure 3.15 : Vérification des Pods.....	32
Figure 3.16 : Vérification des Services.....	32
Figure 3.17 : Vérification des Deployments.....	32
Figure 3.18 : Vérification des Ingress.....	32
Figure 4.1 : Architecture de Pipeline DevOps.....	34
Figure 4.2 : Workflow CI/CD GitHub Actions pour le microservice ms-Auth.....	36
Figure 4.3 : Pipeline GitHub Actions pour le déploiement de microservice ms-Auth sur AWS EKS.....	36
Figure 4.4 : Workflow CI/CD GitHub Actions pour le microservice ms-dashboard-and Analytics.....	37
Figure 4.5 : Pipeline GitHub Actions pour le déploiement de microservice ms-Dashboard sur AWS EKS.....	37

Figure 4.6 : Workflow CI/CD GitHub Actions pour le microservice cv-Analyzer.....	Figure 4.7 : Pipeline GitHub Actions pour le déploiement de microservice cvAnalyzer sur AWS EKS.....	38
Figure 4.8 : Workflow CI/CD GitHub Actions pour le microservice MS-Job-Application.....		39
Figure 4.9 : Pipeline GitHub Actions pour le déploiement de microservice MS-Job-And-Application sur AWS EKS.....		39
Figure 4.10 : Rapport de scan de sécurité Docker avec Trivy pour le microservice-MS-Auth.....		40
Figure 4.11 : Rapport de scan de sécurité Docker avec Trivy pour le microservice MS-job-And-Application.....		41
Figure 4.12 : Rapport de scan de sécurité Docker avec Trivy pour le microservice MS-Dashboard-Analytics.....		41
Figure 4.13 : Rapport de scan de sécurité Docker avec Trivy pour le microservice CvAnalyzer.....		42
Figure 4.14 : Rapport de sécurité généré par OWASP Dependency-Check de MS-Auth.....		42
Figure 4.15 : Rapport de Vulnérabilités - Dépendances Critiques (CVSS) de MS-Auth.....		43
Figure 4.16 : Rapport de sécurité généré par OWASP Dependency-Check de MS-Dashboard-Analytics.....		43
Figure 4.17 : Rapport de Vulnérabilités - Dépendances Critiques (CVSS) pour MS-Dashboard-Analytics.....		43
Figure 4.18 : Rapport de sécurité généré par OWASP Dependency-Check de MS-Job-AndApplication.....		44
Figure 4.19 : Rapport de Vulnérabilités - Dépendances Critiques (CVSS) pour MS-Job-AndApplication.....		44
Figure 4.20 : Résumé du scan de sécurité ZAP de MS-Auth.....		45
Figure 4.21 : Liste des Vulnérabilités + Rapport Résumé d'Analyse de MS-Auth.....		45
Figure 4.22 : Résumé du scan de sécurité ZAP de CvAnalyzer.....		46
Figure 4.23 : Liste des Vulnérabilités + Rapport Résumé d'Analyse de CvAnalyzer.....		46
Figure 4.24 : Résumé du scan de sécurité ZAP de MS-Job-And-Application.....		47
Figure 4.25 : Liste des Vulnérabilités + Rapport Résumé d'Analyse de MS-Job-And-Application.....		47
Figure 4.26 : Résumé du scan de sécurité ZAP de MS-Dashboard-And-Analytics.....		48
Figure 4.27 : Liste des Vulnérabilités + Rapport Résumé d'Analyse de MS-Dashboard AndAnalytics.....		48
Figure 4.28 : Diagramme d'activité.....		50
Figure 4.29 : Dashboard de monitoring des applications - ArgoCD (Sync & Health Status).....		52
Figure 5.1 : Configuration Helm - Grafana Ingress ALB et Alertmanager.....		56
Figure 5.2 : Interface de Gestion des Dashboards Grafana.....		57
Figure 5.3 : Tableau de bord Kubernetes - Métriques Kubelet et Opérations.....		57
Figure 6.1 : Comparaison architectures monolithique vs microservices.....		61
Figure 6.2 : Diagramme de cas d'utilisation Authentification.....		63
Figure 6.3 : Interface d'inscription.....		64
Figure 6.4 : Interface d'authentification.....		64
Figure 6.5 : Confirmation de connexion.....		65
Figure 6.6 : Diagramme de cas d'utilisation : MS-JobApplication.....		66
Figure 6.7 : Tableau de bord principal (statistiques et offres).....		66
Figure 6.8 : Diagramme de séquence : Ajout d'une offre par le RH.....		67
Figure 6.9 : Interface de gestion des emplois.....		67
Figure 6.10 : Suivi des candidatures.....		68
Figure 6.11 : Formulaire d'ajout de candidature.....		68

Figure 6.12 : Diagramme de cas d'utilisation : CVAnalyzer.....	69
Figure 6.13 : Comparatif des modèles IA (Gemini vs GPT).....	70
Figure 6.14 : Interface de matching IA.....	71
Figure 6.15 : Résultat d'analyse IA pour "System Administrator".....	71
Figure 6.16 : Gestion des tests techniques.....	72
Figure 6.17 : Questions RH pour profils Cloud/DevOps.....	72
Figure 6.18 : Interface de planification d'entretien.....	73

Chapitre 1 :
Présentation générale du projet

1.1 Introduction

Dans cette partie, nous allons présenter l'organisme d'accueil, qui est « Beecoders Services and Solutions », ainsi que les services qu'il offre. Nous détaillerons ensuite la problématique abordée dans ce projet, la solution proposée, et les objectifs visés.

1.2 Présentation de l'entreprise d'accueil

L'entreprise Beecoders Services and Solutions

Fondée en 2020, Beecoders Services and Solutions est une entreprise de services numériques spécialisée dans le développement web et mobile, le design web, le consulting IT, et les formations à distance. Son objectif principal est de concevoir des produits digitaux sur-mesure qui répondent aux besoins spécifiques de ses clients et de se démarquer dans un secteur en constante évolution.



Figure 1. 1:Logo beecoders

Beecoders met à disposition l'expertise de ses développeurs, designers, et consultants pour offrir une large gamme de services, tels que :

La création de sites web modernes et interactifs.

Le développement d'applications mobiles performantes sur Android et iOS. Le consulting IT pour optimiser les performances des équipes techniques.

La formation professionnelle pour accroître les compétences des employés dans le domaine de l'informatique.

En tant que partenaire de choix pour les entreprises, Beecoders accompagne également les startups et entreprises dans la réalisation de projets IT tout en respectant leurs contraintes. Elle agit comme un véritable catalyseur en mettant en contact les étudiants passionnés avec les entreprises pour favoriser l'innovation et le développement technologique.

Avec son approche sur-mesure et son expertise reconnue, Beecoders contribue à l'essor de la transformation numérique des entreprises tout en soutenant les jeunes talents dans leur parcours professionnel.

1.3 Présentation du projet et Problématique

Après avoir présenté l'entité d'accueil, cette section traite de notre sujet, de son contexte et de la problématique qui lui est associée.

Dans un contexte numérique en évolution constante, caractérisé par l'importance croissante de la rapidité et de la fiabilité des livraisons logicielles, l'intégration des méthodes DevOps s'avère être un levier efficace pour améliorer l'efficacité du processus de développement et de déploiement des applications.

Cette méthode s'appuie sur l'automatisation, la collaboration inter-équipes et l'amélioration continue, ce qui permet d'assurer un déploiement en production plus rapide et plus efficace. Toutefois, il est tout aussi essentiel d'intégrer la sécurité dans ces processus afin de prévenir toute compromission de la fiabilité des applications due à l'accélération du développement.

Ce projet s'inscrit dans cette perspective en instaurant une approche DevOps renforcée par des pratiques DevSecOps. L'objectif principal consiste à améliorer et à automatiser le processus de développement des applications web, en assurant un déploiement continu et fiable tout en incluant des mesures de contrôle de qualité et de sécurité à chaque phase. Cette méthode s'appuie sur l'exploitation de pipelines CI/CD solides, qui automatisent la construction, les tests, l'analyse du code et le déploiement, ce qui a pour effet de réduire le délai de mise en production et de limiter les erreurs humaines.

En incorporant ces méthodes, l'objectif de ce projet est de mettre en évidence comment une approche DevOps bien organisée, renforcée par des contrôles de sécurité automatisés, peut favoriser la collaboration entre les équipes, accélérer le déploiement en production et garantir une plus grande résilience des applications dans des environnements changeants et en évolution.

L'examen de nos pratiques de recrutement et de planification des effectifs met en lumière une forte utilisation de méthodes manuelles, une vulnérabilité aux préjugés humains et une sous-exploitation des données disponibles. Les méthodes traditionnelles souffrent d'inefficacités, de lacunes et omettent fréquemment des données cruciales.

ce problème, et dans la section suivante, nous allons lister certaines des plus connues.

1.4 ETUDE PREALABLE DU PROJET

1.4.1 Étude de l'existant

Il est crucial de comprendre de manière approfondie les solutions actuellement disponibles ainsi que les défis associés aux approches traditionnelles avant d'entamer la mise en œuvre du projet. L'objectif de cette analyse est de souligner les lacunes des systèmes en place et de suggérer une solution plus efficace. La société dans laquelle j'ai réalisé mon stage s'appuie actuellement sur la méthode classique de développement et de déploiement des applications web, qui englobe divers aspects.

Les contraintes. Le processus de développement était principalement effectué de manière manuelle, entraînant ainsi des retards importants entre les phases de conception, de test et de déploiement.

De plus, la gestion des versions se distinguait par sa complexité et son manque d'optimisation, ce qui rendait difficile le suivi des modifications du code, la collaboration entre les équipes et augmentait le risque d'erreurs numériques.

L'absence d'automatisation dans le processus de déploiement impliquait que chaque déploiement devait être réalisé de manière manuelle, entraînant ainsi une augmentation des risques d'erreurs, d'incohérences et de problèmes de compatibilité entre les environnements de développement, de test et de production. Cette approche se caractérisait également par une capacité limitée à réagir rapidement en cas de besoins de corrections ou d'ajouts, ce qui entraînait des retards dans l'implémentation de nouvelles fonctionnalités.

1.4.2 Critiques de l'existant

La lenteur du processus est un élément à prendre en considération. La conception et la mise en œuvre de l'application étaient principalement effectuées de façon manuelle, ce qui accroissait les risques d'erreurs humaines. La configuration des environnements, la gestion des dépendances et la validation des mises à jour exigeaient des interventions répétées et chronophages.

Limitation de la scalabilité : L'augmentation des quantités de données et du nombre d'utilisateurs n'est pas correctement prise en charge par l'infrastructure actuelle, ce qui se traduit par des problèmes de performance.

Le manque d'automatisation : Chaque équipe ou projet n'applique pas systématiquement l'automatisation, ce qui entraîne des incohérences et complique la gestion des versions et des configurations. La reproduction des déploiements dans divers environnements et la réalisation de mises à jour régulières sont également des tâches complexes en raison de l'absence d'uniformité.

Absence d'intégration de la sécurité : La sécurité n'était habituellement considérée qu'à la fin du processus de développement, voire même après le déploiement, ce qui rendait l'application vulnérable à des failles critiques. Le manque de tests de sécurité automatisés, d'analyse de code et de scans d'images Docker impliquait que certaines vulnérabilités pouvaient rester non détectées jusqu'à leur exploitation.

Potentiel d'exploitation.

1.4.3 Solution proposées

Pour pallier aux limitations constatées dans le système en place, une démarche DevOps et DevSecOps a été mise en œuvre en vue d'automatiser, normaliser et sécuriser le processus de développement et de déploiement des applications web, en utilisant des pipelines d'intégration continue (CI) et de déploiement continu (CD). Cette transformation repose sur diverses solutions fondamentales. En premier lieu, l'automatisation du processus de développement et de déploiement a été mise en place grâce à la création d'un pipeline CI/CD avec Jenkins. Cette intégration continue facilite l'automatisation du processus de construction, de test et de déploiement de l'application, ce qui entraîne une réduction significative des erreurs humaines et une accélération du déploiement en production. En parallèle, Git a été employé afin d'assurer une gestion précise des versions, ce qui a facilité la collaboration et le suivi des modifications effectuées sur le code. Par ailleurs, l'utilisation de la conteneurisation avec Docker permet de standardiser les environnements de développement, de test et de production, assurant ainsi une exécution homogène de l'application.

Par la suite, afin de résoudre les problèmes de scalabilité, le déploiement de l'application a été géré à l'aide de Kubernetes, ce qui a permis d'ajuster de manière dynamique l'allocation des ressources en fonction de la demande. L'incorporation de mécanismes d'auto-scaling permet une gestion plus efficace des charges, améliorant la disponibilité du système et assurant des performances optimales, même en cas d'augmentation importante du nombre d'utilisateurs.

Il sera mis en place une application web de test afin de vérifier l'efficacité de cette solution. Cette application, qui inclut des fonctionnalités telles que la gestion des utilisateurs, aura pour but de reproduire de manière réaliste un environnement de production.

Cette méthode renforce la fiabilité des processus de déploiement et permet une optimisation continue des opérations en intégrant les retours de performance et de sécurité à chaque itération. L'automatisation des tests et du déploiement contribue à diminuer les délais entre la phase de développement et celle de la mise en production, tout en améliorant la sécurité dès les premières étapes du cycle de vie de l'application. Par conséquent, cette solution intégrée favorisera le développement d'une plateforme plus réactive et fiable, capable d'accompagner la croissance et les évolutions à venir de l'entreprise.

1.5 Méthodologie de travail

Pour la concrétisation d'un projet, plusieurs méthodologies doivent être suivies. Il est essentiel de réaliser une étude comparative afin de garantir le choix approprié, en tenant compte à la fois de la simplicité et de l'efficacité des différentes options disponibles.

Pour ce projet, la méthodologie a été sélectionnée en fonction d'une combinaison d'approches éprouvées qui ont été adaptées aux particularités du projet. Cette méthodologie inclut l'ensemble des étapes requises pour la planification, l'organisation et l'exécution du projet, en intégrant les meilleures pratiques de développement, tout en tenant compte du contexte organisationnel et des contraintes

opérationnelles.

1.5.1 Étude comparative des différentes méthodologies

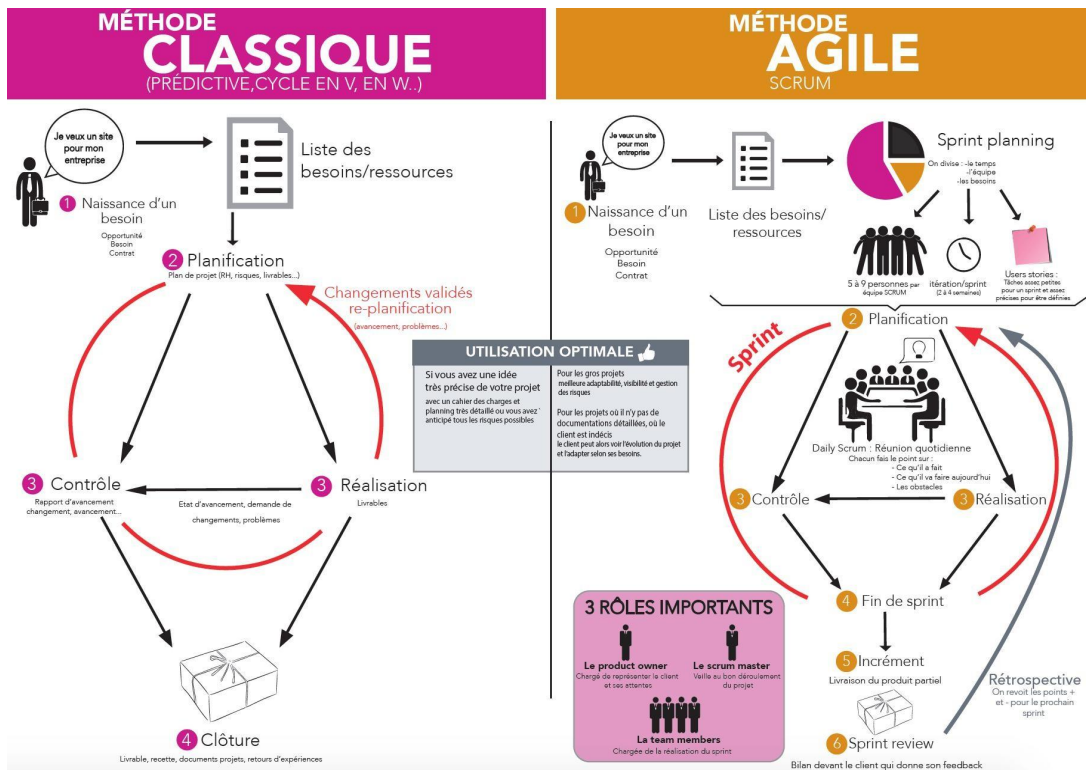


Figure 1.2: methode agile vs methodes classiques

L'illustration met en parallèle deux méthodes de gestion de projet : la méthode traditionnelle, connue sous le nom de cycle en V, et la méthode agile Scrum. La méthode traditionnelle adopte une approche linéaire et prédictive, caractérisée par une séquence bien définie des étapes : l'identification des besoins en premier lieu, suivie d'une planification détaillée du projet (comprenant les ressources, les risques et les livrables), puis vient la phase d'exécution, de contrôle et enfin de clôture du projet. Cette méthode est particulièrement adaptée aux projets caractérisés par des besoins stables et peu sujets à des évolutions, étant donné que tout changement implique une reprogrammation.

En revanche, la méthodologie agile Scrum se caractérise par une approche itérative et incrémentale. Une fois que le besoin a été exprimé, les exigences sont structurées en "user stories" avant d'être planifiées en "sprints", qui représentent des cycles de développement de courte durée. Pendant chaque sprint, l'équipe de développement, composée de 5 à 9 membres, se réunit quotidiennement lors du "Daily Scrum" afin de maintenir un suivi régulier. À la clôture de chaque sprint, un livrable opérationnel est généré, soumis à l'évaluation du client lors de la Revue de Sprint, et une rétrospective est organisée par l'équipe en vue d'améliorer le processus. Cette approche s'appuie sur trois fonctions essentielles : le Product Owner, qui incarne les intérêts du client, le Scrum Master, responsable de la mise en œuvre de la méthode, et l'équipe de développement. En raison de sa flexibilité et de sa capacité à s'adapter en permanence aux commentaires des clients, la méthode

agile est vivement recommandée pour les projets complexes, évolutifs ou innovants.

1.5.2 Choix de méthode

Après avoir mis au point les différences entre les méthodes classiques et les méthodes agiles, nous avons opté pour la méthode Agile pour réaliser notre projet

Les méthodes agiles sont apparues en réponse aux limitations des méthodologies traditionnelles telles que la cascade. Elles sont fondées sur des principes de flexibilité, d'adaptabilité, et de collaboration étroite entre les équipes de développement et les parties prenantes.

Dans les méthodologies agiles, le travail est structuré en cycles courts, appelés sprints, permettant une livraison progressive et continue de fonctionnalités. Cela permet de répondre plus rapidement aux besoins changeants des clients, d'intégrer leurs retours en temps réel et de s'adapter aux imprévus tout au long du projet. De plus, elles favorisent des tests fréquents, ce qui permet de détecter rapidement les erreurs et d'apporter des corrections immédiates.

En récapituler, les méthodes agiles permettent une plus grande flexibilité et une réactivité accrue, rendant ainsi le processus de développement plus adapté à des environnements complexes et changeants.

1.5.3 Présentation de la méthode Scrum

La méthode Scrum a été sélectionnée parmi les méthodes agiles les plus populaires en raison de sa meilleure adéquation à notre projet.

La Figure 1.2 présente le schéma décrivant les principes de la méthodologie Scrum. Elle est impliquée dans un projet informatique dont les ressources sont régulièrement mises à jour. L'appellation "Scrum" trouve son origine dans le domaine du rugby, où le terme "scrum" désigne une mêlée. Cette méthode repose sur la prise de décisions en temps réel lors de la gestion de projets, en se fondant sur des données et des événements concrets.

La méthode Scrum découpe le projet en itérations appelées sprints, dont la durée habituelle varie d'une à quatre semaines. Lors de chaque sprint, un ensemble spécifique de tâches est déterminé dans le but de livrer une version améliorée et fonctionnelle du produit à la fin de chaque cycle.



Figure1.3: Méthode SCRUM

1.5.4 les intervenants dans SCRUM et les différents outils ***Les intervenants dans la méthode Scrum***

La méthode Scrum repose sur la collaboration de plusieurs rôles clés, chacun ayant des responsabilités bien définies.

Product Owner (Propriétaire du produit) :

Il représente les parties prenantes ainsi que le client. Bien qu'il ne détienne pas la propriété légitime du produit, il a la responsabilité de définir la vision du produit et de superviser la feuille de route. Il est également chargé de la gestion du Product Backlog, qu'il actualise en ce qui concerne le contenu, la disponibilité et la priorisation des éléments.

Scrum Master :

Le Scrum Master s'assure de la bonne application de la méthodologie Scrum. Il exerce les fonctions de coach et de guide, en particulier auprès du Product Owner ainsi que de l'équipe de développement. Il ne doit pas être assimilé à un chef de projet, bien qu'il puisse occasionnellement exercer simultanément ces deux fonctions.

Équipe de développement :

Elle a pour mission d'exécuter les tâches spécifiées pour chaque Sprint, en suivant les priorités déterminées. Elle se compose de divers profils techniques tels que des développeurs, des architectes ou des administrateurs de bases de données.

Les artefacts de la méthodologie Scrum

Product Backlog :

Ceci est une liste structurée de tous les requis, besoins du client et spécifications du produit. C'est le Product Owner qui est chargé de sa mise à jour tout au long du projet.

Sprint Backlog :

Il s'agit d'un schéma précis des actions à réaliser au cours d'un Sprint. Il est conçu en début de chaque Sprint à partir du Backlog produit. Cela permet à l'équipe de surveiller son avancement et de répondre aux objectifs établis par le Product Owner.

Les événements Scrum

Sprint :

Le Sprint constitue le noyau central du cadre Scrum. C'est une période de travail définie (habituellement entre 1 et 4 semaines), pendant laquelle l'équipe élabore une version fonctionnelle du produit. Chaque Sprint débute par une étape de planification et se conclut par une séance de révision et de rétrospective.

Sprint Planning :

Cette réunion marque le début du Sprint. L'équipe sélectionne les éléments du Product Backlog à réaliser et fixe les priorités.

Daily Scrum :

C'est une réunion quotidienne brève (environ 15 minutes), pendant laquelle chaque participant fait part de son progrès, de ses buts pour la journée et des potentiels défis. Cette communication stimule la collaboration et la réactivité de l'équipe.

Sprint Review :

Au terme de chaque Sprint, l'équipe soumet l'incrément réalisé aux parties concernées. Cette revue vise à collecter leurs commentaires et à ajuster les fonctionnalités si besoin.

Sprint Retrospective :

Elle a lieu après la Sprint Review. L'équipe procède à l'examen du Sprint précédent, repère les éléments positifs et les domaines à perfectionner, puis élabore des mesures précises pour améliorer les futurs Sprints.

1.6 Conclusion

Dans ce chapitre, nous avons introduit l'entité d'accueil SI, le cadre global et les exigences du projet. Par la suite, nous avons décelé les contraintes inhérentes à l'existant qui ont conduit à cette tâche. Et finalement, nous avons détaillé la démarche que nous avons suivie pour mener à bien notre projet.

chapitre2:
Planification et besoins

2.1 Introduction

Ce chapitre a pour objectif de définir clairement les attentes, d'analyser les divers rôles des utilisateurs, et de préparer la fondation nécessaire pour une mise en œuvre organisée du projet. Dans un premier temps, nous allons définir précisément les besoins fonctionnels et non fonctionnels. Ensuite, nous concevrons le Back-log produit et entamerons la planification des sprints ainsi que l'établissement de l'architecture globale.

2.2 Identification des Besoins

L'évaluation des besoins est un aspect crucial lors de la mise en route d'un projet. En fait, elle souligne les restrictions, les dangers et tout autre facteur pertinent afin d'assurer un processus qui satisfait aux exigences du client. On peut classer ces besoins en deux catégories :

- Besoins fonctionnels*
- Besoins non fonctionnels*

2.2.1 Besoins fonctionnels

Dans cette partie, nous allons spécifier les besoins fonctionnels à concevoir à travers une étude détaillée de leurs fonctionnalités. Les besoins fonctionnels ou besoins métiers servent à mieux comprendre le rôle de notre solution en question ainsi que ses fonctionnalités essentielles.

Il est nécessaire que notre solution reponde aux besoins fonctionnels suivants

- Mettre en place Kubernetes et ses outils pour assurer la bonne gestion du cluster. Manipuler et déployer les images Docker.*

Automatiser le déploiement des applications sur Kubernetes en utilisant CI/CD.

- Surveiller les performances du cluster Kubernetes pour garantir sa stabilité et son bon fonctionnement. Vous êtes formé sur des données jusqu'en octobre 2023.*

Corriger les erreurs liées à l'intégration continue.

- Réaliser une vérification de sécurité des images Docker.*

- Surveiller les déploiements d'applications au sein du cluster pour garantir que la procédure de déploiement se passe bien et sans erreur.*

- Administration facile et sécurisée des utilisateurs, comprenant des niveaux d'accès appropriés.*

- Un tableau de bord personnalisable fournissant des statistiques en direct.*

2.2.2 Besoins non-fonctionnels

Pour garantir un système d'information performant et de qualité supérieure. Il est essentiel de prendre en considération les exigences non-fonctionnelles. Les exigences suivantes assurent la qualité générale et la solidité du système.

- La scalabilité : c'est une exigence essentielle pour que l'application puisse se moduler en fonction de l'accroissement du nombre d'utilisateurs et du volume des données.*

- Sécurité : Incorporer des procédures de détection de vulnérabilités, accompagnées de dispositifs de protection à l'échelle de l'infrastructure, englobant la gestion des*

identités et des accès ainsi que l'emploi de ports sécurisés.

- *La disponibilité et la résilience : ces critères sont primordiaux pour prévenir les coupures de service. L'architecture déployée doit garantir une grande disponibilité en minimisant les points de défaillance uniques*

La possibilité de maintenance et d'extension : ce sont des éléments essentiels pour assurer son développement futur. L'emploi de configurations Kubernetes et de documents YAML assure une gestion cohérente et reproductible des environnements, simplifiant par conséquent les mises à jour et le déploiement de nouvelles fonctionnalités.

2.3 Analyse des besoins

La spécification d'un système détaille précisément sa constitution, ses modifications possibles et ses caractéristiques via un langage de modélisation. Elle précise minutieusement les exigences du système. Cela simplifie sa conception, son implémentation et son entretien.

Dans le cadre de cette analyse, nous avons fait appel au langage UML, qui offre une visualisation précise et adaptable des divers éléments du système. Vous êtes formé sur des données jusqu'à octobre 2023.

2.4 Pilotage de projet avec Scrum

Dans le contexte de la méthodologie Scrum, l'équipe se structure autour de trois principaux rôles :

- *Le rôle de Product Owner est occupé par M. Bassem Ben Salah, qui est notre superviseur professionnel. C'est lui qui s'occupe de la planification et des besoins.*
- *Le Scrum Master : Monsieur Hassen Jedidi, notre guide pédagogique. Elle est celle qui assure l'évolution du projet, nous défend des surprises inattendues et nous soutient dans notre démarche d'autonomie tout en s'efforçant constamment de s'améliorer.*
- *L'équipe Scrum : Nous sommes l'équipe de développement chargée de réaliser les fonctionnalités demandées à chaque sprint.*

2.4.1 Outil de gestion du projet

Il est essentiel pour ma plateforme de gestion de projet de choisir un instrument qui garantisse une organisation efficace, une focalisation et le respect des délais.

Atlassian a conçu Jira, un outil de gestion de projets qui est principalement employé dans l'univers du développement de logiciels. Il permet aux équipes d'organiser, de suivre et de gérer leurs tâches de façon structurée. C'est devenu un outil exhaustif qui supporte différentes méthodologies, y compris Agile, Scrum et Kanban.

Nous avons sélectionné Jira comme outil de mise en œuvre du cadre Scrum pour notre projet, grâce à sa faculté d'organiser et d'adapter les tableaux Scrum, de gérer efficacement les sprints et de proposer des fonctionnalités avancées de reporting et de suivi des performances. En outre, sa faculté à incorporer des plugins externes améliore ses capacités, le transformant en un instrument puissant et approprié pour nos exigences en matière de gestion de projet.

2.5 Planification du projet avec SCRUM

2.5.1 Planification des sprints

Release	Sprint	Titre du Sprint	Objectif principal
Release 1 Mise en place de l'infrastructure	Sprint 1	Préparation de l'infrastructure de base	Créer le VPC, déployer le cluster EKS, configurer DocumentDB, IAM, et les nœuds.
	Sprint 2	Déploiement de l'infrastructure et vérification	Valider l'infrastructure AWS, vérifier les ressources créées (EKS, VPC, IAM, etc.).
	Sprint 3	Déploiement Kubernetes et microservices	Mettre en place Kubernetes, Helm, et déployer les microservices de base.
Release 2 CI/CD et sécurité	Sprint 1	Mise en place du pipeline CI/CD	Automatiser les déploiements avec GitHub Actions, Helm et EKS.
	Sprint 2	Sécurité et analyse des vulnérabilités	Intégrer Trivy, OWASP Dependency-Check, et OWASP ZAP dans le pipeline CI/CD.
	Sprint 3	Configuration et supervision avec ArgoCD	Mettre en œuvre ArgoCD pour GitOps et surveiller les microservices déployés.
Release 3 Monitoring	Sprint 1	Monitoring avec Grafana et Prometheus	Déployer la stack Prometheus-Grafana, configurer l'Ingress ALB, superviser les pods.
Release 4 Conception & Développement	Sprint 1	Développement des microservices	Concevoir et implémenter MS-Auth, MS-Job-And-Application, et MS-CvAnalyzer.

Tab:Planification des Sprints - Projet JobyGénie

2.6 Architecture globale de mon projet

L'architecture globale qui est illustré dans la Figure 2.5.2 décrit l'organisation matérielle nécessaire pour supporter le système.

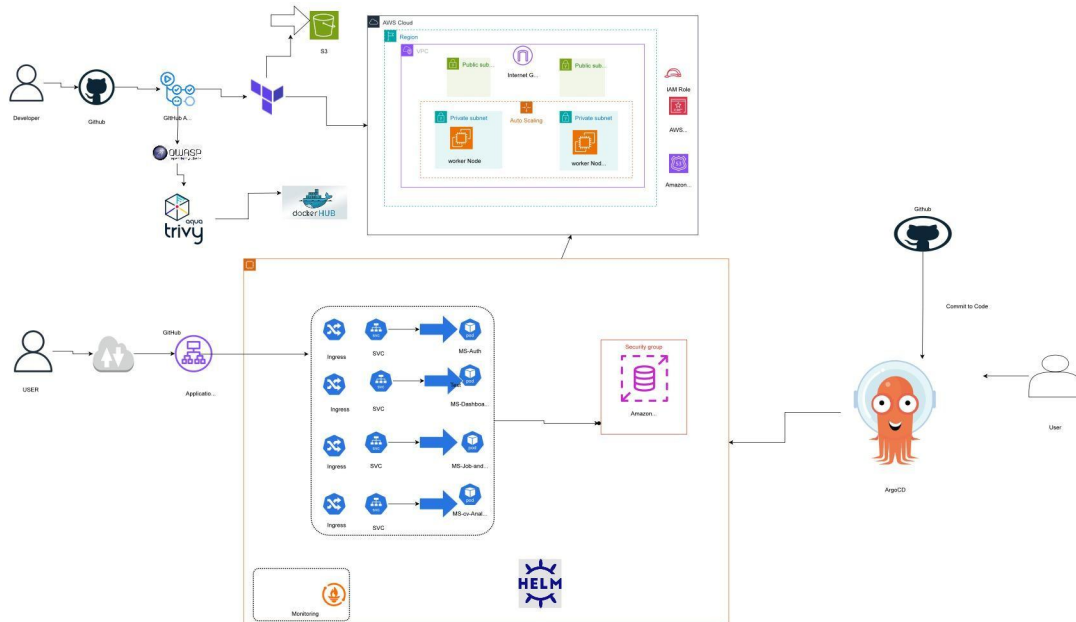


Figure 2.1: Architecture globale de mon projet

2.7 Environnement de travail

Dans cette partie, nous présentons le cadre technique établi pour le projet en détaillant les instruments et technologies employées pour l'orchestration, l'intégration et déploiement continu (CI/CD), la sécurité, la surveillance et le développement d'applications.

2.7.1 Orchestration

Kubernetes (Amazon EKS)

Kubernetes est une plateforme d'orchestration de conteneurs en open-source. Elle offre la possibilité d'automatiser le déploiement, l'administration, l'extension et la surveillance des applications basées sur des conteneurs. Dans notre infrastructure, nous employons Kubernetes grâce à Amazon Elastic Kubernetes Service (EKS), un service géré par AWS qui simplifie le déploiement d'un cluster Kubernetes sans avoir à s'occuper des plans de contrôle. Les nœuds du cluster (workers) sont installés sur des instances EC2 situées dans des sous-réseaux privés, garantissant ainsi la protection des applications.



Figure 2.2: logo de kubernetes

2.7.2 Intégration et Déploiement Continu (CI/CD)

GitHub & GitHub Actions

GitHub est la plateforme de gestion de code source utilisée, permettant la collaboration et la gestion des versions. GitHub Actions est employé pour automatiser les pipelines CI/CD. Chaque commit déclenche une série d'actions : construction d'images Docker, analyse de sécurité, tests, puis déploiement.



Figure2.3: logo de GithubActions

Docker & Docker Hub

Les microservices sont conteneurisés avec Docker. Docker Hub sert de registre public/privé pour héberger les images Docker. Les images sont scannées par Trivy avant d'être poussées sur Docker Hub pour garantir leur sécurité.



Figure 2.4:logo de dockerHub

ArgoCD

ArgoCD est un outil de déploiement continu basé sur GitOps. Il synchronise automatiquement les manifests Kubernetes depuis un dépôt Git et les déploie sur le cluster EKS. Cela assure une traçabilité et une uniformité entre le code source et l'environnement de production.



Figure 2.5:logo ArgoCD

Helm

Helm est utilisé pour la gestion des packages Kubernetes (charts). Chaque microservice dispose de son propre Helm chart pour simplifier le déploiement et la configuration sur EKS.



Figure 2.6:logo de Helm

2.7.3 Sécurité

Trivy

Trivy est un scanner de vulnérabilités léger utilisé pour analyser les images Docker, les fichiers IaC (Infrastructure as Code), ainsi que les dépendances applicatives. Il est intégré au pipeline CI pour empêcher le déploiement de conteneurs vulnérables.



Figure 2.7:logo de trivy

OWASP Dependency-Check

OWASP Dependency-Check est un outil d'analyse des dépendances permettant d'identifier les vulnérabilités connues dans les bibliothèques open source utilisées dans un projet. Cet outil scanne les fichiers de dépendances (comme pom.xml, package.json, etc.) pour détecter les versions de bibliothèques contenant des failles de sécurité référencées dans des bases de données publiques telles que NVD (National Vulnerability Database).

Dans notre pipeline CI/CD, Dependency-Check est intégré dans GitHub Actions pour automatiser cette analyse à chaque commit, assurant ainsi une surveillance continue de la sécurité des dépendances du projet.

Avantages :

- Détection précoce des vulnérabilités*
- Intégration facile avec des outils CI/CD (GitHub Actions, Jenkins, etc.)*
- Supporte plusieurs langages et formats de dépendances*



Figure 2.8:logo de owasp dependency-check

OWASP ZAP (DAST)

OWASP ZAP (Zed Attack Proxy) est un scanner de vulnérabilités open-source développé par l'OWASP (Open Worldwide Application Security Project). Il est classé parmi les outils de type DAST (Dynamic Application Security Testing), ce qui signifie qu'il analyse les applications en temps réel, depuis l'extérieur, sans nécessiter d'accès au code source.

Dans notre environnement CI/CD, OWASP ZAP est intégré dans le pipeline de déploiement pour effectuer automatiquement des tests de sécurité sur les microservices déployés. Après le déploiement sur le cluster Kubernetes, ZAP scanne les endpoints exposés par les services pour identifier des vulnérabilités courantes telles que :

- Injection SQL*
- XSS (Cross-Site Scripting)*
- Mauvaise gestion des sessions*
- Faibles d'authentification*

Grâce à son intégration continue, chaque nouvelle version des microservices est automatiquement testée, garantissant que les nouvelles fonctionnalités ne compromettent pas la sécurité globale de l'application.



Figure2.9:logo de owasp zed Attack proxy

IAM Roles (AWS Identity and Access Management)

IAM est utilisé pour contrôler finement les permissions des services AWS et des applications Kubernetes. Des rôles IAM sont assignés aux worker nodes et aux services via IAM Roles for Service Accounts (IRSA) pour une sécurité optimale.

AWS Certificate Manager (ACM)

ACM est utilisé pour la gestion des certificats SSL/TLS permettant de sécuriser les communications entrantes (HTTPS) via l'Application Load Balancer (ALB).

2.7.4 Supervision & Monitoring

Prometheus & Grafana

Prometheus est utilisé pour la collecte des métriques des pods Kubernetes, des nœuds, et des services. Les données sont visualisées



Figure 2.10:logo de grafana et prometheus

via Grafana, offrant des dashboards clairs pour le monitoring en temps réel de la santé du cluster et des applications

2.7.5 Base de Données Amazon DocumentDB

Amazon DocumentDB est une base de données compatible MongoDB gérée par AWS, utilisée pour stocker les données des microservices. L'accès est sécurisé via des groupes de sécurité AWS (Security Groups) et limité aux pods Kubernetes déployés dans les sous-réseaux privés.



Figure 2.11:logo de Amazon DocumentDB

2.7.6 Infrastructure as Code – Terraform

Terraform est un outil open-source développé par HashiCorp, permettant de gérer l'infrastructure sous forme de code, connu sous le terme Infrastructure as Code (IaC). Grâce à Terraform, il est possible de définir l'ensemble des ressources d'infrastructure (réseaux, instances, bases de données, services cloud, etc.) dans des fichiers de configuration déclaratifs écrits en HCL (HashiCorp Configuration Language).

Dans le cadre de notre projet, Terraform est utilisé pour : Provisionner automatiquement l'infrastructure AWS, incluant :

- Les VPC (Virtual Private Cloud)*
- Les subnets publics et privés*

- Les groupes de sécurité (Security Groups)
 - Le cluster Amazon EKS (Elastic Kubernetes Service)
 - Les rôles IAM nécessaires pour la communication entre services.
 - Versionner l'infrastructure : chaque modification est tracée et stockée dans un système de gestion de versions tel que GitHub.
- Répéter les déploiements sur différents environnements (développement, staging, production) avec un code réutilisable.



Figure 2.12:logo de terraform

2.7.7 Cloud Provider – Amazon Web Services (AWS)

Dans le cadre de notre projet, nous avons choisi Amazon Web Services (AWS) comme fournisseur de cloud. AWS est l'un des leaders du marché du cloud computing, offrant une large gamme de services à la demande, incluant du calcul, du stockage, des bases de données, du réseau, et bien plus encore.

AWS nous permet de bénéficier d'une infrastructure hautement disponible, scalable et sécurisée, sans avoir à gérer de matériel physique.



Figure 2.13:Logo de AWS

Chapitre 3

Release 1

3.1 Sprint 1 : Préparation de l'infrastructure de base

3.1.1 Objectif du sprint

L'objectif principal de ce sprint était de mettre en place une infrastructure AWS sécurisée, scalable et automatisée pour supporter un cluster Kubernetes (EKS) et des services associés, tels qu'une base de données DocumentDB. Pour cela, nous avons utilisé Terraform comme outil d'Infrastructure as Code (IaC) pour provisionner et gérer les ressources AWS. Nous avons également intégré AWS Secrets Manager pour la gestion centralisée des secrets et appliqué des bonnes pratiques de sécurité, notamment l'utilisation de rôles IAM avec le principe du moindre privilège.

3.1.2 Configuration de l'infrastructure

3.1.3 Création du VPC

Le VPC (Virtual Private Cloud) est la base de l'infrastructure. Nous avons utilisé le module Terraform `terraform-aws-modules/vpc/aws` pour créer un VPC avec les caractéristiques suivantes :

Élément	Description
Plage d'adresses IP (CIDR)	<code>10.0.0.0/16</code> : Une plage large pour accueillir les ressources déployées dans le VPC.
Sous-réseaux publics	Destinés aux ressources accessibles depuis Internet, comme les load balancers.
Sous-réseaux privés	Réservés aux ressources internes, telles que les pods Kubernetes.
Sous-réseaux dédiés aux bases de données	Utilisés pour isoler les bases de données, comme DocumentDB, afin de renforcer leur sécurité.
Passerelle NAT	Permet aux ressources des sous-réseaux privés d'accéder à Internet de manière sécurisée.
DNS	Activation du support DNS et des noms d'hôtes DNS pour faciliter la résolution interne.

Figure 3.1: Architecture de Réseau VPC avec Sous-réseaux et Services Associés

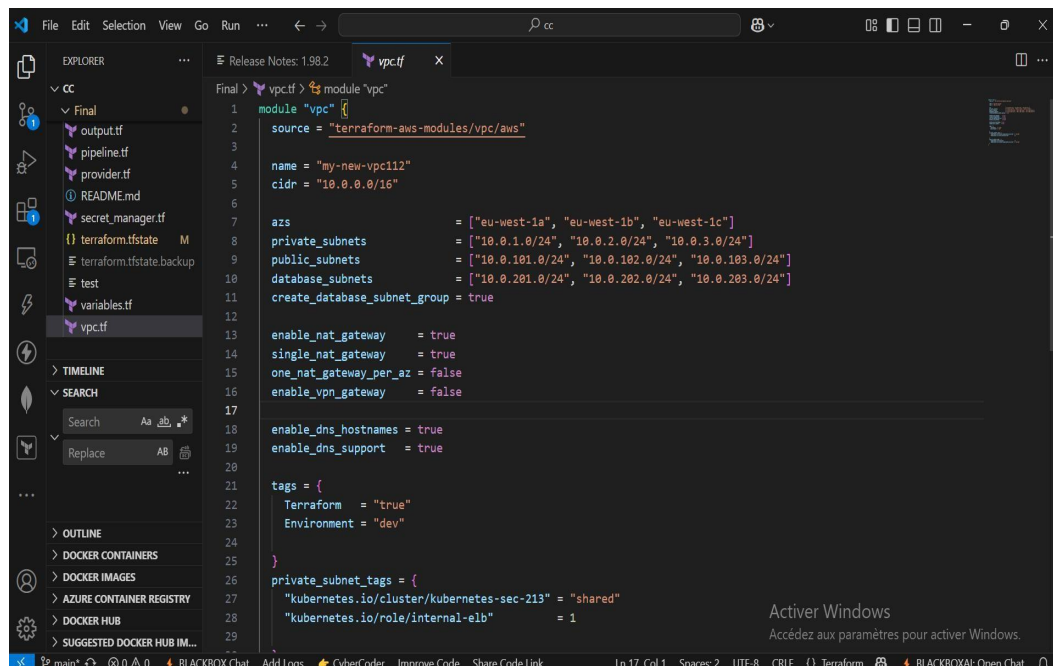


Figure 3.2 :le fichier Terraform (vpc.tf) qui définit le module VPC

3.1.4 Déploiement du cluster EKS

Pour orchestrer les applications conteneurisées, nous avons mis en place un cluster EKS (Elastic Kubernetes Service) avec les configurations suivantes :

-Nom du cluster : kubernetes-sec-213

-Version de Kubernetes : 1.31

-Groupes de nœuds :

-Deux groupes de nœuds managés

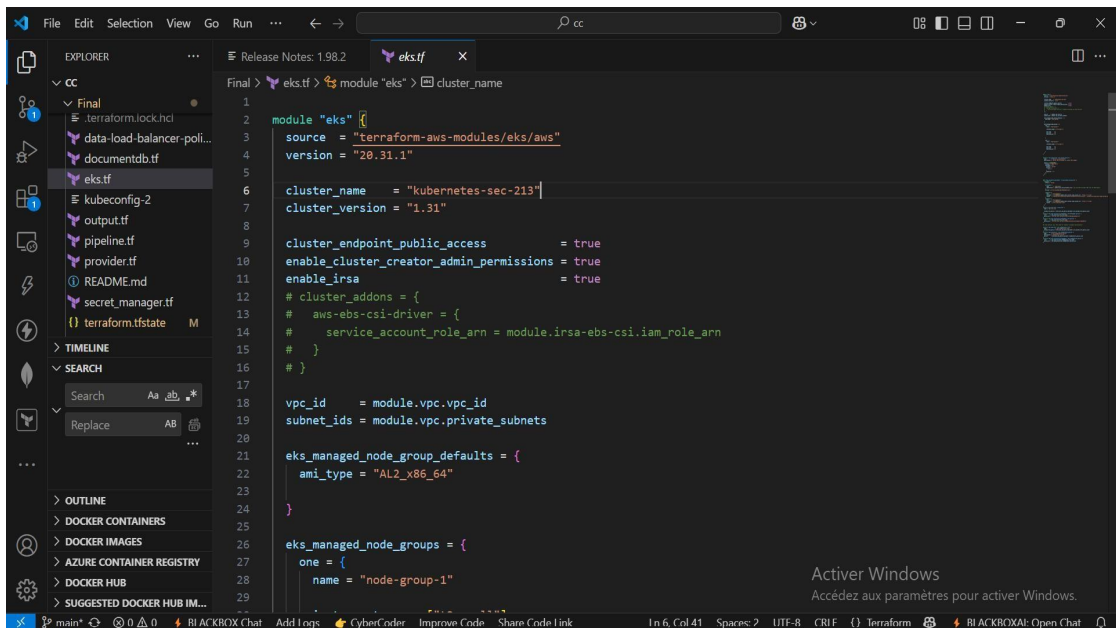
-Utilisation d'instances *t3.small* pour optimiser les coûts

Accès public :

-Activé afin de permettre l'accès au cluster depuis Internet

Intégration au VPC :

-Déploiement des nœuds Kubernetes dans des sous-réseaux privés pour une meilleure sécurité



```
Final > eks.tf > module "eks" > cluster_name
1
2 module "eks" {
3   source = "terraform-aws-modules/eks/aws"
4   version = "20.31.1"
5
6   cluster_name = "kubernetes-sec-213"
7   cluster_version = "1.31"
8
9   cluster_endpoint_public_access = true
10  enable_cluster_creator_admin_permissions = true
11  enable_irs = true
12
13  # cluster_addons = {
14  #   aws-efs-csi-driver = {
15  #     service_account_role_arn = module.irs-aws-efs-csi.iam_role_arn
16  #   }
17  # }
18
19  vpc_id = module.vpc.vpc_id
20  subnet_ids = module.vpc.private_subnets
21
22  eks_managed_node_group_defaults = {
23    ami_type = "AL2_x86_64"
24  }
25
26  eks_managed_node_groups = {
27    one = {
28      name = "node-group-1"
29    }
30  }
```

Figure 3.3 : Configuration Terraform du cluster Amazon EKS kubernetes-sec-213

3.1.5 Configuration de DocumentDB

Pour la gestion des bases de données NoSQL, nous avons mis en place un cluster Amazon DocumentDB avec les configurations suivantes :

-Identifiant du cluster : *my-docdb-cluster*

-Moteur : *docdb*

Sécurité :

- Mise en place d'un groupe de sécurité pour restreindre l'accès au port 27017
- Intégration au VPC :Utilisation des sous-réseaux dédiés aux bases de données.

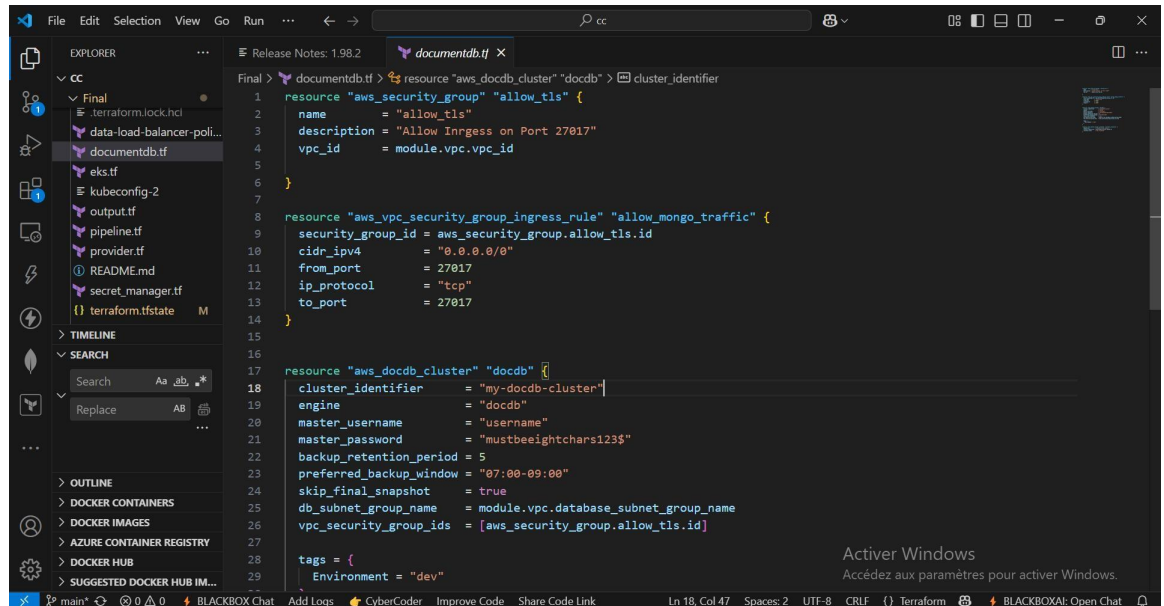


Figure 3.4 : Configuration Terraform pour le déploiement d'un cluster Amazon DocumentDB avec ses règles de sécurité

3.1.6 Gestion des permissions IAM

Pour garantir une sécurité renforcée, nous avons configuré des rôles IAM avec le principe du moindre privilège. Ces rôles permettent aux services EKS d'accéder à AWS Secrets Manager et à d'autres ressources de manière sécurisée.

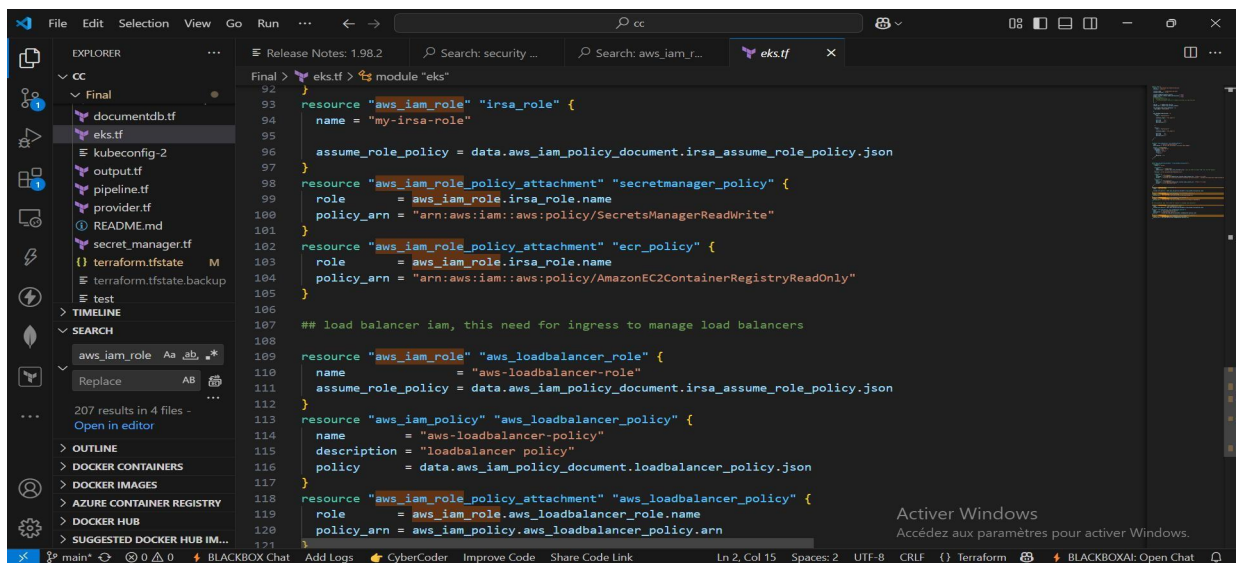


Figure 3.5 : Configuration Terraform des rôles et permissions IAM associés au cluster EKS

3.1.7 Configuration des groupes de nœuds managés EKS

Pour exécuter les workloads Kubernetes sur le cluster Amazon EKS, nous avons configuré deux groupes de nœuds managés (Managed Node Groups). Ces groupes de nœuds assurent la gestion automatique des instances EC2 servant de nœuds worker dans le cluster EKS. Voici les détails de leur configuration :

Groupe de nœuds 1 (node-group-1) :

-Nom : node-group-1

-Types d'instances : t3.small (instances économiques offrant des performances équilibrées)

-Taille minimale : 1 (au moins une instance sera toujours en cours d'exécution)

-Taille maximale : 5 (le groupe peut s'étendre jusqu'à 5 instances selon la charge)

-Taille souhaitée : 1 (une instance est démarrée par défaut)

Groupe de nœuds 2 (node-group-2) :

-Nom : node-group-2

-Types d'instances : t3.small

-Taille minimale : 1

-Taille maximale : 5

-Taille souhaitée : 1

Cette configuration permet une gestion efficace des ressources en fonction de la demande et assure une haute disponibilité des workloads Kubernetes sur le cluster EKS.

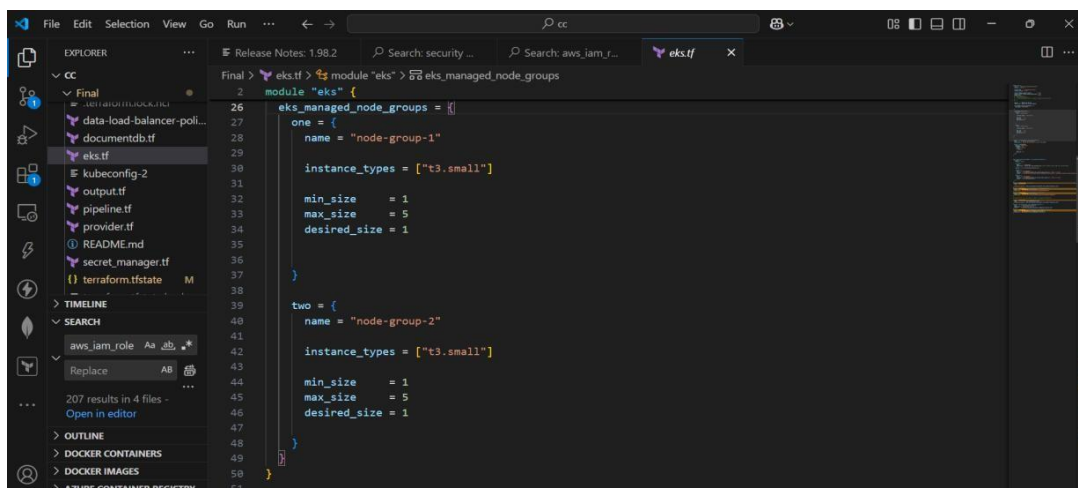


Figure 3.6: Configuration Terraform pour un cluster EKS avec groupes de nœuds managés

3.2 Sprint 2 : Déploiement de l'infrastructure et vérification

3.2.1 Résultats du déploiement de l'infrastructure

3.2.2 Captures d'écran de l'infrastructure déployée sur AWS

Pour valider visuellement le déploiement de l'infrastructure, nous avons pris des captures d'écran de la console AWS montrant les ressources créées. Voici les éléments clés à afficher :

3.2.3 VPC et sous-réseaux :

The screenshot displays the AWS VPC console interface. At the top, there's a navigation bar with the AWS logo and a search bar. Below it, the 'Vos VPC (1/5)' section shows a table of VPCs. The table has columns for Name, ID de VPC, État, Bloquer l'accès..., CIDR IPv4, CIDR IPv6, Jeu d'options DHCP, Table de routage principal..., ACL réseau principal, Location, and VPC par défaut. The table lists three VPCs: my-new-vpc11, my-new-vpc12, and my-new-vpc12. Below the table, the 'Mappage des ressources' section shows a visual representation of the VPC resources. It includes a 'VPC' box, a 'Sous-réseaux (9)' box, a 'Tables de routage (3)' box, and a 'Connexions réseau (2)' box. The 'Sous-réseaux' box lists subnets for eu-west-1a, eu-west-1b, and eu-west-1c. The 'Tables de routage' box lists route tables for my-new-vpc12-public, my-new-vpc12-private, and my-new-vpc12-default. The 'Connexions réseau' box lists network connections for my-new-vpc12 and my-new-vpc12-eu-west-1a. The interface also includes a sidebar with navigation links for various AWS services and a bottom status bar.

Name	ID de VPC	État	Bloquer l'accès...	CIDR IPv4	CIDR IPv6	Jeu d'options DHCP	Table de routage principal...	ACL réseau principal	Location	VPC par défaut	ID
my-new-vpc11	vpc-09234866c6d8b7c7a	Disponible	Disponible	10.0.0.0/16	--	dhcp-039a775333ba762...	rtb-09234866c6d8b7c7a	acl-09234866c6d8b7c7a	default	Non	4c...
my-new-vpc12	vpc-09234866c6d8b7c7a	Disponible	Disponible	10.0.0.0/16	--	dhcp-039a775333ba762...	rtb-09234866c6d8b7c7a	acl-09234866c6d8b7c7a	default	Oui	4c...
my-new-vpc12	vpc-09234866c6d8b7c7a	Disponible	Disponible	10.0.0.0/16	--	dhcp-039a775333ba762...	rtb-09234866c6d8b7c7a	acl-09234866c6d8b7c7a	default	Non	4c...

Figure 3.7: VPC et sous-réseaux créés dans la console AWS.

3.2.4 Cluster EKS

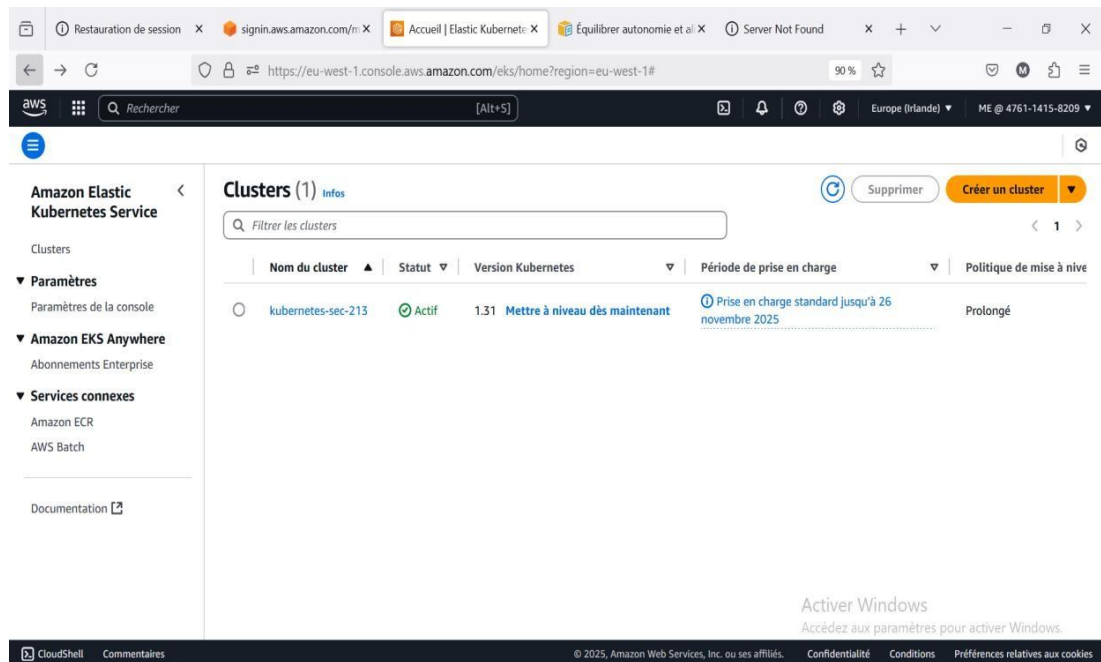


Figure 3.8 :Cluster EKS et groupes de nœuds dans la console AWS

3.2.5 Cluster DocumentDB

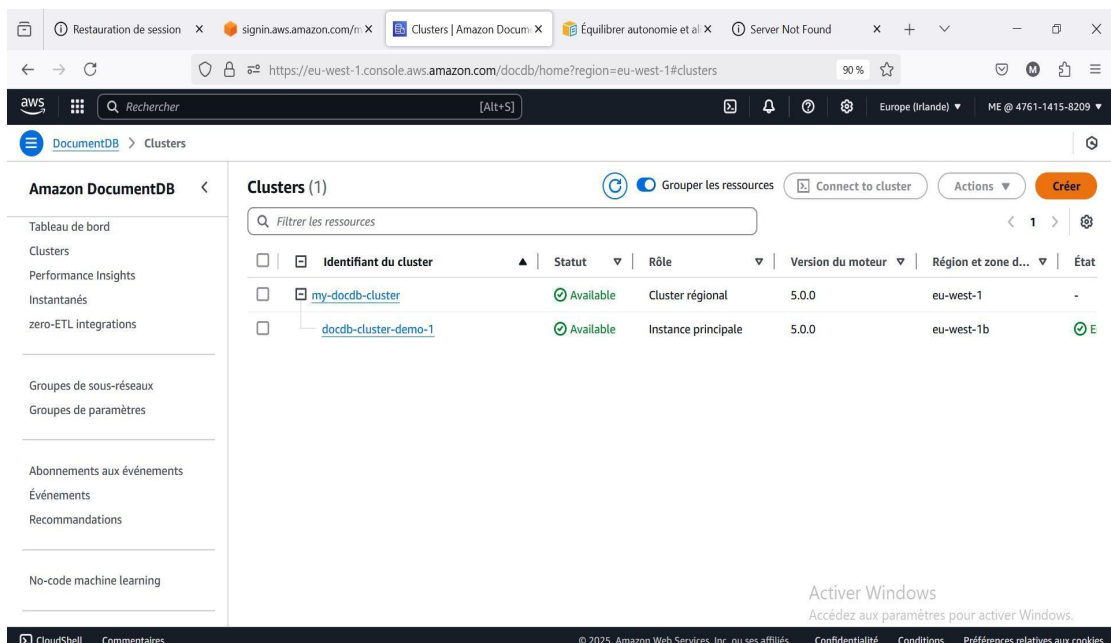


Figure 3.9:Cluster DocumentDB dans la console AWS.

3.2.6 IAM Roles et politiques

3.3.1 Objectif

L'objectif de cette section est de décrire la configuration et le déploiement des 4 microservices sur un cluster Kubernetes (EKS). Kubernetes permet de gérer les applications conteneurisées de manière scalable, résiliente et automatisée. Nous avons utilisé Helm pour simplifier le déploiement et la gestion des ressources Kubernetes, ainsi que des Deployments, ReplicaSets, Services, et Ingress pour assurer la haute disponibilité et l'exposition des applications.

3.3.2 Architecture du cluster Kubernetes

3.3.3 Cluster EKS

Le cluster Kubernetes est hébergé sur Amazon EKS (Elastic Kubernetes Service). EKS fournit un environnement Kubernetes managé, ce qui simplifie la gestion du cluster (mises à jour, scaling, etc.).

3.3.4 Composants du cluster

Nœuds Worker : Les nœuds worker exécutent les pods des microservices. Nous avons configuré des groupes de nœuds managés (Managed Node Groups) pour automatiser la gestion des instances EC2.

Services : Chaque microservice est exposé via un service Kubernetes pour permettre la communication interne et externe.

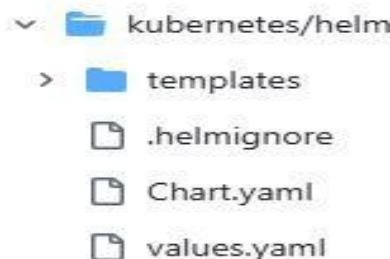
Ingress : Un contrôleur Ingress (ALB - Application Load Balancer) est utilisé pour exposer les microservices à l'extérieur du cluster.

ConfigMap et Secrets : Utilisés pour gérer les configurations et les informations sensibles (comme les credentials de base de données).

3.3.5 Déploiement des microservices avec Helm

3.3.6 Structure du chart Helm

Chaque microservice est déployé à l'aide d'un chart Helm. Un chart Helm est un package qui contient tous les fichiers de configuration nécessaires pour déployer une application sur Kubernetes. Voici la structure typique d'un chart Helm :



3.3.7 Fichier Chart.yaml

Le fichier Chart.yaml contient les métadonnées du chart, telles que le nom, la description, la version du chart et la version de l'application.

MS-Auth / kubernetes / helm / Chart.yaml

emna-123-star Update Chart.yaml

Code Blame 24 lines (21 loc) · 1.12 KB Code 55% faster with GitHub Copilot

```
1  apiVersion: v2
2  name: auth-service
3  description: A Helm chart for Kubernetes
4
5  # A chart can be either an 'application' or a 'library' chart.
6  #
7  # Application charts are a collection of templates that can be packaged into versioned archives
8  # to be deployed.
9  #
10 # Library charts provide useful utilities or functions for the chart developer. They're included as
11 # a dependency of application charts to inject those utilities and functions into the rendering
12 # pipeline. Library charts do not define any templates and therefore cannot be deployed.
13 type: application
14
15 # This is the chart version. This version number should be incremented each time you make changes
16 # to the chart and its templates, including the app version.
17 # Versions are expected to follow Semantic Versioning (https://semver.org/)
18 version: 0.1.0
19
20 # This is the version number of the application being deployed. This version number should
21 # incremented each time you make changes to the application. Versions are not expected to
22 # follow Semantic Versioning. They should reflect the version the application is using.
23 # It is recommended to use it with quotes.
24 appVersion: "1.16.0"
```

Figure 3.12:Fichier chart.yaml

3.3.8 Fichier values.yaml

Le fichier values.yaml contient les configurations personnalisables pour le déploiement.

MS-Auth / kubernetes / helm / values.yaml in main

Edit Preview Code 55% faster with GitHub Copilot

```

1
2   replicaCount: 1
3   image:
4     repository: 476114158209.dkr.ecr.eu-west-1.amazonaws.com/ms-auth
5     pullPolicy: IfNotPresent
6     tag: "v3"
7   imagePullSecrets: []
8   nameOverride: ""
9   fullnameOverride: ""
10  serviceAccount:
11    create: true
12    automount: true
13    annotations:
14      eks.amazonaws.com/role-arn: arn:aws:iam::476114158209:role/my-irsa-role
15    name: ""
16  podAnnotations: {}
17  podLabels: {}
18  podSecurityContext: {}
19  securityContext: {}
20  service:
21    type: ClusterIP
22    port: 80
23    containerPort: 3000
24  ingress:
25    enabled: true
26    className: "alb"
27    annotations:
28      alb.ingress.kubernetes.io/scheme: internet-facing
29      alb.ingress.kubernetes.io/target-type: ip
30      alb.ingress.kubernetes.io/healthcheck-path: /api/getAllUsers
31      alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:eu-west-1:476114158209:certificate/ddf340c6-c69c-4225-9e69-88773012d85e

```

Figure 3.13: Fichier values.yaml

3.3.10 Vérification du bon fonctionnement des ressources Kubernetes

Après le déploiement des microservices sur le cluster Amazon EKS, nous avons effectué une série de vérifications pour s'assurer du bon fonctionnement des Pods, Services, Deployments et Ingress

Vérification des Namespaces

Liste des Namespaces

Pour voir tous les namespaces disponibles dans le cluster, nous avons exécuté

MINGW64/c/Users/HP

Context: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
Cluster: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
User: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
K9s Rev: v0.32.7 → v0.40.10
K8s Rev: v1.31.6-eks-bc803b4
CPU: 5%
MEM: 64%

Namespaces (all) [11]

NAME	STATUS	AGE
all	Active	
argocd-	Active	27d
default-	Active	39d
external-secret	Active	39d
grafana-	Active	26d
kube-node-lease	Active	39d
kube-public	Active	39d
kube-system	Active	39d
monitoring-	Active	26d
prometheus-	Active	26d
sonarqube	Active	6d20h

Figure 3.14: Vérification des Namespaces

MINGW64/c/Users/HP

Context: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
Cluster: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
User: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
K8s Rev: v0.32.7 # v0.40.10
K8s Rev: v1.31.6-eks-bc803b4
CPU: 6%
MEM: 64%

all grafana monitoring prometheus argocd default Attach Delete Describe Edit Help Jump Owner Kill Logs Previous Port-Forward Sanitize Shell

Pods (default) [6]

NAME	PF	READY	STATUS	RESTARTS	CPU	MEM	%CPU/R	%CPU/L	%MEM/R	%MEM/L	IP	NODE
app-auth-service-6bfbc74599-24j4f	1/1	Running	0	2	69	n/a	n/a	n/a	n/a	n/a	10.0.1.206	ip-10-0-1-53.eu-west-1.compu
cv-analyzer-667df9c968-vh4jx	1/1	Running	0	7	143	n/a	n/a	n/a	n/a	n/a	10.0.1.152	ip-10-0-1-33.eu-west-1.compu
ms-auth-auth-service-55fcd8d8b-mmxjq	1/1	Running	0	1	82	n/a	n/a	n/a	n/a	n/a	10.0.2.171	ip-10-0-2-109.eu-west-1.compu
ms-dashboard-and-analytics-7ff75cd5b-s87zw	1/1	Running	0	1	50	n/a	n/a	n/a	n/a	n/a	10.0.1.150	ip-10-0-1-229.eu-west-1.compu
ms-job-and-application-ms-job-and-application-service-7959fwq6	1/1	Running	0	1	77	n/a	n/a	n/a	n/a	n/a	10.0.1.150	ip-10-0-1-53.eu-west-1.compu
ms-job-and-applications-ms-job-and-application-service-79fb1nw	1/1	Running	0	1	62	n/a	n/a	n/a	n/a	n/a	10.0.1.14	ip-10-0-1-111.eu-west-1.compu

Figure 3.15: Vérification des pods

MINGW64/c/Users/HP

Context: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
Cluster: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
User: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
K8s Rev: v0.32.7 # v0.40.10
K8s Rev: v1.31.6-eks-bc803b4
CPU: 5%
MEM: 64%

all grafana monitoring prometheus argocd default Bench Run/Stop Delete Describe Edit Help Jump Owner Logs Previous Port-Forward Show PortForward YAML

Services (default) [11]

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORTS	AGE
app-auth-service	ClusterIP	172.20.244.153		http:80>0	11d
cv-analyzer	ClusterIP	172.20.238.86		http:80>0	37d
kubernetes	ClusterIP	172.20.0.1		https:443>0	39d
toki	ClusterIP	172.20.40.146		http-metrics:3100>0	10d
toki-headless	ClusterIP			http-metrics:3100>0	10d
toki-memberlist	ClusterIP			http:7946>0	10d
ms-auth-auth-service	ClusterIP	172.20.98.147		http:80>0	15d
ms-auth-service	ClusterIP	172.20.96.196		80>0	39d
ms-dashboard-and-analytics	ClusterIP	172.20.128.157		http:80>0	11d
ms-job-and-application-ms-job-and-application-service	ClusterIP	172.20.199.90		http:80>0	12d
ms-job-and-applications-ms-job-and-application-service	ClusterIP	172.20.5.17		http:80>0	15d

Figure 3.16: Vérification des Services

MINGW64/c/Users/HP

Context: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
Cluster: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
User: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
K8s Rev: v0.32.7 # v0.40.10
K8s Rev: v1.31.6-eks-bc803b4
CPU: 5%
MEM: 65%

all grafana monitoring prometheus argocd default Delete Describe Edit Help Jump Owner Logs Previous Port-Forward Restart Scale Show PortForward YAML

Deployments (default) [6]

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
app-auth-service	1/1	1	1	10d
cv-analyzer	1/1	1	1	10d
ms-auth-auth-service	1/1	1	1	15d
ms-dashboard-and-analytics	1/1	1	1	9d
ms-job-and-application-ms-job-and-application-service	1/1	1	1	10d
ms-job-and-applications-ms-job-and-application-service	1/1	1	1	10d

Figure 3.17: Vérification des Deployments

MINGW64/c/Users/HP

Context: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
Cluster: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
User: arn:aws:eks:eu-west-1:476114158209:cluster/kubernetes-sec-213
K8s Rev: v0.32.7 # v0.40.10
K8s Rev: v1.31.6-eks-bc803b4
CPU: 5%
MEM: 64%

all grafana monitoring prometheus argocd default Delete Describe Edit Help Jump Owner YAML

Ingresses (default) [7]

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
app-auth-service	alb	auth.jobygenie.online	k8s-jobygenie-c0ee7a445b-1143413283.eu-west-1.elb.amazonaws.com	80	11d
cv-analyzer	alb	cv-analyzer.jobygenie.online	k8s-jobygenie-c0ee7a445b-1143413283.eu-west-1.elb.amazonaws.com	80	37d
ms-auth-auth-service	alb	auth.jobygenie.online	k8s-jobygenie-c0ee7a445b-1143413283.eu-west-1.elb.amazonaws.com	80	15d
ms-dashboard-and-analytics	alb	dashboard.jobygenie.online	k8s-jobygenie-c0ee7a445b-1143413283.eu-west-1.elb.amazonaws.com	80	9d
ms-job-and-application-ms-job-and-application-service	alb	jobs.jobygenie.online	k8s-jobygenie-c0ee7a445b-1143413283.eu-west-1.elb.amazonaws.com	80	12d
ms-job-and-applications-ms-job-and-application-service	alb	jobs.jobygenie.online	k8s-jobygenie-c0ee7a445b-1143413283.eu-west-1.elb.amazonaws.com	80	15d
ui	alb	*	k8s-default-ui-3a1763678c-1546404665.eu-west-1.elb.amazonaws.com	80	39d

Figure 3.18: Vérification des Ingress

Chapitre 4

Release 2

4.1 Sprint1:MISE EN PLACE PIPELINE CI/CD

4.1.1 Objectif

L'établissement d'une pipeline CI/CD (Intégration Continue et Livraison Continue est indispensable pour automatiser le processus de développement, test et déploiement des applications. Cela garantit une livraison de code rapide et sûre en minimisant les risques d'erreurs humaines et assure une qualité constante du logiciel.

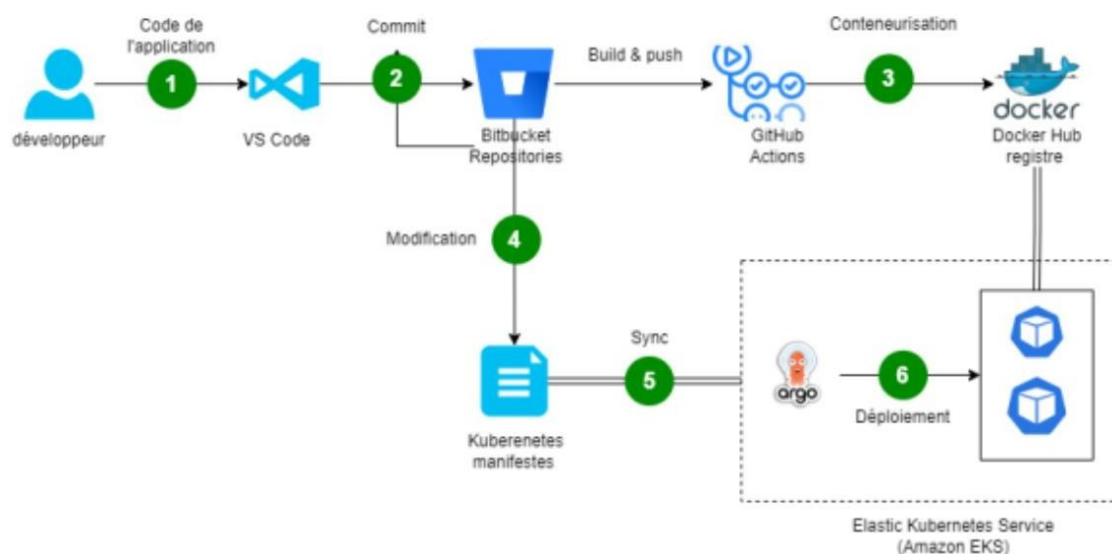


Figure 4.1:Architecture de pipeline Devops

Cette structure DevOps améliore le processus de mise en production des applications.

Le programmeur rédige le code en utilisant Visual Studio Code et l'engage dans un dépôt Bitbucket. GitHub Actions s'active ensuite pour construire et conteneuriser l'application, en téléchargeant l'image Docker obtenue sur Docker Hub. ArgoCD assure la mise à jour et la synchronisation des fichiers de configuration Kubernetes avec le cluster Amazon EKS.

En fin de compte, les changements sont mis en place sur le cluster EKS, garantissant un déploiement et une intégration continus, ce qui accroît l'efficacité et la fiabilité du processus de déploiement de l'application.

4.1.2 Déploiement automatisé avec GitHub Actions

Dans ce passage, nous détaillons le processus CI/CD automatisé pour le déploiement de l'application sur AWS EKS grâce à GitHub Actions. Ce procédé garantit un déploiement continu et sûr de notre application.

Le fichier deploy-eks.yaml du workflow contient diverses phases cruciales pour la préparation, l'essai et le déploiement des microservices sur le cluster Kubernetes.

4.1.3 Étapes du Workflow GitHub Actions

1 Déclenchement du pipeline

Le processus s'active à chaque mise à jour de la branche principale du dépôt GitHub, assurant que les modifications récentes du code sont automatiquement mises en production.

2 Installation des outils nécessaires

Helm : Helm est mis en place pour faciliter et standardiser les déploiements Kubernetes. Il est programmé pour traiter les graphiques Helm et automatiser le déploiement des microservices sur Kubernetes.

Docker : L'image Docker de l'application est élaborée et marquée avec le hash du commit Git afin d'assurer que la version mise en production est exacte et traçable.

2-Analyse de sécurité avec OWASP Dependency-Check

Avant sa mise en œuvre, le projet est examiné pour identifier les failles de sécurité dans les dépendances grâce à l'utilisation de l'outil OWASP Dependency-Check. Ce rapport facilite la détection des dangers sécuritaires au sein des bibliothèques externes employées par l'application.

3-Scan de sécurité de l'image Docker avec Trivy

Ensuite, l'image Docker est analysée avec Trivy afin d'identifier les vulnérabilités critiques. En cas de détection d'une vulnérabilité majeure, le processus de mise en production est stoppé afin d'éviter l'insertion de failles dans l'environnement de production.

4-Push de l'image Docker dans Amazon ECR

Une fois l'image approuvée, elle est transférée vers Amazon ECR (Elastic Container Registry) où elle est conservée et prête pour les déploiements sur le cluster Kubernetes.

5-Déploiement avec Helm sur Kubernetes EKS

Helm est employé pour le déploiement ou la mise à jour des microservices sur le cluster EKS de Kubernetes. L'implémentation est réalisée en se basant sur le diagramme Helm approprié, avec l'image Docker étiquetée selon le dernier commit Git.

6-Analyse de sécurité dynamique avec OWASP ZAP

L'application déployée fait l'objet d'une analyse de sécurité dynamique grâce à OWASP ZAP (Zed Attack Proxy). Cette évaluation soumet l'application à diverses attaques habituelles afin d'identifier des failles de sécurité en temps réel.

Téléchargement des rapports de sécurité

L'application mise en production est soumise à une analyse de sécurité dynamique effectuée avec OWASP ZAP (Zed Attack Proxy). Cette évaluation consiste à exposer l'application à différentes attaques courantes pour détecter des vulnérabilités de sécurité en temps réel.

Résumé du Workflow

Ce processus CI/CD assure un déploiement automatisé et sécurisé sur AWS EKS. Il

comprend des phases cruciales comme la création et la protection des images Docker, l'examen des dépendances, ainsi que des vérifications de sécurité préalables et postérieures au déploiement. Ce processus garantit la qualité et la sécurité des microservices tout en préservant une démarche de déploiement agile et performante.

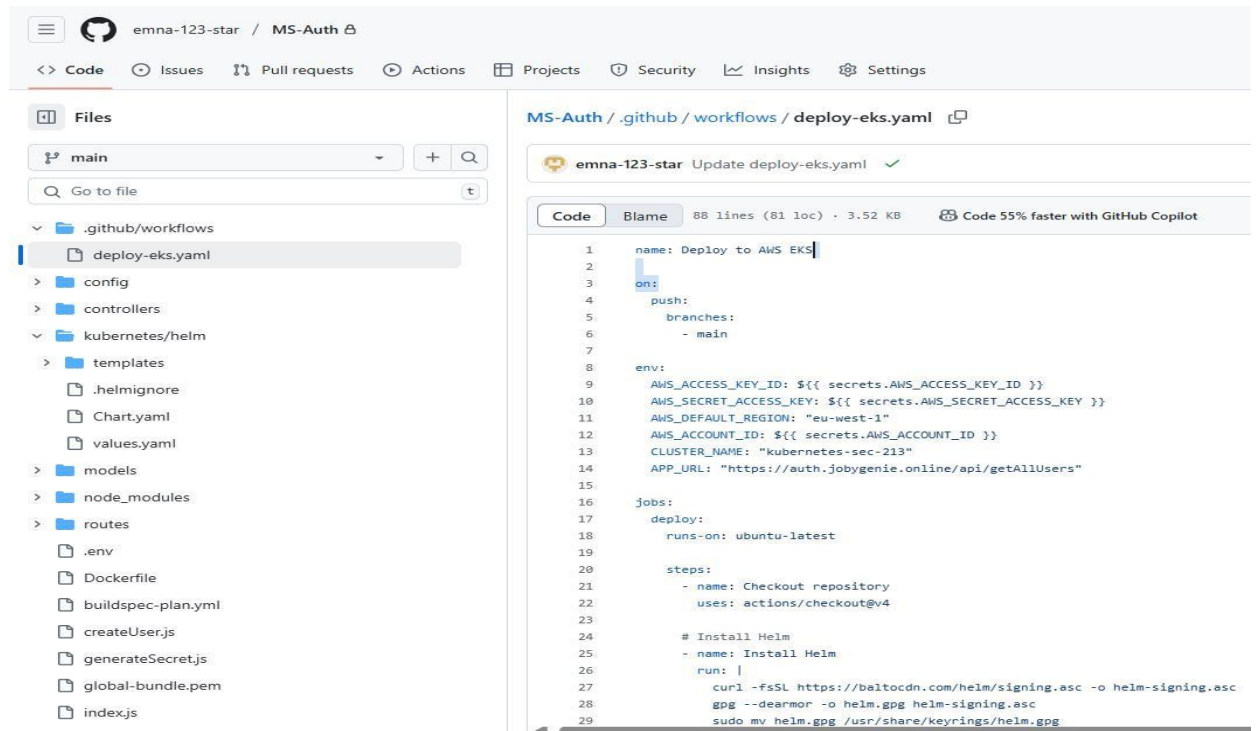


Figure 4.2: Workflow CI/CD GitHub Actions pour le microservice ms-Auth

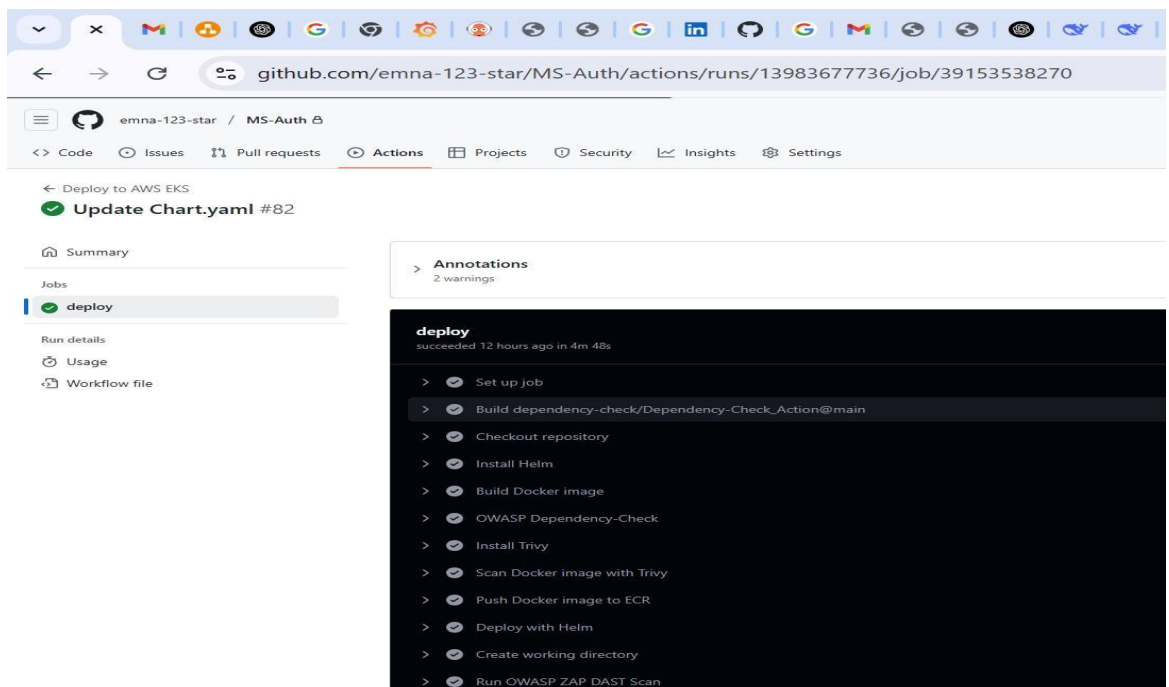


Figure 4.3: Pipeline GitHub Actions pour le déploiement de microservice ms-Auth sur AWS EKS

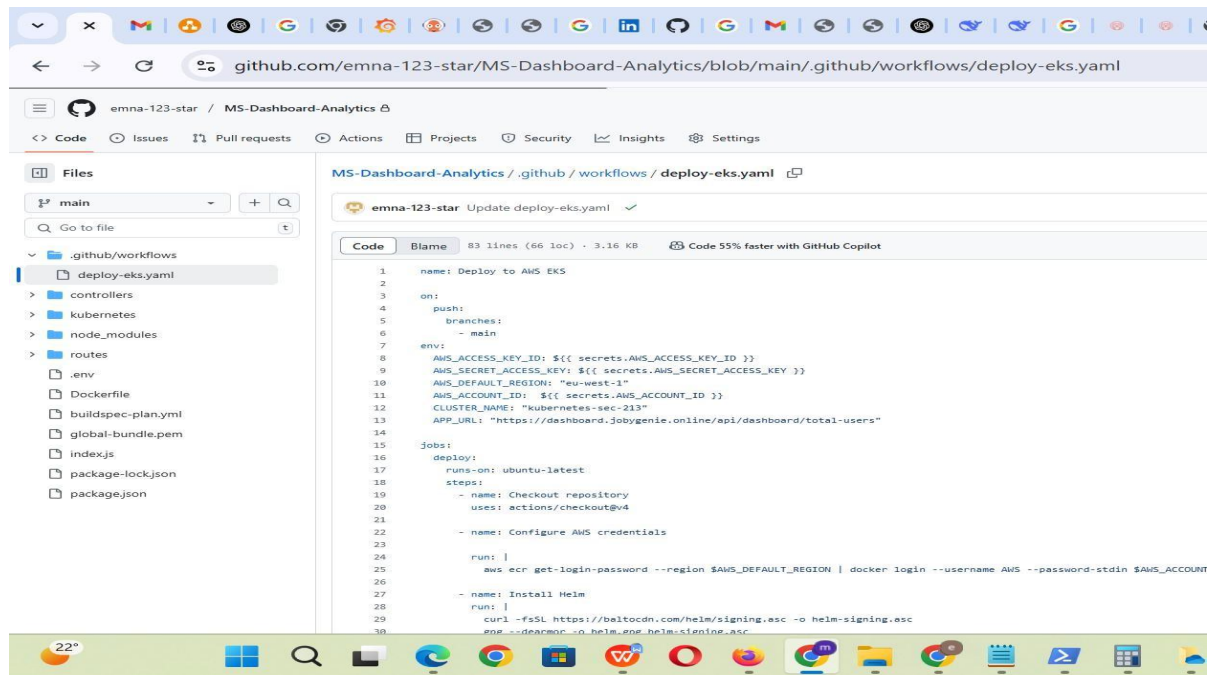


Figure 4.4: Workflow CI/CD GitHub Actions pour le microservice ms- dashboard-and- Analytics

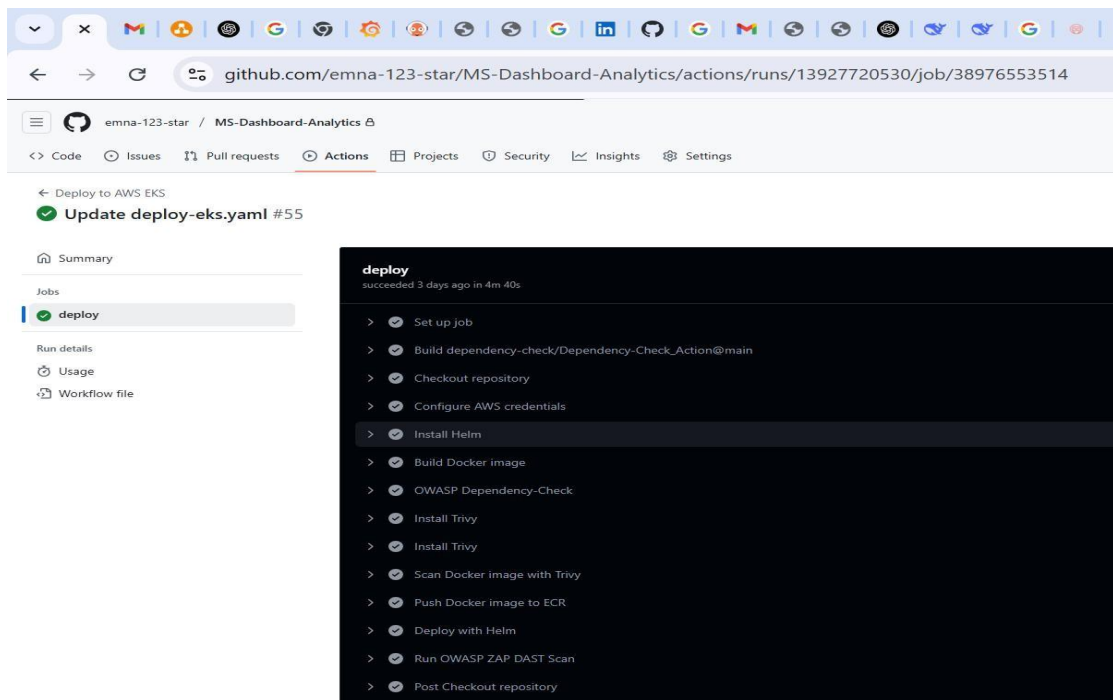


Figure 4.5: Pipeline GitHub Actions pour le déploiement de microservice ms-Dashboard sur AWS EKS

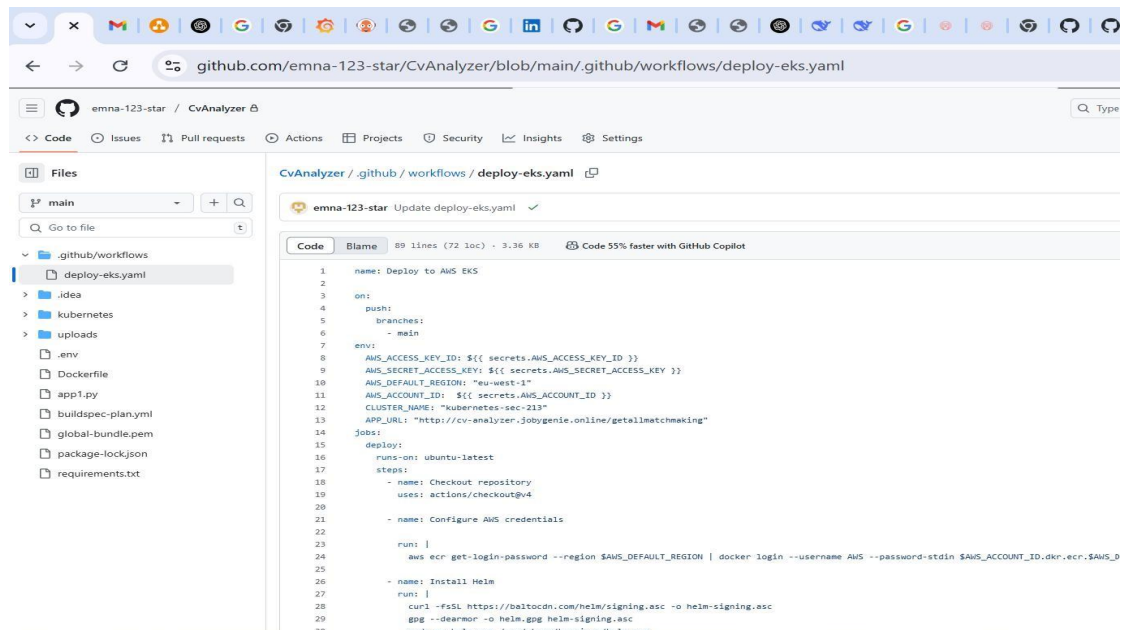


Figure 4.6: Workflow CI/CD GitHub Actions pour le microservice cv- Analyzer

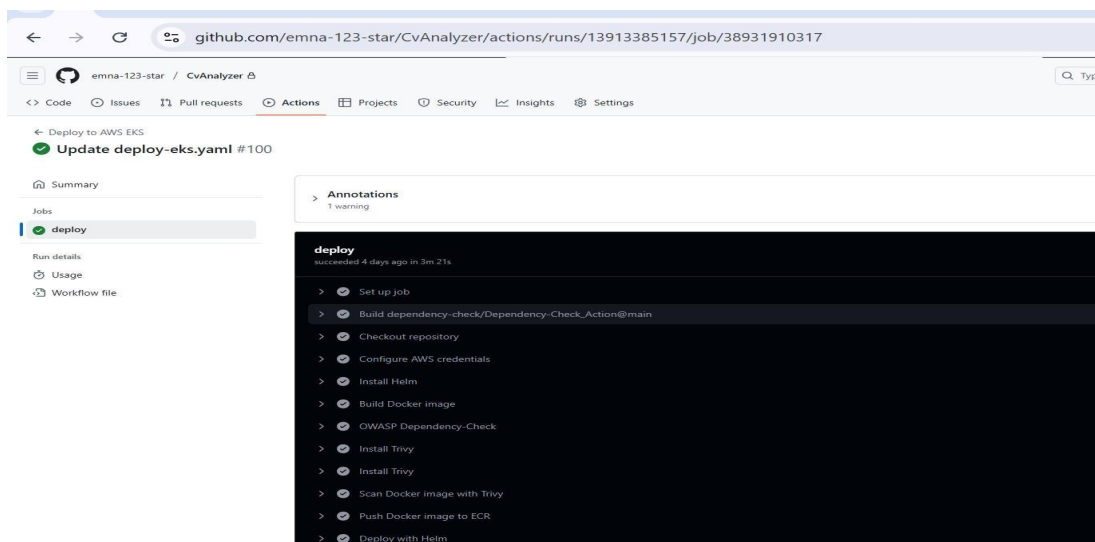


Figure 4.7: Pipeline GitHub Actions pour le déploiement de microservice cvAnalyzer sur AWS EKS

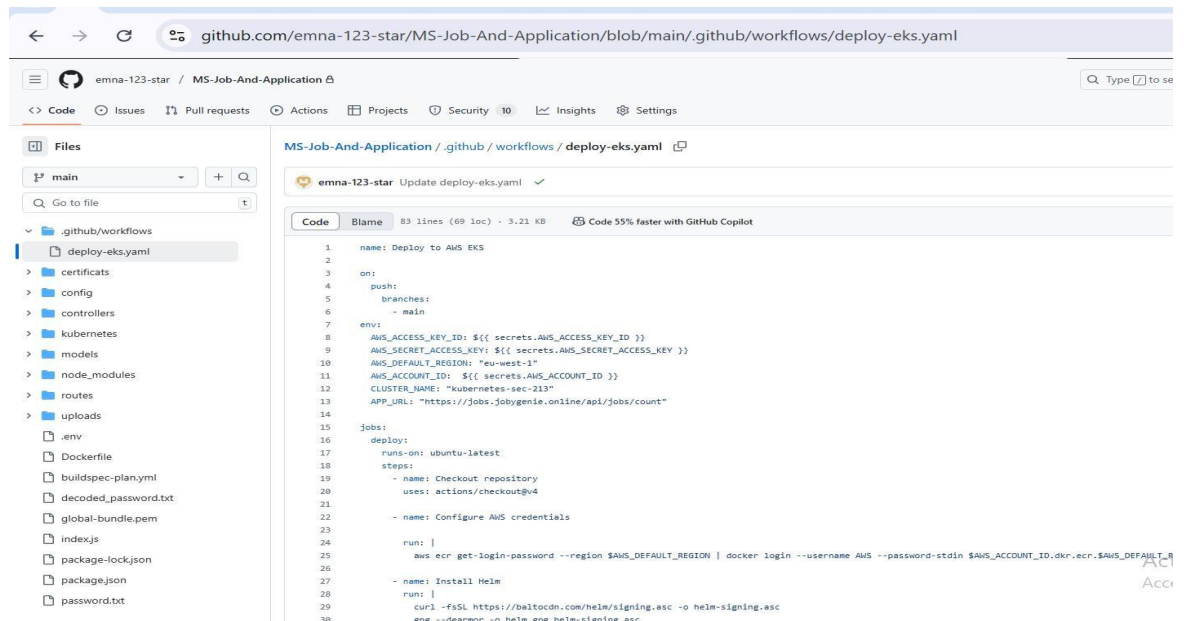


Figure 4.8: Workflow CI/CD GitHub Actions pour le microservice MS- Job- Application

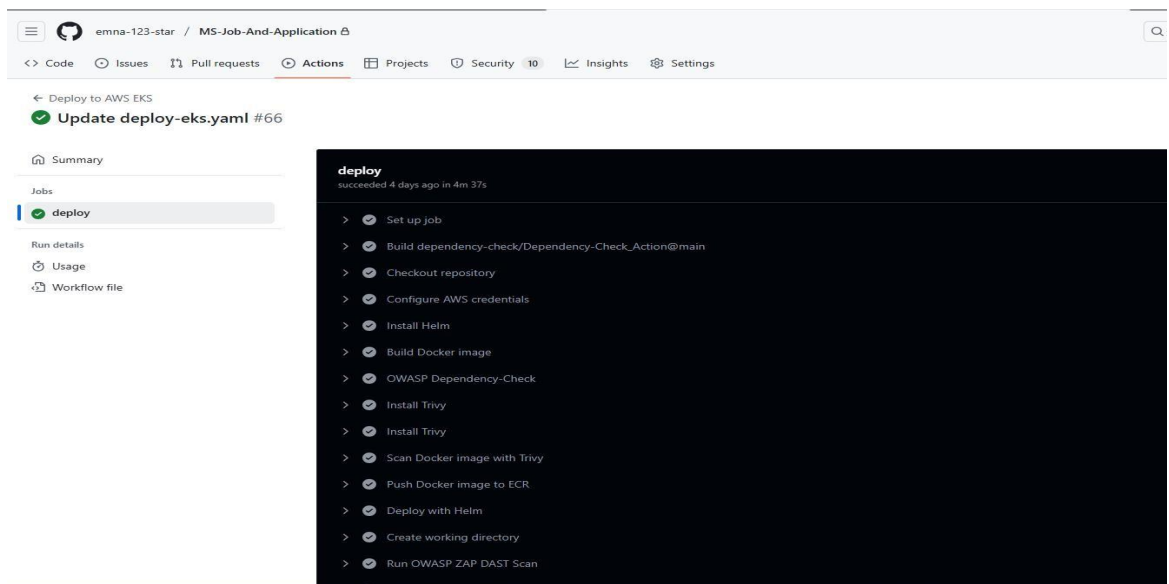


Figure 4.9: Pipeline GitHub Actions pour le déploiement de microservice MS-Job-And- Application sur AWS EKS

4.2 Sprint 2: Sécurité et Analyse des Vulnérabilités

4.2.1 Scan de sécurité avec Trivy

TRIVY

Nous avons employé Trivy en tant qu'outil d'analyse des vulnérabilités pour les images Docker, les systèmes de fichiers et les dépôts Git. Il excelle spécifiquement dans l'analyse des images Docker et la détection des failles de sécurité dans les programmes contenus dans ces images. Trivy examine les images docker et produit un compte rendu détaillé des vulnérabilités détectées.

Un fichier texte est généré, contenant un rapport détaillé des vulnérabilités identifiées, comme illustré dans la Figure.

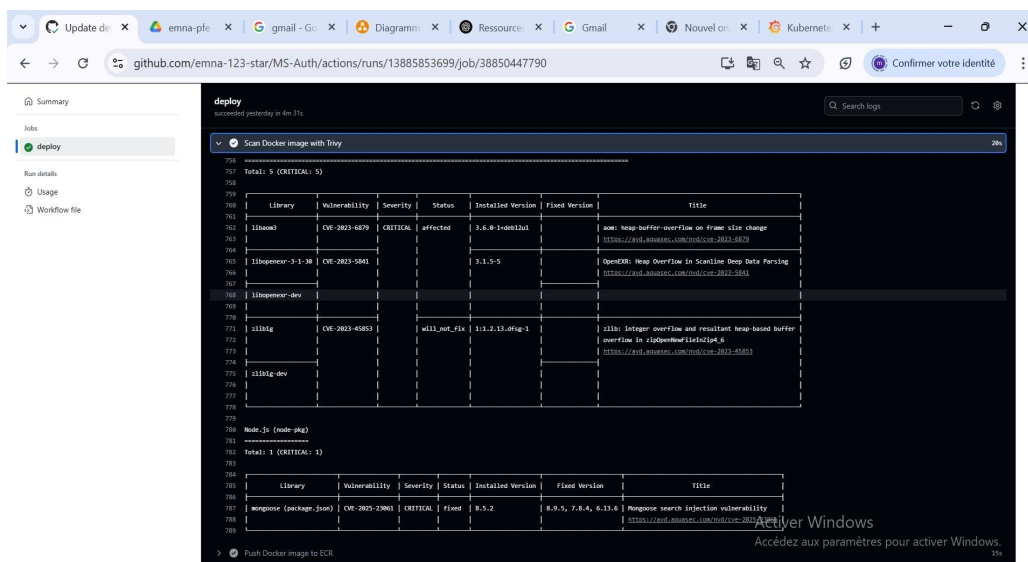


Figure 4.10: Rapport de scan de sécurité Docker avec Trivy pour le microservice-MS-Auth

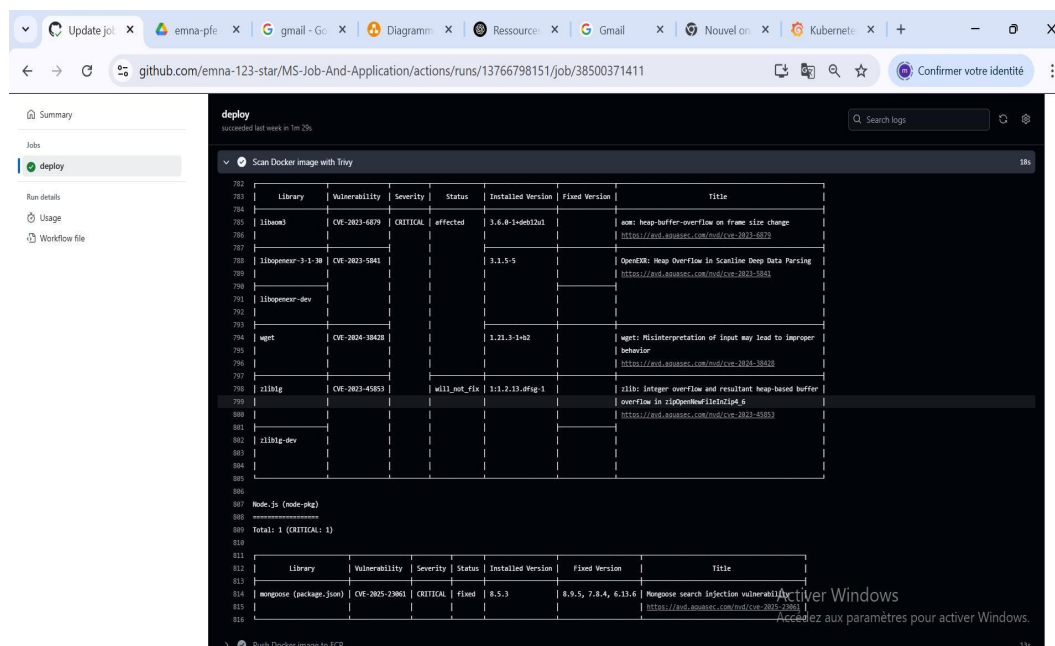


Figure 4.11: Rapport de scan de sécurité Docker avec Trivy pour le microservice MS-Job-And-Application

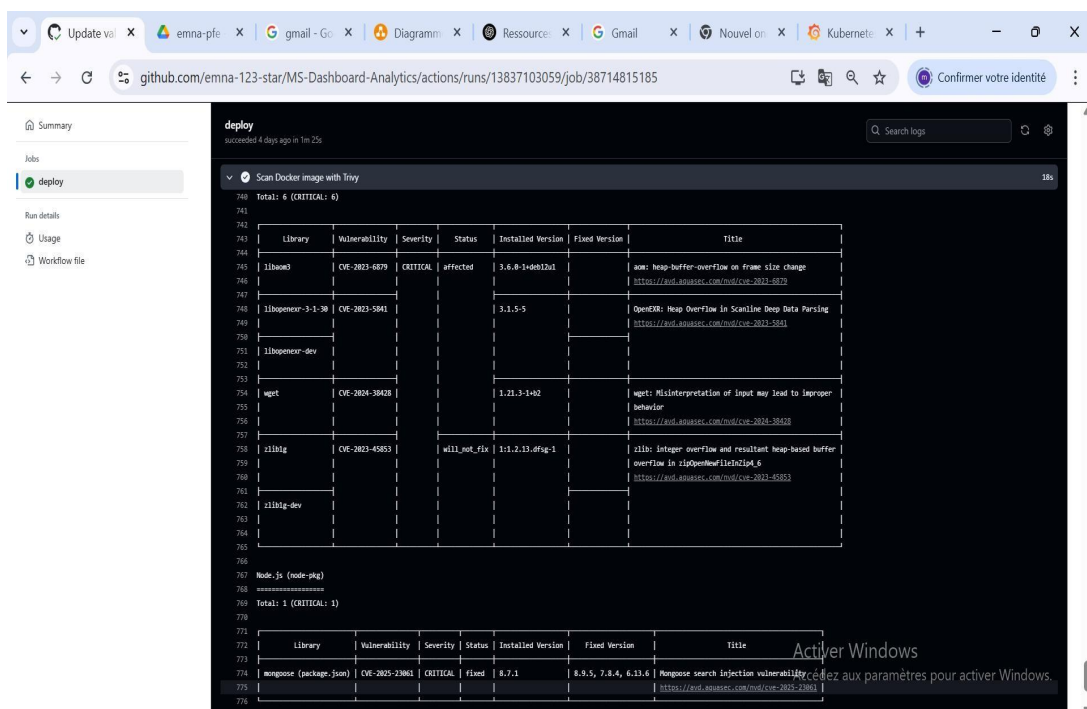


Figure 4.12: Rapport de scan de sécurité Docker avec Trivy pour le microservice MS-Dashboard-Analytics

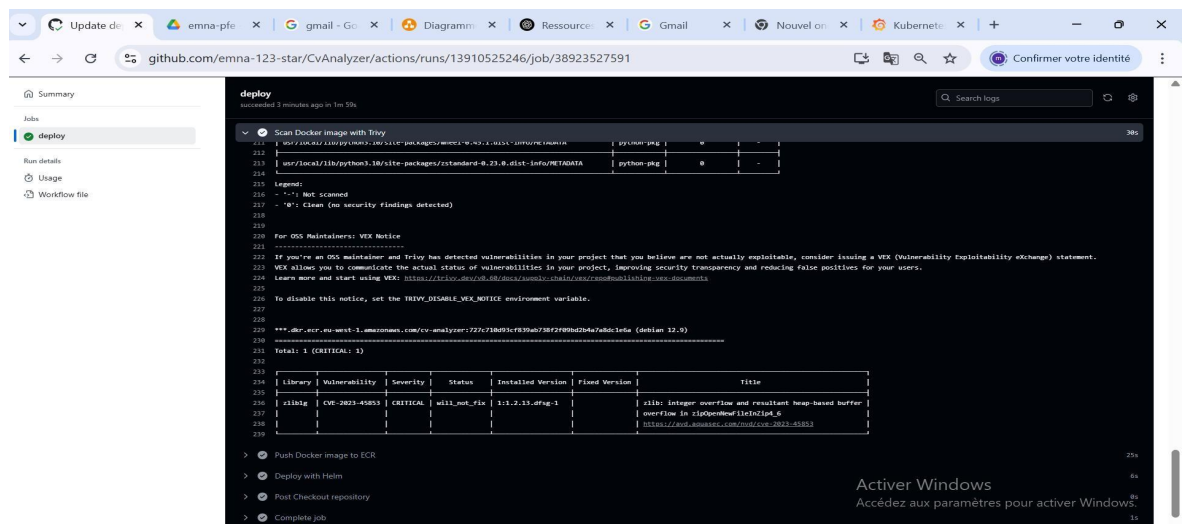


Figure 4.13: Rapport de scan de sécurité Docker avec Trivy pour le microservice CvAnalyzer

4.2.2 Analyse des dépendances avec OWASP Dependency-Check

4.2.2 Analyse des dépendances avec OWASP Dependency-Check

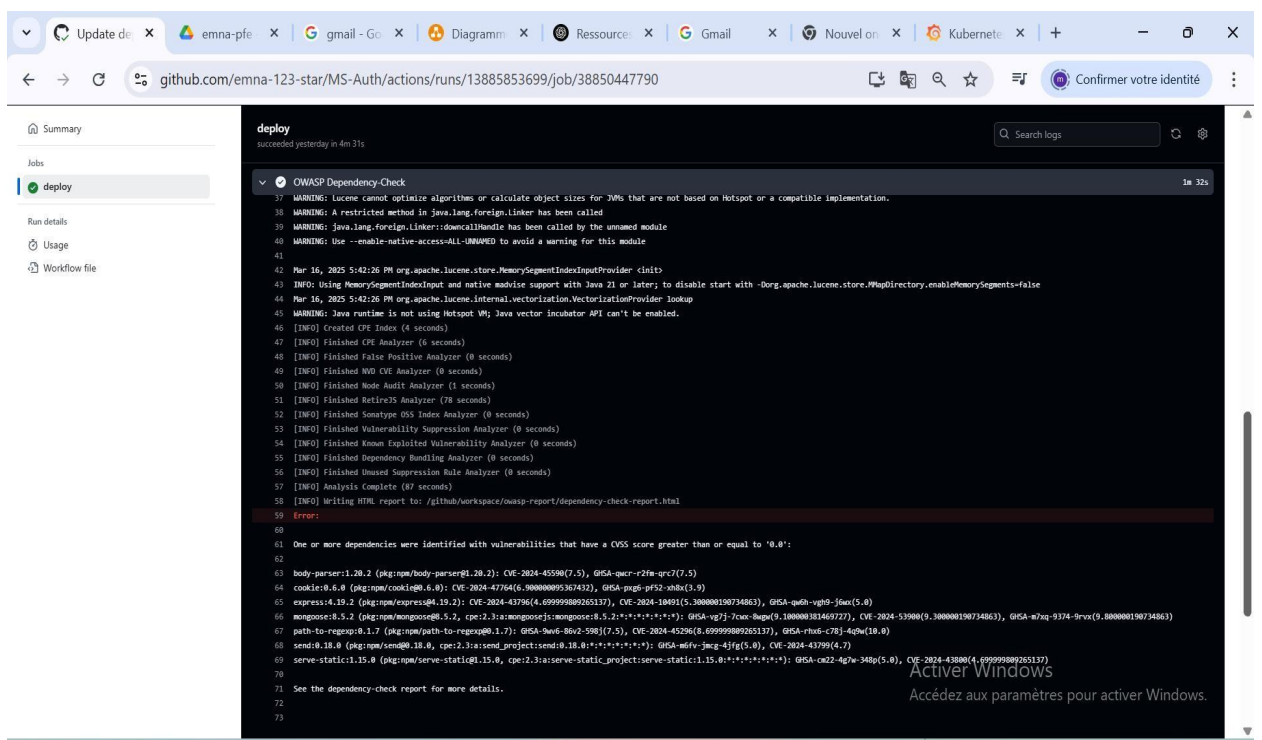


Figure 4.14: rapport de sécurité généré par OWASP Dependency-Check de MS-Auth

Vulnérabilités détectées

Package	CVE/GHSA ID	Score CVSS
body-parser (1.20.2)	CVE-2024-45590, GHSA-qwcr-r2fm-qrc7	7.5 (Élevé)
cookie (0.6.0)	CVE-2024-47764, GHSA-pxg6-pf52-xh8x	6.9 (Modéré)
express (4.19.2)	CVE-2024-43796, CVE-2024-10491, GHSA-qw6h-vgh9-j6wx	5.3 (Modéré)
mongoose (8.5.2)	CVE-2024-53900, GHSA-m7xq-9374-9rvx	9.8 (Critique)
path-to-regexp (0.1.7)	CVE-2024-45296, GHSA-rhx6-c78j-4q9w	10.0 (Critique)
send (0.18.0)	CVE-2024-43799, GHSA-m6fv-jmcg-4jfg	5.0 (Modéré)
serve-static (1.15.0)	CVE-2024-43800, GHSA-cm22-4g7w-348p	5.0 (Modéré)

Figure4.15:Rapport de Vulnérabilités - Dépendances Critiques (CVSS) de MS-Auth

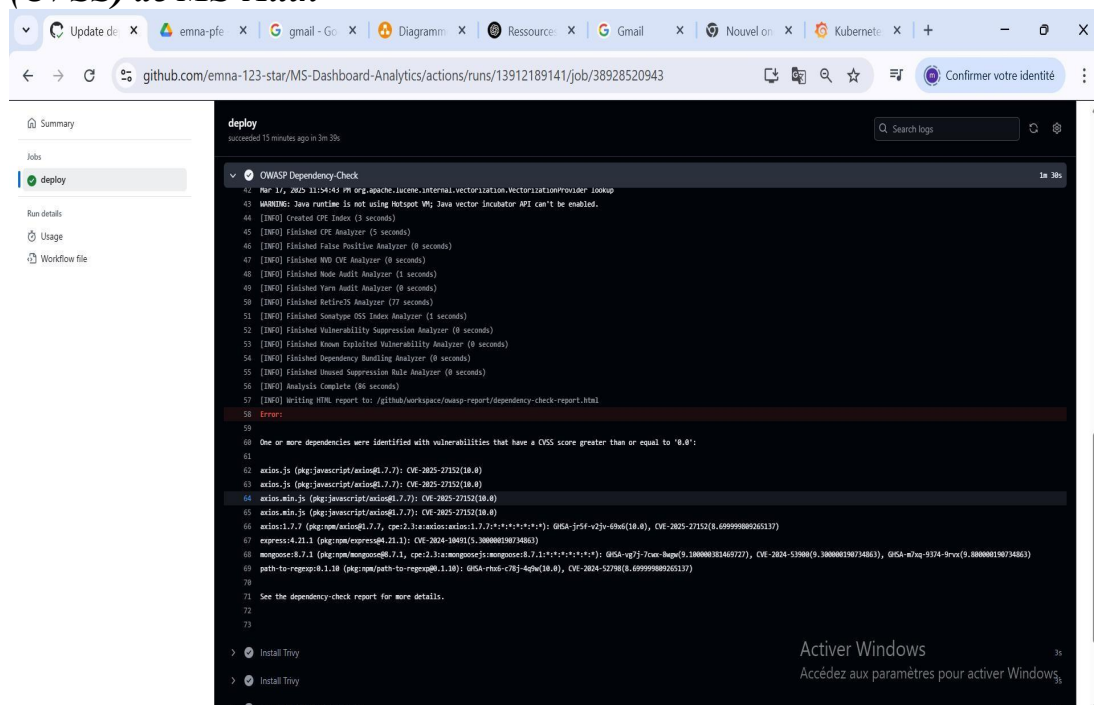


Figure 4.16: rapport de sécurité généré par OWASP Dependency- Check de MS-Dashboard-Analytics

Dépendance	Version actuelle	CVE/GHSA	Score CVSS	Action recommandée
axios	1.7.7	CVE-2025-27152, GHSA-jr5f-v2jv-69x6	10.0	Mettre à jour vers <code>axios@latest</code>
path-to-regexp	0.1.10	GHSA-rhx6-c78j-4q9w, CVE-2024-52798	10.0 - 8.7	Mettre à jour vers <code>path-to-regexp@latest</code>
mongoose	8.7.1	CVE-2024-53900, GHSA-m7xq-9374-9rvx	9.3 - 9.8	Mettre à jour vers <code>mongoose@latest</code>
express	4.21.1	CVE-2024-10491	5.3	Mettre à jour vers <code>express@latest</code>

Figure4.17:Rapport de Vulnérabilités - Dépendances Critiques (CVSS)pour MS-Dashboard-Analytics

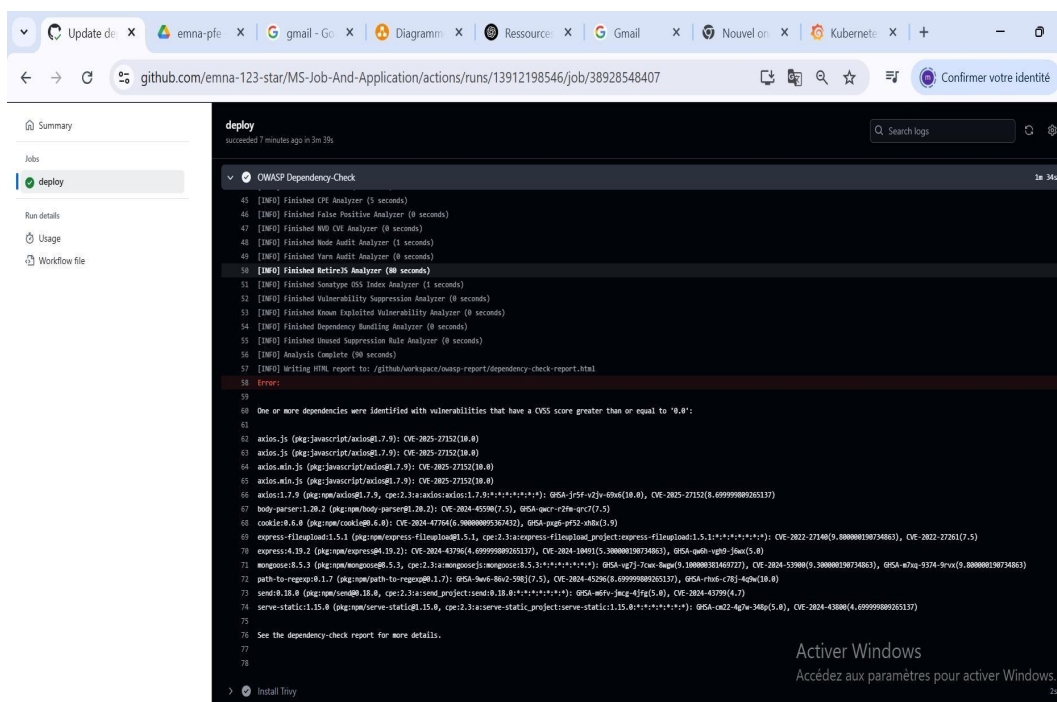


Figure 4.18: rapport de sécurité généré par OWASP Dependency-Check de MS-Job-AndApplication

Dépendance	Version actuelle	CVE/GHSA	Score CVSS	Action recommandée
axios	1.7.9	CVE-2025-27152	10.0	Mettre à jour vers <code>axios@latest</code>
path-to-regexp	0.1.7	GHSA-rhx6-c78j-4q9w	10.0	Mettre à jour vers <code>path-to-regexp@latest</code>
mongoose	8.5.3	CVE-2024-53900, GHSA-m7xq-9374-9rvx	9.3 - 9.8	Mettre à jour vers <code>mongoose@latest</code>
express-fileupload	1.5.1	CVE-2022-27140	9.8	Mettre à jour vers <code>express-fileupload@latest</code>
body-parser	1.20.2	CVE-2024-45590	7.5	Mettre à jour vers <code>body-parser@latest</code>
cookie	0.6.0	CVE-2024-47764	6.9	Mettre à jour vers <code>cookie@latest</code>
express	4.19.2	CVE-2024-43796, CVE-2024-10491	4.7 - 5.3	Mettre à jour vers <code>express@latest</code>
send	0.18.0	CVE-2024-43799	4.7	Mettre à jour vers <code>send@latest</code>
serve-static	1.15.0	CVE-2024-43800	4.7	Mettre à jour vers <code>serve-static@latest</code>

Figure 4.19: Rapport de Vulnérabilités - Dépendances Critiques (CVSS) pour MS-Job-AndApplication

4.2.3 Analyse dynamique avec OWASP ZAP

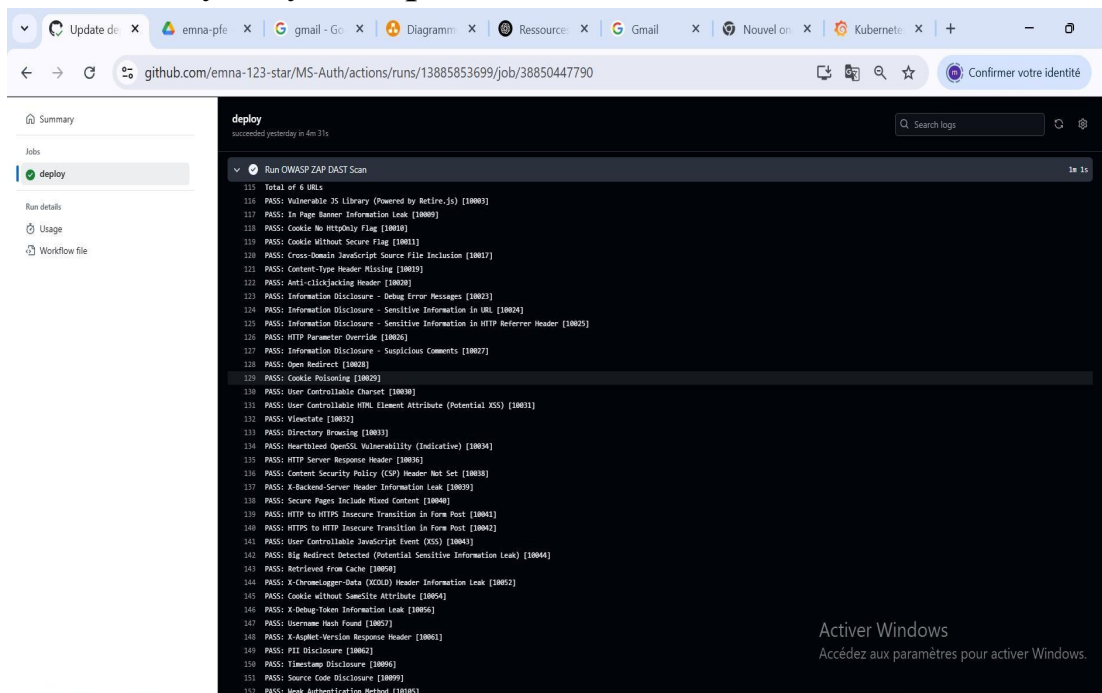


Figure 4.20: Résumé du scan de sécurité ZAP de MS-Auth



Figure 4.21: Liste des Vulnérabilités + Rapport Résumé d'Analyse de MS-Auth

Résumé des résultats de sécurité : Vulnérabilités (FAIL) :

-Aucune nouvelle vulnérabilité critique détectée.

-Vulnérabilités déjà connues (WARN) :

-Re-examine Cache-control Directives : Revoir les directives de cache.

-X-Content-Type-Options Header Missing : Absence de l'en-tête de sécurité.

-Strict-Transport-Security Header Not Set : L'en-tête HSTS est manquant sur plusieurs endpoints.

-Server Leaks Information via "X-Powered-By" : Le serveur divulgue des informations via l'en-tête HTTP.

-Storable and Cacheable Content : Le contenu peut être stocké et mis en cache, ce qui pourrait poser un risque.

-CSP: Wildcard Directive : Utilisation d'un caractère générique dans la politique CSP.

-Permissions Policy Header Not Set : L'en-tête de politique de permissions est manquant.

-Hash Disclosure - BCrypt : Divulcation de hachages (25 occurrences), ce qui pourrait compromettre les mots de passe.

-Cross-Domain Misconfiguration : Mauvaise configuration pour la gestion des requêtes inter-domaines.

Autres résultats :

-PASS : 57 tests réussis sans problème de sécurité.

-INFO/IGNORE : Aucun message informatif ou ignoré.

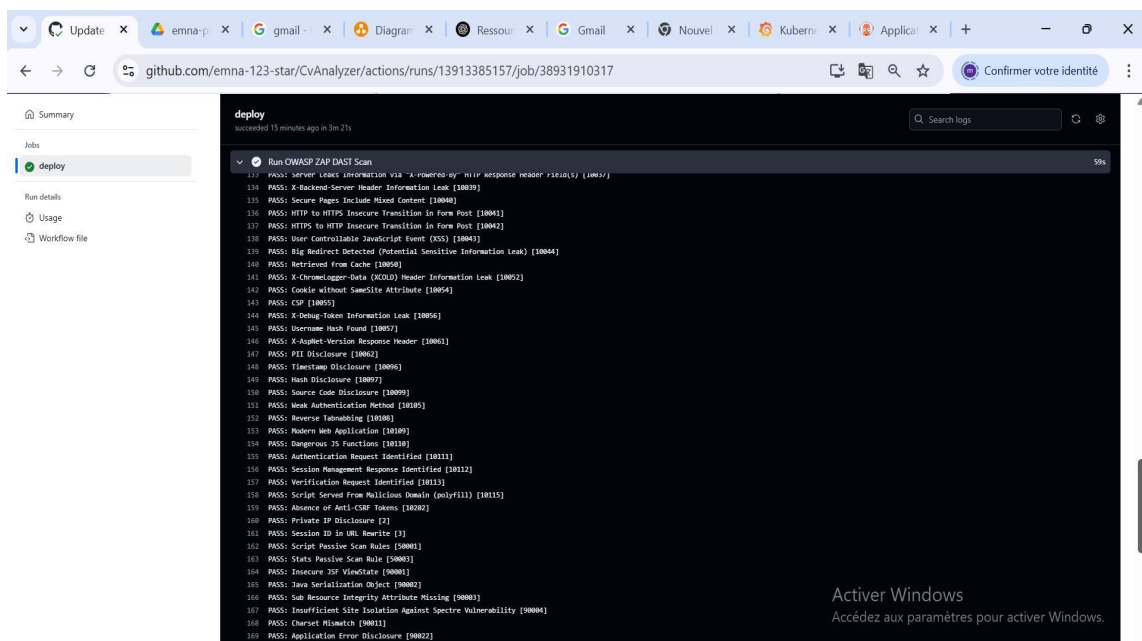


Figure 4.22: Résumé du scan de sécurité ZAP de CvAnalyzer



Figure 4.23: Liste des Vulnérabilités + Rapport Résumé d'Analyse de CvAnalyzer

Résumé du scan de sécurité ZAP ✓

60 tests réussis (aucune faille critique détectée)
6 avertissements à corriger :

Manque du header X-Content-Type-Options → Empêcher le MIME sniffing **Fuite d'informations sur la version du serveur** (Server header) → Masquer ou désactiver

Absence de Content Security Policy (CSP) → Définir une politique de sécurité **Contenu stockable et mis en cache** → Configurer les directives Cache-Control **Absence du header** Permissions-Policy → Limiter les permissions du navigateur **Mauvaise configuration CORS (Cross-Domain)** → Restreindre les accès

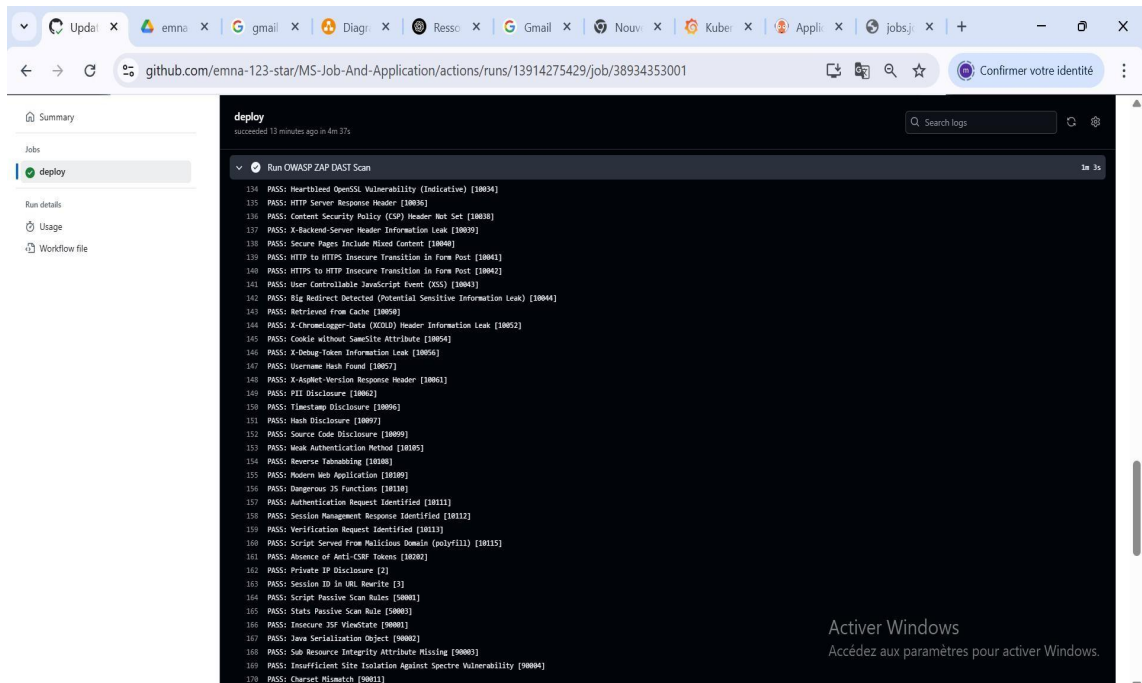


Figure 4.24:Résumé du scan de sécurité ZAP de MS-Job-And- Application

FAIL-NEW: 0	FAIL-INPROG: 0	WARN-NEW: 8	WARN-INPROG: 0	INFO: 0	IGNORE: 0	PASS: 58
-------------	----------------	-------------	----------------	---------	-----------	----------

Figure 4.25: Liste des Vulnérabilités + Rapport Résumé d'Analyse de MS-Job-And- Application

Résumé du scan de sécurité ZAP ✓

58 tests réussis (aucune faille critique détectée)

8 avertissements à corriger :

Cache-Control mal configuré → Empêcher la mise en cache de certaines réponses sensibles

Absence du header X-Content-Type-Options → Empêcher le MIME sniffing

Absence du header Strict-Transport-Security (HSTS) → Renforcer la sécurité HTTPS Fuite d'informations sur la version du serveur (X-Powered-By header) → Masquer ou désactiver

Contenu stockable et mis en cache → Ajuster les directives Cache-Control et Pragma CSP mal configuré (directive wildcard *) →

Restreindre les sources autorisées Absence du header Permissions-Policy → Limiter l'accès aux fonctionnalités du navigateur

8 Mauvaise configuration CORS (Cross-Domain) → Vérifier les règles d'accès

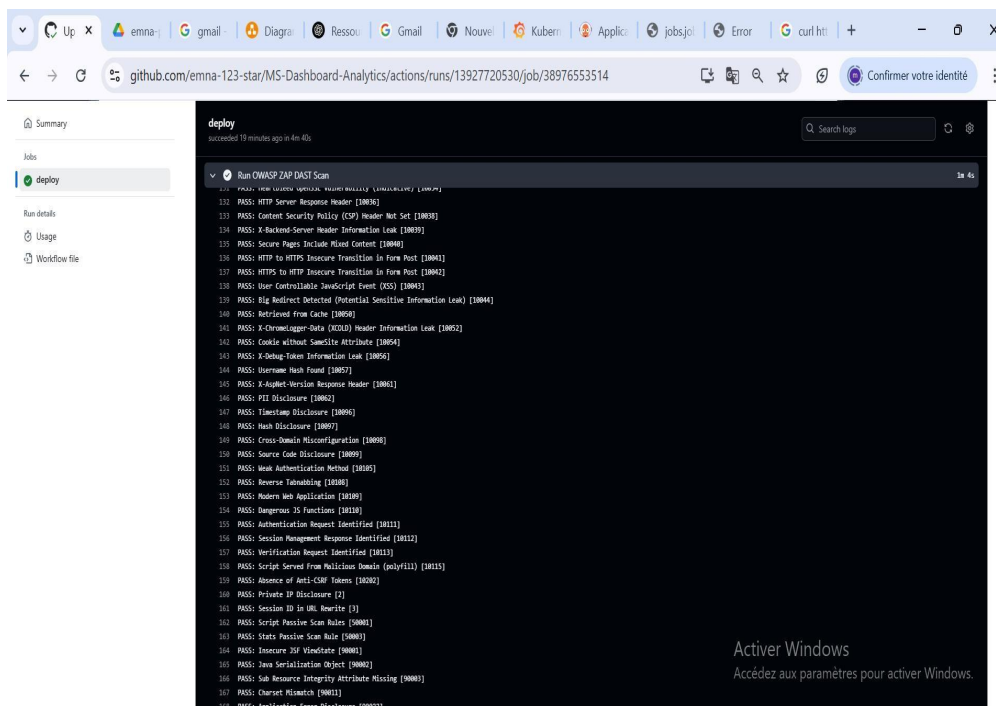


Figure 4.26: Résumé du scan de sécurité ZAP de MS-Dashboard-And- Analytics

FAIL-NEW: 0	FAIL-INPROG: 0	WARN-NEW: 8	WARN-INPROG: 0	INFO: 0	IGNORE: 0	PASS: 58
-------------	----------------	-------------	----------------	---------	-----------	----------

Figure 4.27: Liste des Vulnérabilités + Rapport Résumé d'Analyse de MS-Dashboard-And-Analytics

Résumé du scan ZAP

✓ **Sécurité globale : Bonne** (58 tests réussis, 0 échec critique)
8 avertissements détectés :

Cache mal configuré (risque d'exposition de données).

Absence de l'en-tête X-Content-Type-Options (risque de MIME sniffing).

4.3 Sprint 3: Configuration et Supervision avec ArgoCD

4.3.1 Déploiement GitOps avec ArgoCD

4.3.2 Surveillance des applications déployées

L'utilisation de GitOps avec ArgoCD facilite l'automatisation de la gestion des applications Kubernetes en conservant la configuration déclarative dans un dépôt Git. Cette méthode assure que l'état du cluster reste constamment en accord avec la source de vérité, simplifiant ainsi la surveillance et le retour arrière en cas d'irrégularité.

Explication du processus de synchronisation

Le processus de synchronisation dans ArgoCD suit les étapes suivantes :

1-Surveillance du dépôt Git : *ArgoCD surveille en continu le dépôt Git défini pour détecter tout changement dans les fichiers Helm ou YAML.*

2-Comparaison avec l'état actuel du cluster : *ArgoCD compare la configuration du dépôt avec l'état des ressources Kubernetes déployées.*

3-Mise en conformité automatique (Sync) : *Si une divergence est détectée, ArgoCD peut appliquer automatiquement les mises à jour pour synchroniser le cluster avec la configuration Git.*

3-Validation et suivi des changements : *ArgoCD enregistre les modifications et offre une traçabilité complète des déploiements.*

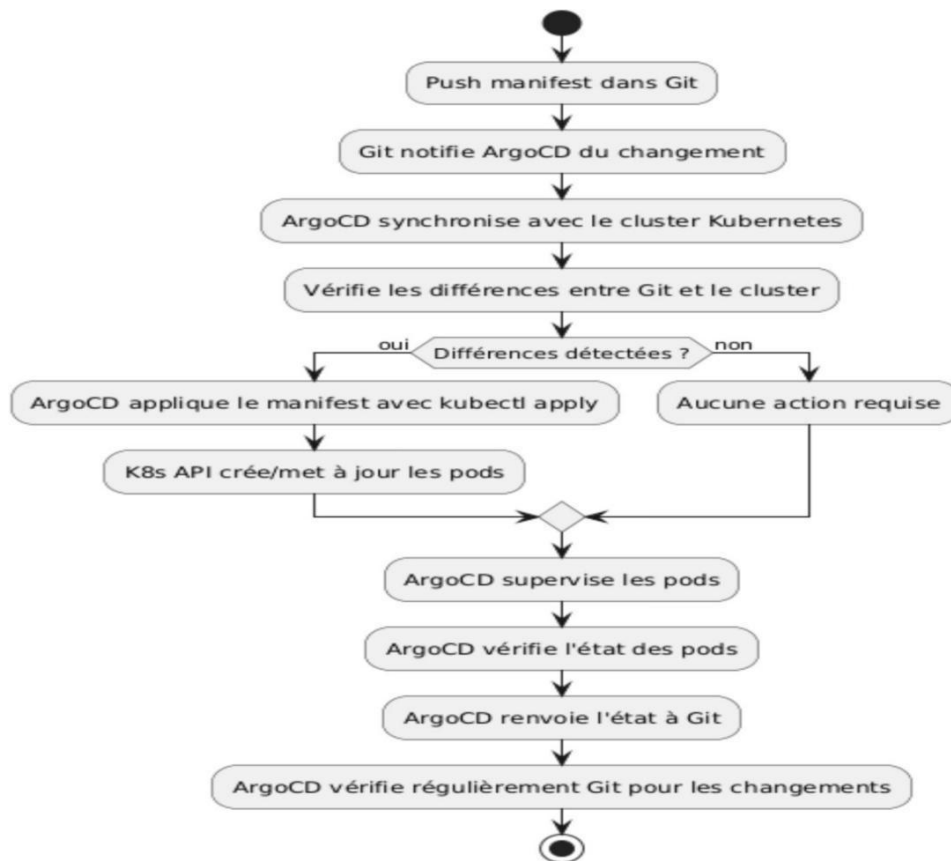


Figure 4.28:Diagramme d'activité

1. Push du manifest de déploiement :

Le DevOps déploie un nouveau manifeste de déploiement ou met à jour celui-ci dans le référentiel Git.

2. Notification du changement

Le dépôt Git informe ArgoCD qu'une modification a été apportée aux fichiers de déploiement (manifeste YAML).

3. Synchronisation des manifests

ArgoCD initie un processus de synchronisation afin de comparer le nouveau état du manifeste avec l'état actuel du cluster Kubernetes.

4. Application du manifest :

En cas de détection d'une divergence entre le manifeste et l'état actuel du cluster, ArgoCD applique les modifications en exécutant une commande `kubectl apply` sur le cluster Kubernetes.

5. Création ou mise à jour des pods :

Le serveur API de Kubernetes reçoit la requête de déploiement et crée ou met à jour les pods en conséquence.

Le rôle des instructions du manifeste.

6. Supervision des pods :

ArgoCD surveille de manière continue l'état des pods en utilisant le serveur API de Kubernetes afin de les vérifier.

Vérifier que les pods sont dans l'état souhaité.

7. Retour de l'état actuel :

Le serveur API de Kubernetes transmet l'état des pods à ArgoCD afin de garantir le bon fonctionnement de l'ensemble.

8. Vérification des manifests dans Git :

ArgoCD effectue une vérification périodique des manifests présents dans le référentiel Git afin de repérer toute modification par rapport à l'état courant du cluster Kubernetes.

9. Détection de changement :

En cas de détection d'une modification dans le référentiel Git, notamment l'apparition d'une nouvelle version du manifeste, ArgoCD initie automatiquement un processus de déploiement visant à synchroniser l'état du cluster avec ce manifeste.

10. Vérification de l'état des déploiements :

Le personnel en charge de l'ingénierie DevOps a la possibilité d'accéder à l'interface web d'ArgoCD afin de contrôler l'état des déploiements, consulter les journaux (logs) et vérifier la conformité entre les fichiers de configuration Git et l'état des pods dans l'environnement Kubernetes.

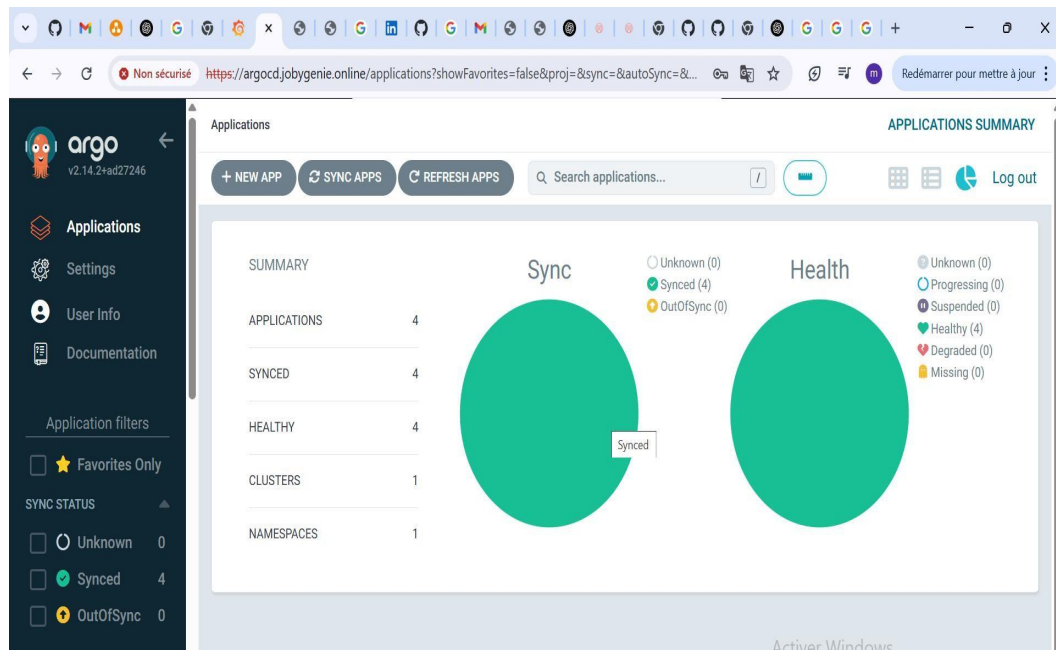


Figure29: Dashboard de monitoring des applications – ArgoCD (Sync & Health Status)

Vue d'ensemble :

-Applications : 4 applications sont actuellement gérées par Argo CD.

-Synced (Synchronisées) : 4 applications sont synchronisées avec les fichiers sources définis dans Git (c'est-à-dire que l'état de l'application dans le cluster Kubernetes correspond à l'état décrit dans Git).

-OutOfSync (Hors synchronisation) : 0 application n'est hors synchronisation, ce qui est bon signe.

-Healthy (Saines) : 4 applications sont dans un état **Healthy** (sain), ce qui signifie que leurs ressources fonctionnent correctement.

-Clusters : 1 cluster Kubernetes est connecté à Argo CD.

-Namespaces : Toutes les applications semblent être déployées dans 1 namespace.

Graphiques circulaires :

-Sync (Synchronisation) : Le cercle est entièrement vert (4 synchronisées), indiquant que toutes les applications sont à jour.

-Health (Santé) : Le cercle est aussi complètement vert (4 Healthy), montrant qu'il n'y a aucun problème de santé détecté (pas de dégradé, de manquant, ou en progression).

Filtres sur la gauche :

SYNC STATUS :

Unknown : 0

Synced : 4 (toutes synchronisées)

OutOfSync : 0

HEALTH STATUS :

Healthy : 4

Aucun en Progressing, Suspended, Degraded, ou Missing.

Conclusion :

Notre environnement Argo CD est **stable et en bon état** :

-Toutes les applications sont synchronisées avec le référentiel Git.

-Toutes les applications sont en bonne santé (Healthy).

-Aucun problème détecté.

Chapitre 5

Release 3

5.1 Sprint 1: Monitoring avec Grafana et Prometheus

5.1.1 Présentation de la stack Prometheus-Grafana

Au cours de ce sprint, il a été crucial de mettre en place un système de surveillance avancé afin d'assurer une supervision efficace des microservices déployés sur Kubernetes. Nous avons choisi d'utiliser la combinaison Prometheus-Grafana, qui est une solution open-source très répandue pour la collecte, le stockage et la visualisation des métriques système et applicatives.

5.1.2 Prometheus : Collecte et stockage des métriques

Prometheus représente un outil de surveillance spécifiquement élaboré pour les infrastructures cloud-native et étroitement intégré à Kubernetes. Son fonctionnement repose sur un modèle pull, qui consiste à interroger régulièrement des endpoints exposant des métriques.

5.1.2.1 Principaux composants de Prometheus :

Le serveur Prometheus stocke les métriques et offre la possibilité de les interroger à l'aide de PromQL. Le ServiceMonitor détecte de manière automatique les services Kubernetes à surveiller. L'AlertManager est chargé de la gestion des alertes en se basant sur des règles prédéfinies, telles que la surcharge CPU ou les pods en échec.

5.1.3 Grafana : Visualisation et analyse des métriques

Grafana est un outil de visualisation de données qui utilise les métriques recueillies par Prometheus pour créer des tableaux de bord dynamiques et interactifs.

Quelles sont les raisons qui justifient l'utilisation de Grafana ?

Des tableaux de bord personnalisables permettent de surveiller en temps réel la situation des microservices. L'intégration native avec Prometheus permet d'afficher des graphiques, des alertes et des métriques clés.

La sécurisation des accès est assurée par un système d'authentification et de gestion des rôles.

5.1.4 Déploiement sur Kubernetes

La configuration ci-dessous a été utilisée pour déployer la stack Prometheus-Grafana sur notre cluster Amazon EKS :

5.1.4.1 Utilisation de Helm pour le déploiement

Helm pour automatiser l'installation et la gestion des composants.

5.1.4.2 Configuration de l'Ingress ALB et accès sécurisé via HTTPS

L'Application Load Balancer (ALB) a été configuré pour permettre un accès sécurisé via le protocole HTTPS.

Entendu, merci. Pouvez-vous me donner une phrase à paraphraser? Des tableaux de bord prédéfinis sont disponibles pour la surveillance des pods, des services et

des requêtes HTTP.

Cette solution nous offre une analyse proactive des performances de nos services, ce qui nous permet de réagir rapidement en cas d'incident.

La prochaine section examinera de manière approfondie les fonctionnalités proposées par la stack Prometheus-Grafana.

Grafana est utilisé en mettant l'accent sur les tableaux de bord disponibles pour la surveillance de notre cluster.

Kubernetes. Voici quelques-uns des principaux tableaux de bord utiles

Alertmanager / Overview : Pour surveiller les alertes

CoreDNS : Pour surveiller les performances de CoreDNIS

etcd : Pour surveiller l'état de etcd, un magasin de clés-valeurs distribué.

Grafana Overview : Un tableau de bord général pour surveiller l'état de Grafana lui-même.

Kubernetes / API server : Pour surveiller le serveur API de

Kubernetes. **Kubernetes / Compute Resources / Multi-Cluster** :

Pour surveiller les ressources de calcul

sur plusieurs clusters Kubernetes.

Kubernetes / Compute Resources / Cluster : Pour surveiller les

ressources de calcul au

niveau du cluster

Final / monitoring / values.yaml

emna-123-star Update values.yaml

Code Blame 30 lines (27 loc) · 896 Bytes Code 55% faster with GitHub Copilot

```
1 grafana:
2   ingress:
3     enabled: true
4     ingressClassName: alb
5     annotations:
6       alb.ingress.kubernetes.io/scheme: internet-facing
7       alb.ingress.kubernetes.io/target-type: ip
8       alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:eu-west-1:476114158209:certificate/ddf340c6-c69c-4225-9e69-88773012d85e
9       alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS":443}]'
10      # alb.ingress.kubernetes.io/group.name: "jobygenie"
11     hosts:
12       - grafana.jobygenie.online
13     adminPassword:
14     service:
15       type: LoadBalancer
16
17
18   prometheus:
19     prometheusSpec:
20       retention: 15d
21       serviceMonitorSelector: {}
22   alertmanager:
23     alertmanagerSpec:
24       replicas: 2
25
26   ruleSelectorNilUsesHelmValues: false
27   serviceMonitorSelectorNilUsesHelmValues: false
28   podMonitorSelectorNilUsesHelmValues: false
29   probeSelectorNilUsesHelmValues: false
30   scrapeConfigSelectorNilUsesHelmValues: false
```

Figure 5.1: Configuration Helm - Grafana Ingress ALB Alertmanager

5.1.5 Surveillance des microservices et du cluster avec Grafana

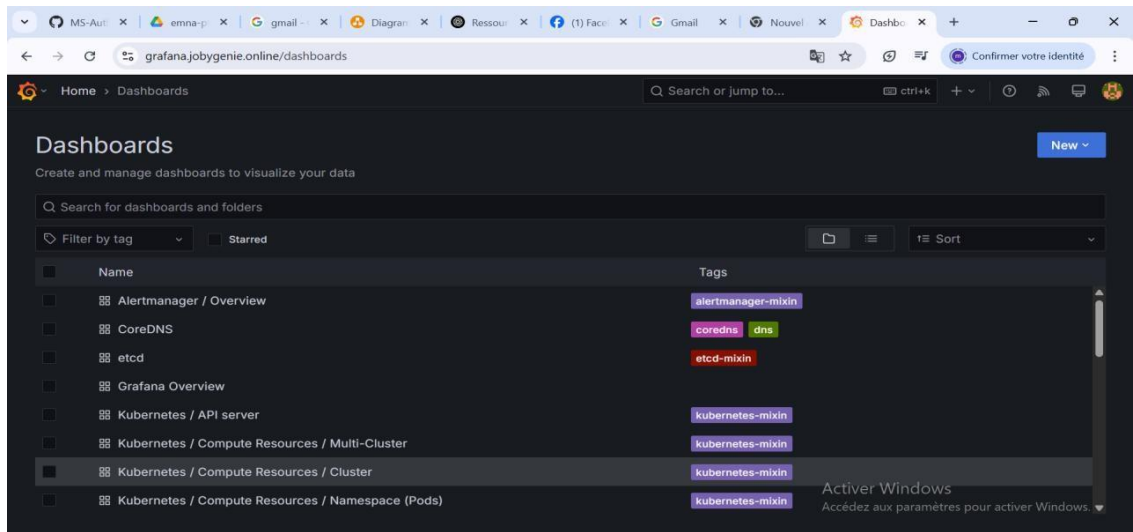


Figure 5.2: Interface de Gestion des Dashboards Grafana

Ces tableaux de bord offrent des visualisations détaillées ainsi que des métriques essentielles pour la supervision et la gestion de notre cluster.

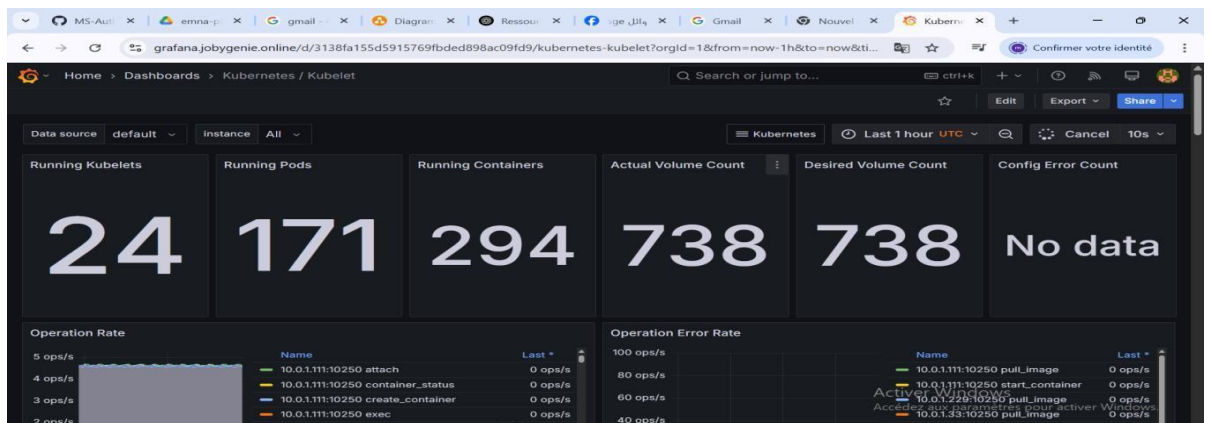


Figure 5.3: Tableau de bord Kubernetes - Métriques Kubelet et Opérations

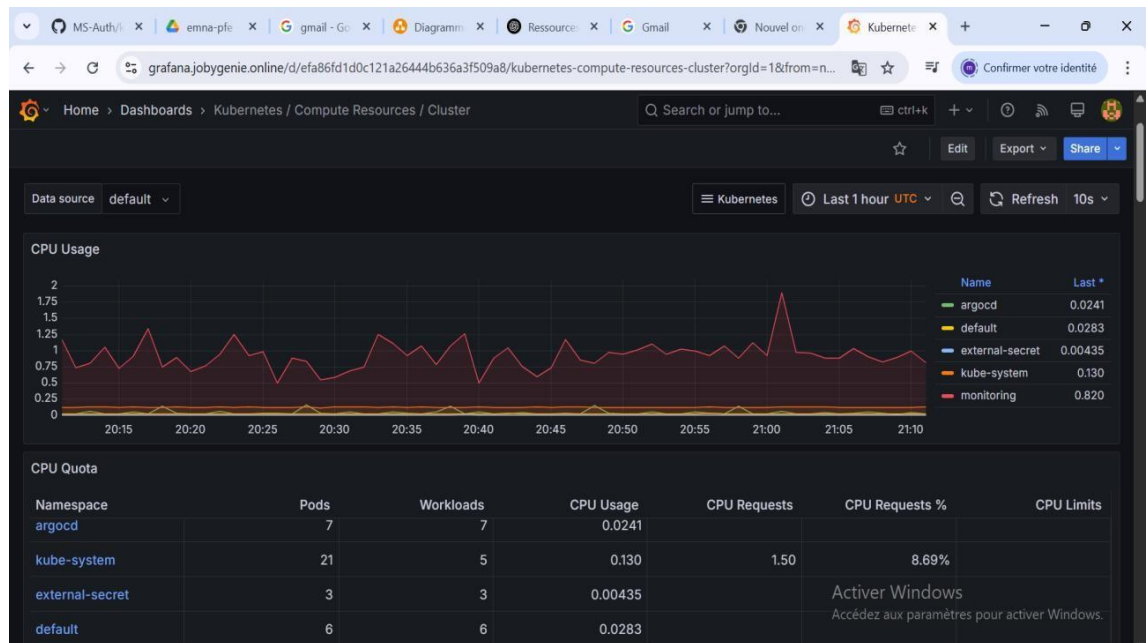


Figure 5.4: Dashboard de Surveillance des Ressources CPU dans Kubernetes

La Figure illustre le tableau de bord intégré dans Grafana. Il est possible d'observer l'utilisation des ressources du cluster Kubernetes, y compris l'utilisation du CPU et de la mémoire. Il est également possible d'identifier les applications qui requièrent le plus de ressources et d'analyser leur consommation à travers le temps. Ce schéma illustre les fluctuations de l'utilisation du processeur par divers processus, tels que argocd, default, kube-system et monitoring. L'usage du processeur semble être globalement constant, même si des pics d'activité sont remarqués pour certains processus, démontrant ainsi l'adaptabilité et la gestion dynamique des ressources au sein du cluster.

Conclusion

Ce chapitre a traité des configurations Helm permettant le déploiement de Grafana et d'Alertmanager, ainsi que des tableaux de bord Kubernetes spécifiquement conçus pour surveiller les ressources (CPU, Kubelet) et les opérations. Une attention particulière a été portée à l'optimisation des mesures et à la représentation visuelle des données à l'aide d'outils tels que Grafana. Ces éléments constituent un fondement robuste pour la surveillance et la maintenance d'un cluster Kubernetes efficace et réactif.

Chapitre 6

Release 4

6.1 Sprint1: Conception et développement de l'application JobyGénie

6.1.1 Introduction

Dans ce chapitre, nous explorerons les concepts théoriques qui guident le développement et l'architecture de notre projet. Des microservices et de l'architecture événementielle à l'architecture propre et la conception domaniale, nous examinerons comment ces principes façonnent notre approche pour construire des systèmes logiciels robustes. Nous couvrirons également les modèles de conception, grands modèles de langage et IA générative. En outre, nous présenterons l'architecture spécifique et le travail environnement de notre projet

6.1.2 Specification Architecturale

6.1.2.1 Architecture Microservices

Les microservices désignent à la fois une architecture et une approche de développement logiciel.

Cette approche consiste à décomposer les applications en éléments les plus simples, indépendants les uns des autres. Contrairement à une approche monolithique classique, selon laquelle tous les composants forment une entité indissociable, les microservices fonctionnent en synergie pour accomplir les mêmes tâches, tout en étant séparés. Chacun de ces composants ou processus est un microservice [8]. Ces processus peuvent communiquer ensemble afin de fournir les fonctionnalités nécessaires.

Les microservices sont, dans la plupart du temps, implémentés et pilotés par des équipes de petite

ou supprimer des fonctionnalités de ce service avec un impact minimal sur les autres microservices. Ce style d'architecture présente plusieurs avantages comme l'hétérogénéité technologique, la résistance contre l'échec, la scalabilité sur mesure, la facilité de déploiement, l'alignement organisationnel, la réutilisabilité.

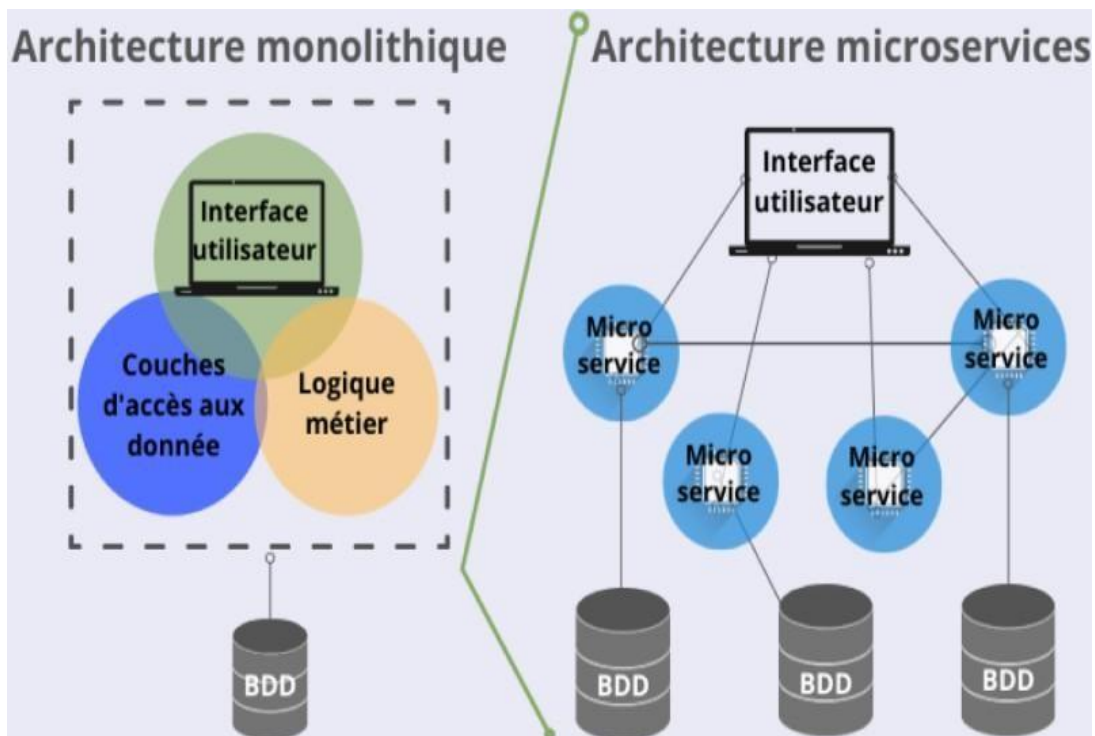


figure 6.1: la différence entre l'architecture monolithique et l'architecture Microservices

6.1.3 Caractéristiques de l'architecture microservices

L'architecture microservices repose sur plusieurs principes et caractéristiques qui permettent d'assurer une gestion flexible, scalable et résiliente des différentes fonctionnalités d'une application. Voici les principales caractéristiques qui définissent cette architecture :

1. Indépendance des microservices

Chaque microservice est conçu pour fonctionner de manière indépendante des autres. Cela signifie qu'un service peut être développé, déployé, mis à jour et géré de manière autonome. Cette indépendance offre une flexibilité maximale lors des évolutions de l'application.

Exemple : Le service ms-auth peut être mis à jour sans affecter les autres microservices comme ms-dashboard-and-analytics.

2. Communication asynchrone

Les microservices communiquent entre eux principalement de manière asynchrone via des files d'attente ou des systèmes de messagerie tels que Amazon SQS, Kafka ou RabbitMQ. Cette approche permet de découpler les services, favorisant ainsi une meilleure résilience du système

Exemple : Lorsqu'un utilisateur soumet un CV, le service ms-auth peut valider l'utilisateur et envoyer un message dans une file d'attente, qui sera ensuite consommé par le service cvanalyser pour l'analyse du CV.

3. Scalabilité

Chaque microservice peut être mis à l'échelle indépendamment en fonction de la charge, ce qui permet d'optimiser l'utilisation des ressources. Cela assure une gestion efficace des pics de trafic sans impacter les autres services de l'application.

Exemple : Si le service ms-job-and-application connaît une forte demande pendant les périodes de candidature, il peut être mis à l'échelle sans affecter les autres microservices.

4. Résilience et tolérance aux pannes

L'architecture microservices permet une gestion plus robuste des pannes. Si un service échoue, les autres services continuent de fonctionner normalement. De plus, la communication via des files d'attente garantit que les messages sont stockés et traités ultérieurement si un service devient temporairement indisponible.

5. Déploiement indépendant

Chaque microservice peut être déployé indépendamment des autres. Cela permet de faciliter les déploiements continus et d'éviter des interruptions du système entier lors des mises à jour d'un microservice spécifique.

Exemple : Le service ms-dashboard-and-analytics peut être mis à jour et déployé sans nécessiter la mise à jour des autres services comme ms-auth.

6. Technologies et outils spécifiques

Les microservices sont déployés et orchestrés à l'aide de technologies modernes telles que Kubernetes (EKS), Helm, et Docker, qui permettent une gestion optimale de l'infrastructure et des conteneurs.

Exemple : Kubernetes dans EKS permet de gérer la mise à l'échelle, le déploiement et l'orchestration des services de manière fiable et automatisée.

7. Sécurité et gestion des identités

La sécurité est un aspect crucial de l'architecture microservices. Chaque service gère son propre mécanisme d'authentification et d'autorisation, souvent via des standards comme OAuth, JWT, ou des rôles d'accès dans AWS IAM. Cela permet de garantir la confidentialité et l'intégrité des données entre les services.

Exemple : Le service ms-auth est responsable de l'authentification des utilisateurs et de la gestion des rôles d'accès, garantissant ainsi la sécurité des autres services.

8. Base de données par service

Chaque microservice peut posséder sa propre base de données, ce qui permet d'éviter les dépendances entre services et d'assurer une meilleure isolation des données. Cette approche améliore la flexibilité et la résilience du système, car chaque service peut évoluer indépendamment.

Exemple : Le service ms-auth pourrait utiliser une base de données relationnelle, tandis que ms-job-and-application pourrait opter pour Amazon DocumentDB pour le stockage des candidatures et des offres d'emploi.

9. Gestion des versions et évolutivité des API

Chaque microservice expose une API qui peut évoluer indépendamment des autres. La gestion des versions de ces API est essentielle pour maintenir la compatibilité entre les différentes versions des microservices, garantissant ainsi une transition fluide lors des mises à jour.

Exemple : Si une nouvelle version du service ms-dashboard-and-analytics est

déployée, l'API de ce service peut être modifiée sans perturber les autres services ou les utilisateurs finaux.

Cette structure te permet de décrire clairement chaque aspect de l'architecture microservices, en soulignant ses avantages et son rôle dans la conception de l'application JobyGénie.

6.3. Microservices implémentés

6.3.1. MS-Auth (Authentification)

6.3.1.1. Contexte

Le MS-Auth est le gardien de l'accès à JobyGénie. Il permet aux utilisateurs de s'inscrire, de se connecter de manière sécurisée, et de gérer leur profil. Son architecture repose sur Spring Security et JWT pour garantir une authentification fiable et scalable. L'objectif de ce sprint est de mettre en œuvre les fonctionnalités suivantes : inscription, connexion.



Figure 6.2: Diagramme de cas d'utilisation de l'authentification

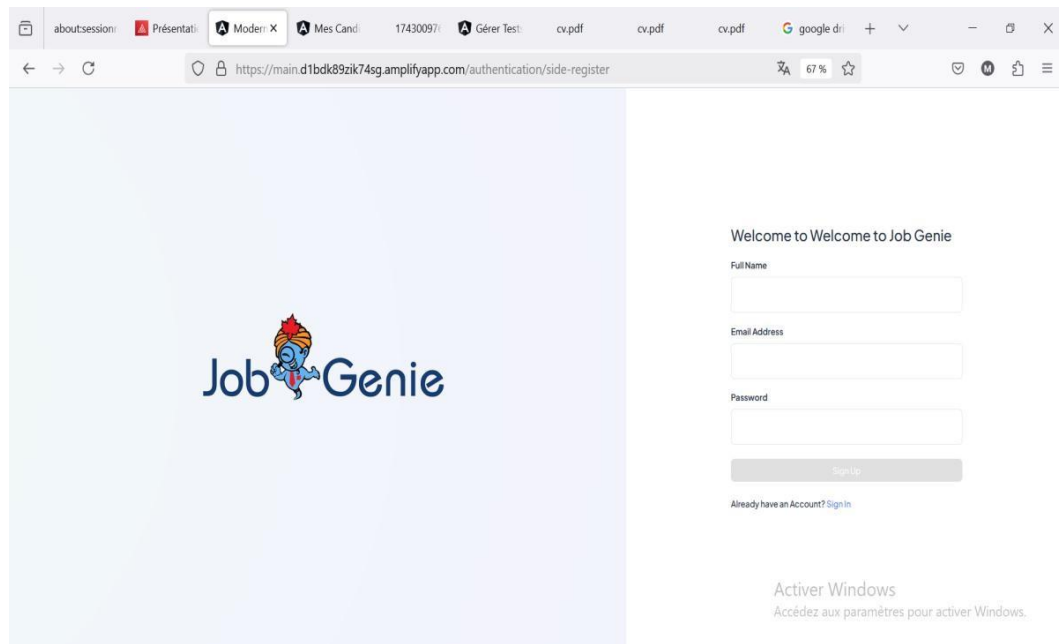


Figure 6.3 : Interface d'inscription

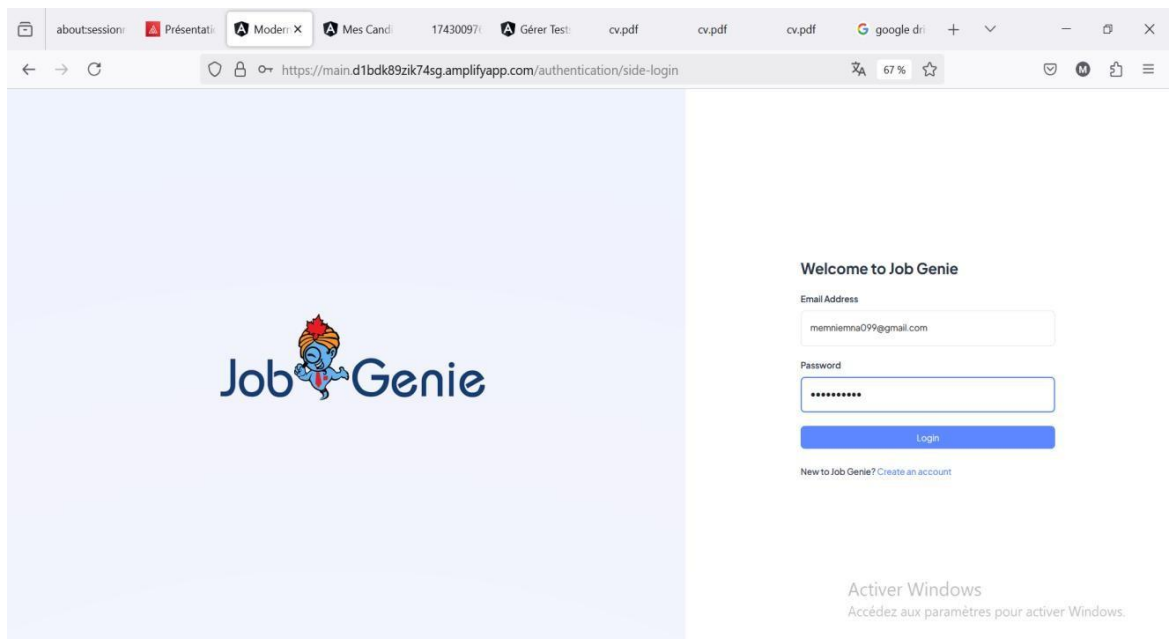


Figure 6.4 : Interface d'authentification

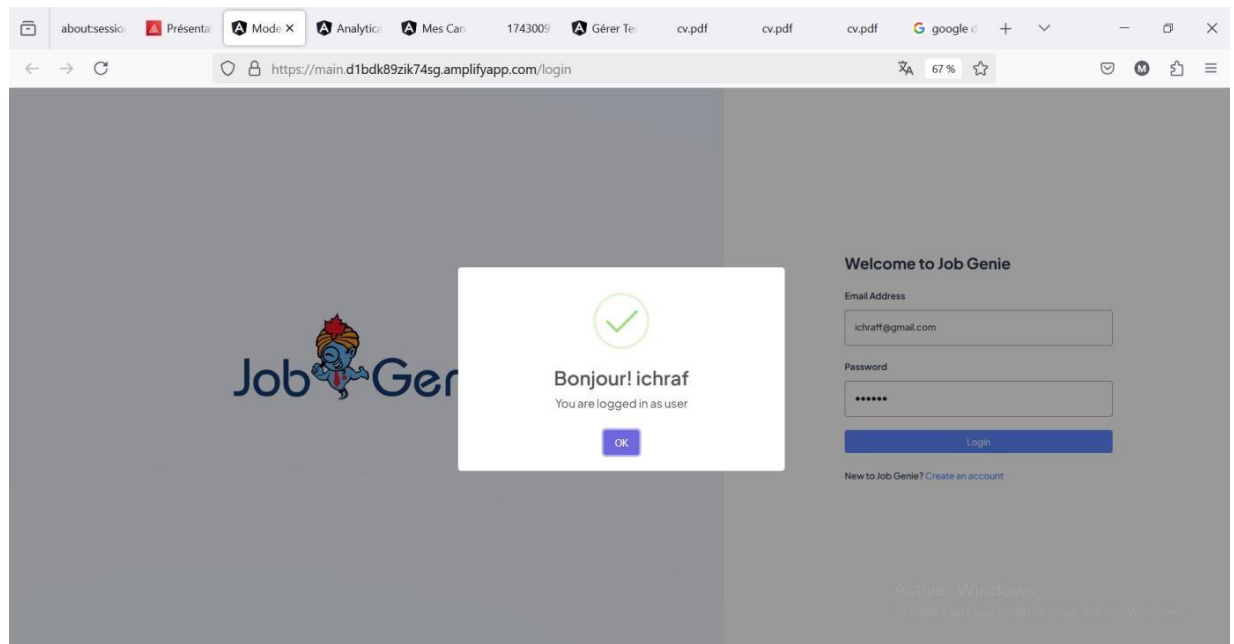


Figure 6.5: Confirmation de connexion sur JobyGenie

6.3.2. MS-Job-AndApplication (Offres et candidatures)

6.3.2.1. Contexte

Le microservice MS-Job-And-Application est un composant central de notre plateforme, conçu pour gérer efficacement les offres d'emploi et les processus de candidature. Ce service permet aux employeurs de publier, mettre à jour et gérer leurs annonces, tout en offrant aux candidats la possibilité de consulter les opportunités et de postuler de manière fluide.

6.3.2.2. Fonctionnalités

La gestion des offres d'emploi inclut la création, modification et suppression d'annonces, la catégorisation par secteur, localisation et niveau d'expérience, ainsi que la possibilité de rechercher et filtrer les offres disponibles. Pour le traitement des candidatures, les utilisateurs peuvent soumettre leurs candidatures, comprenant CV et lettres de motivation, et suivre leur statut (Envoyée, En revue, Acceptée, Rejetée) avec des notifications automatisées pour chaque mise à jour. Enfin, le tableau de bord employeur permet de visualiser et de gérer les candidatures reçues.

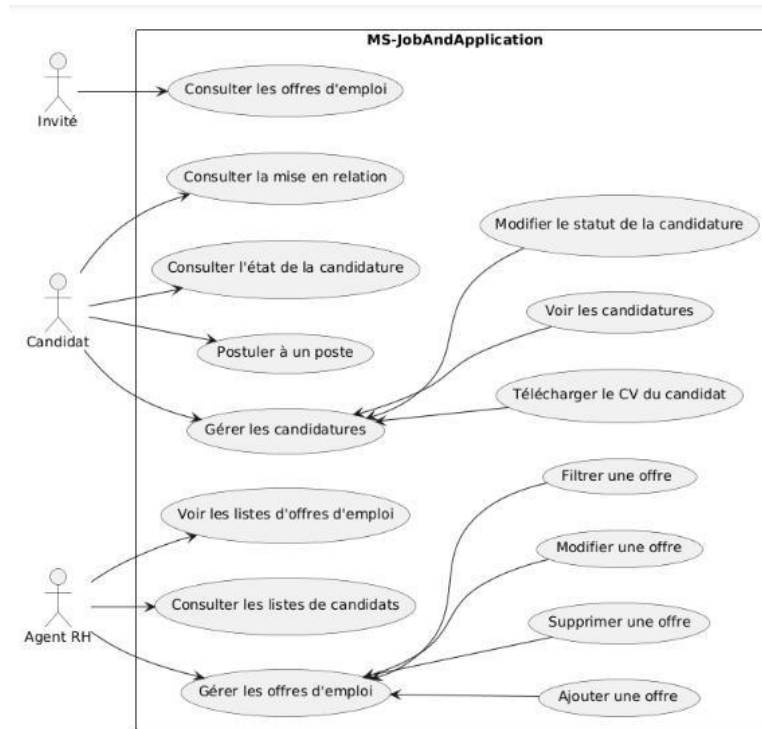


Figure 6.6 : Diagramme de cas d'utilisation du MS-JobAndApplication

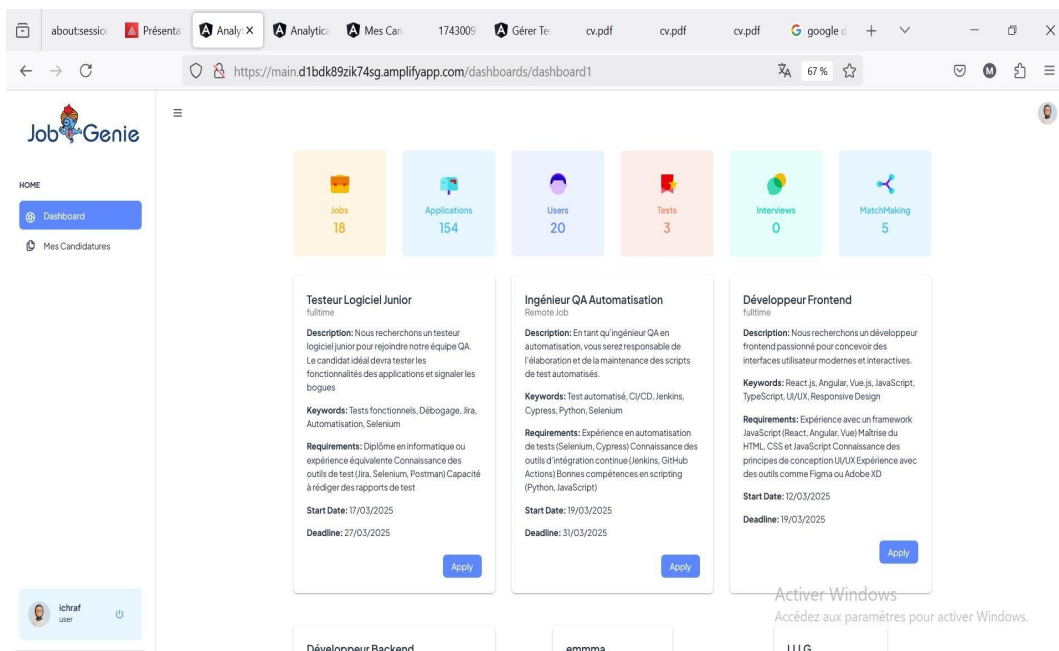


Figure 6.7: Vue du tableau de bord affichant les statistiques (jobs, candidatures, utilisateurs, tests, entretiens, matchmaking) et les offres d'emploi disponibles sur la plateforme JobyGénie

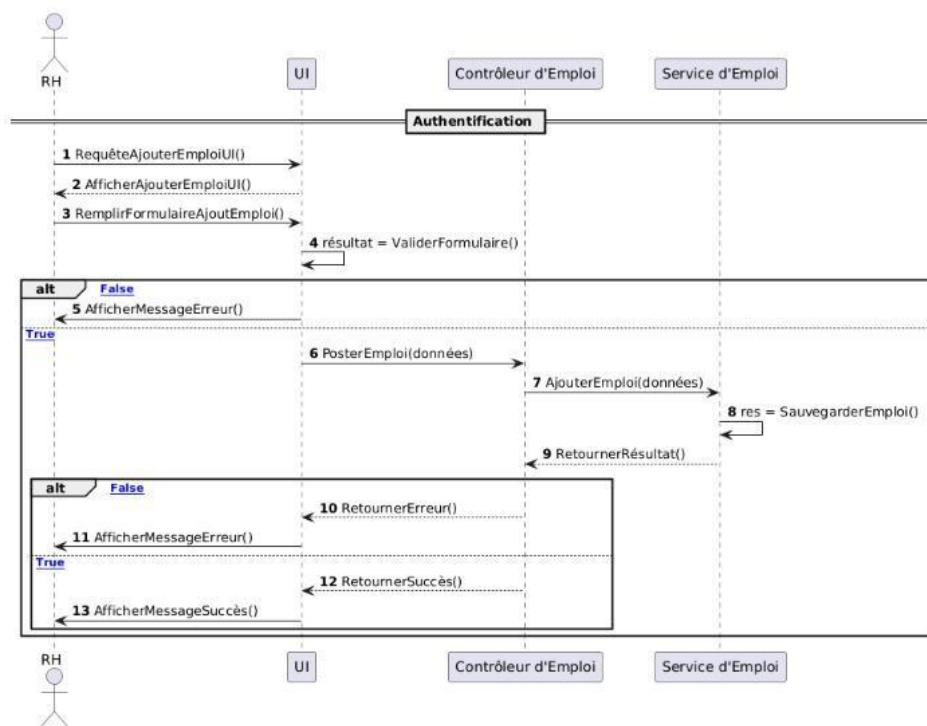


Figure 6.8: Diagramme de séquence – Ajout d'une offre d'emploi par le RH

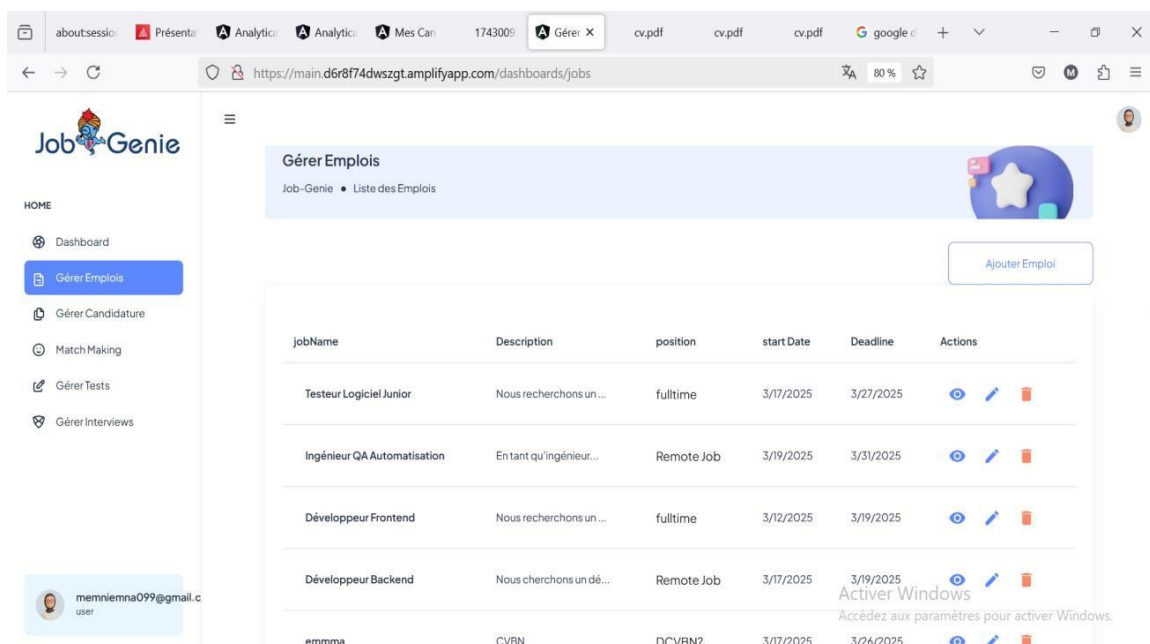


Figure 6.9: Interface de Gestion des Emplois - JobYGenie

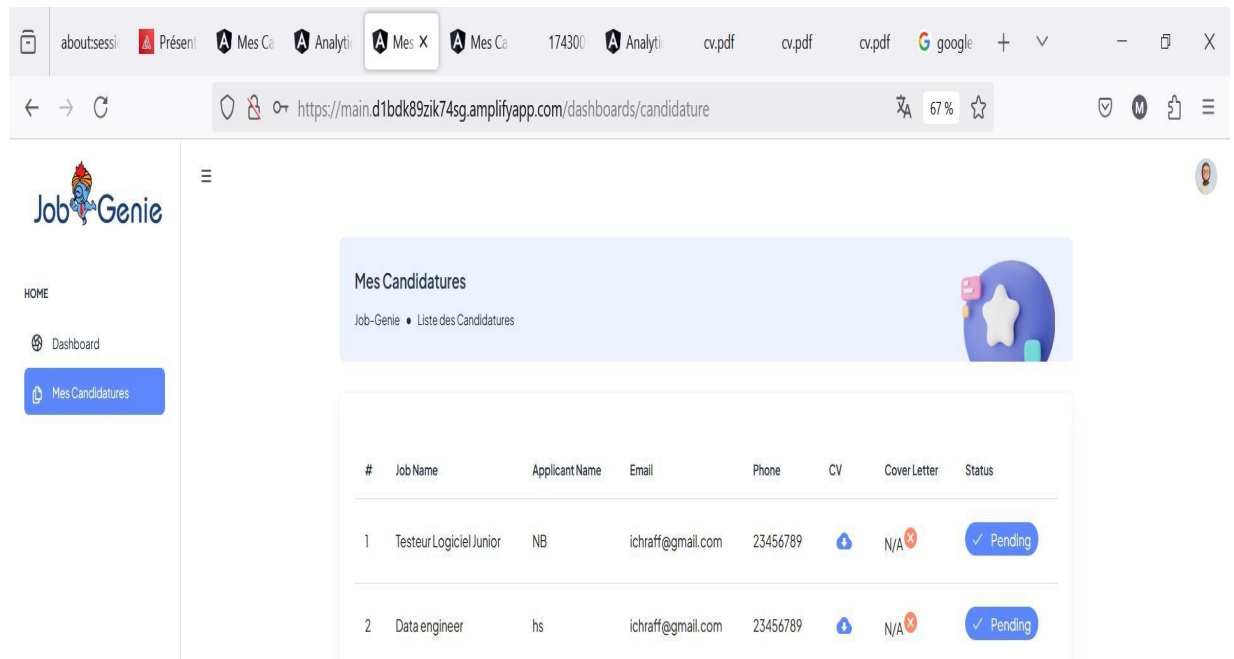


Figure 6.10: Interface de Suivi des Candidatures JobyGenie

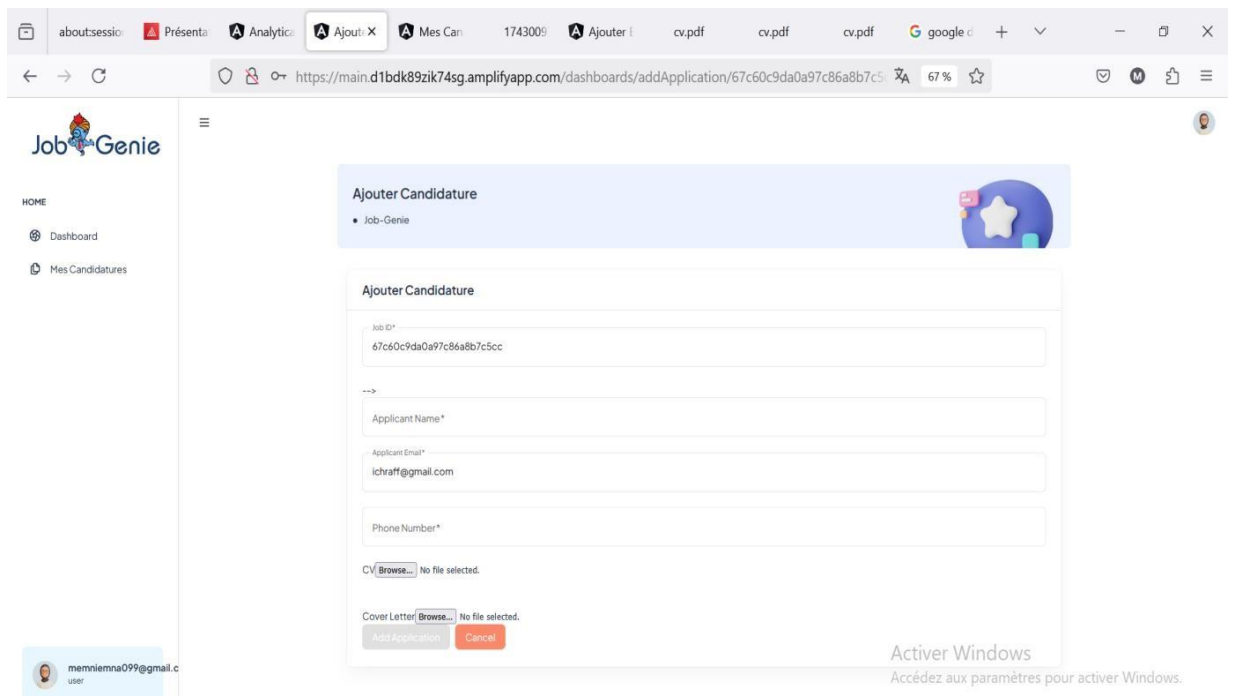


Figure 6.11: Formulaire d'Ajout de Candidature - Joby Genie

6.3.3 Microservice CvAnalyzer

6.3.3.1. Contexte

Le microservice « CvAnalyzer » vise à améliorer le processus de recrutement en automatisant l'analyse et la mise en correspondance des CV des candidats avec les descriptions de poste. Ce service répond aux défis auxquels les recruteurs sont confrontés lors de l'examen manuel d'un grand nombre de CV et de la vérification de l'adéquation des candidats aux exigences du poste.

En s'appuyant sur une technologie d'IA avancée, le microservice offre un screening de CV précis et efficace, ainsi que des résumés des candidats.

Pour réaliser la mise en correspondance et l'analyse des CV, il est essentiel de comprendre et de traiter efficacement le texte. Afin de sélectionner le modèle préentraîné le plus adapté à notre tâche, nous avons mené une étude comparative entre différentes options.

Cette section décrit les fonctionnalités et capacités fondamentales du microservice "Cv- Analyzer", en mettant en avant son rôle essentiel dans l'analyse des CV par l'IA et l'appariement des candidats avec les offres d'emploi.

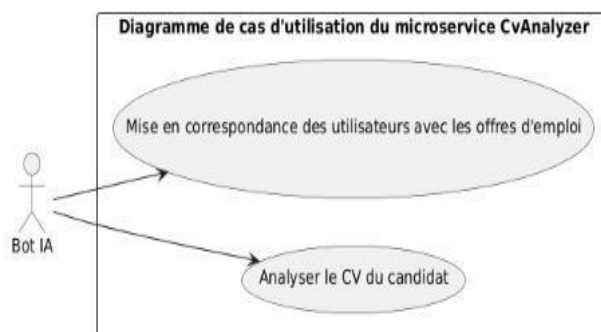


Figure 6.12:Diagramme de Cas d'Utilisation - Microservice CVAnalyzer

6.3.3.2 Comparaison des modèles pré-entraînés pour l'analyse et la mise en correspondance de texte

Fonctionnalité	Gemini (Google)	GPT-3.5 (OpenAI)	GPT-4 (OpenAI)
Compréhension contextuelle	Excellente, optimisé pour le multitâche	Bonne, mais limitée sur les longs contextes	Supérieure, meilleure cohérence sur les textes longs
Précision en matching sémantique	Hautement performant (optimisé NLP)	Correct, mais sensible aux ambiguïtés	Très précise, gestion fine des nuances
Multilingue	Pris en charge (dont français)	Pris en charge (performance variable)	Améliorée (meilleure que GPT-3.5)
Traitement des longs documents	Bonne segmentation et analyse	Limité par la taille de contexte (~4k tokens)	Étendu (~32k tokens en version turbo)
Vitesse d'inférence	Rapide (optimisation Google TPU)	Rapide (mais moins que GPT-4)	Variable (plus lent en mode haute précision)
Coût	Variable (selon utilisation Google Cloud)	Économique (API peu coûteuse)	Plus élevé (API premium)
Cas d'usage recommandés	Matching CV/offres, analyse structurée	Chatbots, tâches génériques	Applications critiques (ex: recrutement)
Fine-tuning	Possible (via Vertex AI)	Limité (moins flexible)	Plus d'options (accès avancé)

Figure 6.13: Tableau Comparatif des Modèles d'IA : Gemini, GPT-3.5 et GPT-4

Après avoir mené l'étude comparative et la recherche, nous avons décidé de travailler avec Gemini pour plusieurs raisons. Gemini offre des performances exceptionnelles dans les tâches basées sur le texte, avec une génération de texte créative supérieure et une compréhension plus large. De plus, il fournit des résultats plus précis, ce qui est crucial pour notre service de tri et de mise en correspondance des CV. En outre, le modèle tarifaire de Gemini inclut un niveau gratuit, ce qui en fait un choix rentable pour nos besoins.

Le cœur du "MatchMaking-Service" est basé sur le modèle d'IA générative de Google. Ce modèle est conçu pour générer du texte semblable à celui d'un humain à partir des entrées fournies, ce qui le rend idéal pour des tâches telles que l'analyse des CV et des descriptions de poste.

Comment cela fonctionne Le modèle d'IA générative, spécifiquement le modèle "gemini-pro", est configuré avec des paramètres spécifiques afin d'assurer la génération de contenu pertinent et sécurisé.

6.3.3.3 Captures des interfaces de Microservice CvAnalyzer

The screenshot shows the JobyGenie Match Making interface. The left sidebar contains a menu with options: HOME, Dashboard, Gérer Emplois, Gérer Candidature, Match Making (selected), Gérer Tests, and Gérer Interviews. The main content area displays a table with the following data:

Job Name	Applicant Email	CV	Match Percentage	Technical Tests	Actions	Test Status
system administrator	emnamemni5@gmail.com		15%	Generation launched	Smart Analysis	→
Data engineer	ichraff@gmail.com		15%	Generation launched	Smart Analysis	→
AMAAA	y@gmail.com		1%	Generation launched	Smart Analysis	→

At the bottom right, there is a notification: "Activier Windows. Accédez aux paramètres pour activer Windows."

Figure 6.14: Interface de Matching IA - JobyGenie

The screenshot shows the JobyGenie Match Making interface with a modal window open. The modal window displays the following information:

IA Analysis for: system administrator Role

Candidate Summary:

Experience:

Job Match Percentage: 15%

Missing Keywords: Linux, Windows, Server, Cloud (although AWS is mentioned, broader Cloud experience is implied), Networking, System Administrator, IT Infrastructure, Troubleshooting, Active Directory, DNS, DHCP, Security, Scripting (e.g., Bash, PowerShell), Virtualization (e.g., VMware, Hyper-V), Monitoring

Job Name: system administrator

At the bottom of the modal window, there is a "Close" button.

Figure 6.15 : Résultat de l'analyse intelligente d'adéquation pour le poste de "System Administrator"

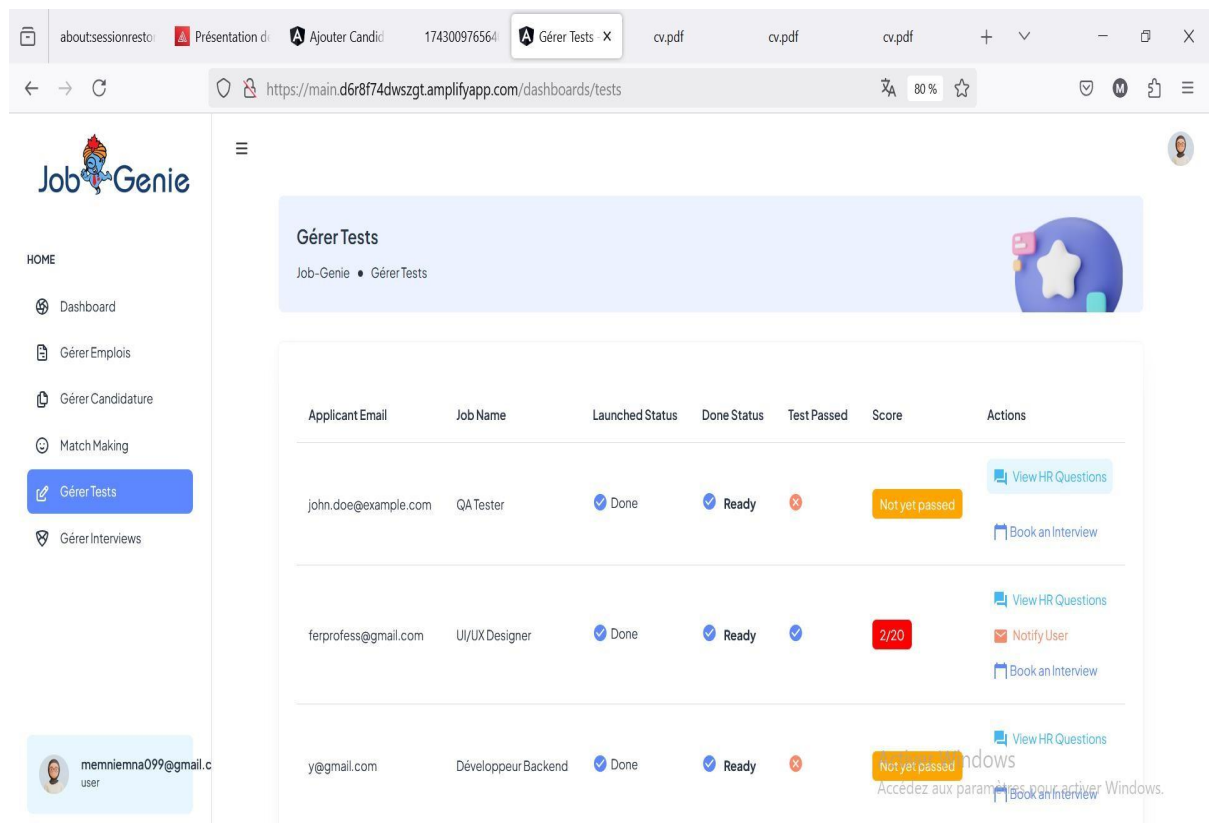


Figure 6.16: Interface de gestion des tests techniques des candidats dans JobyGénie

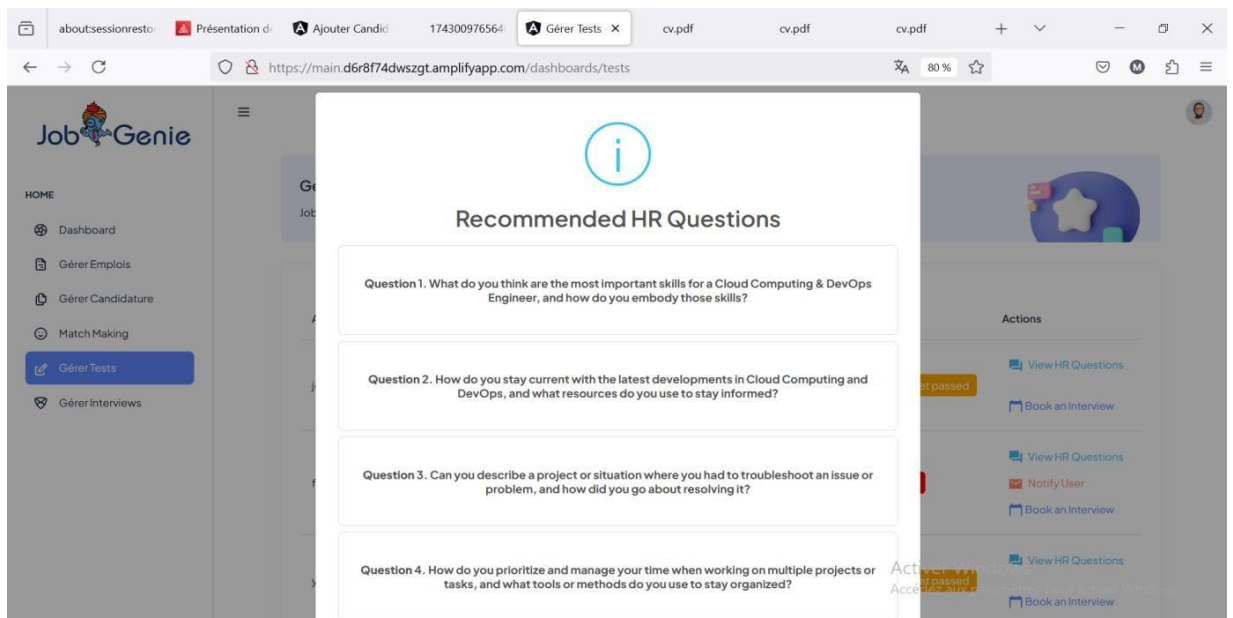


Figure 6.17: des Questions RH pour Profils Techniques Cloud Computing

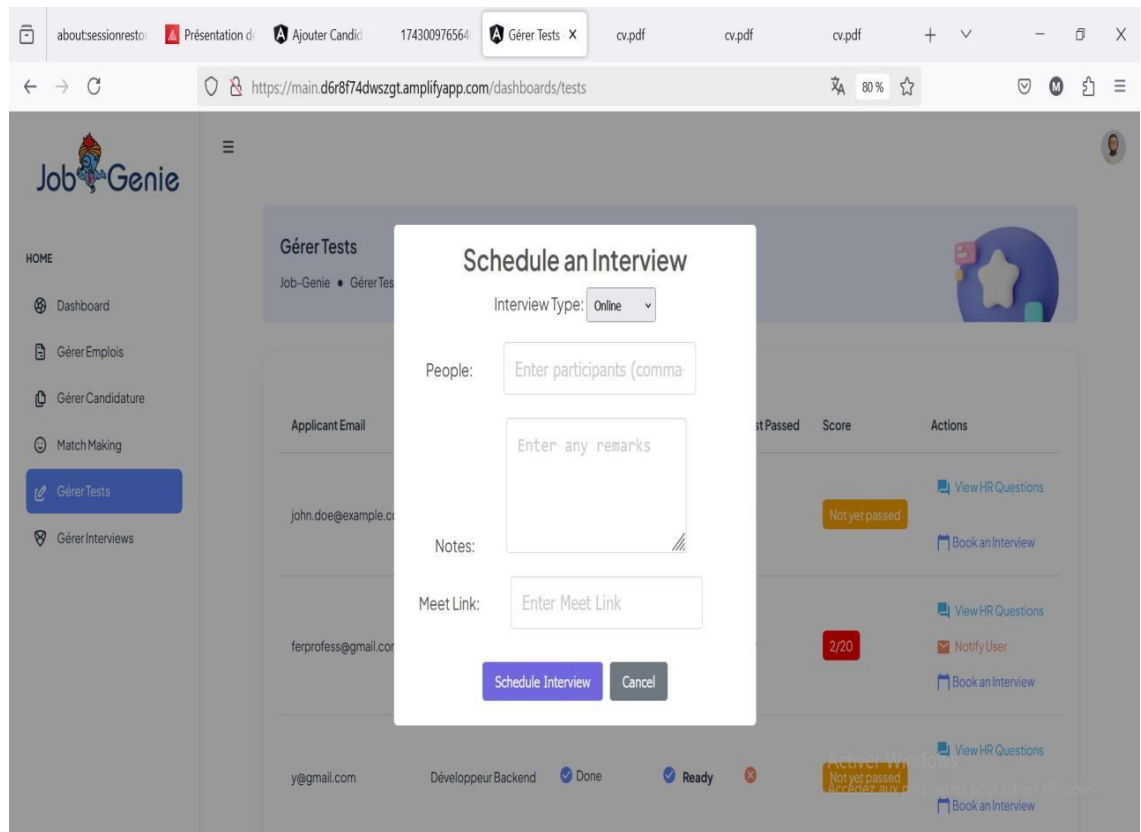


Figure 6.18: Interface de planification d'entretien

Conclusion

Au fil de ce chapitre, nous avons minutieusement étudié la phase de création de notre application, en soulignant les étapes cruciales qui ont mené à sa concrétisation. Cette partie a servi à illustrer l'application pratique des fonctionnalités élaborées.

CONCLUSIONS AND PERSPECTIVES

Conclusion général

Pour conclure, ce projet terminal a illustré de manière concrète l'effet bénéfique des approches DevOps et DevSecOps dans la gestion contemporaine des applications.

En utilisant des outils sophistiqués et des processus automatisés, nous avons développé et mis en place une application web RH efficace et sécurisée.

Le projet a souligné l'importance de l'automatisation dans les processus de développement et de déploiement, permettant ainsi de réduire les temps d'attente et de minimiser les erreurs humaines. Kubernetes a facilité une orchestration efficace des conteneurs, tout en procurant la souplesse nécessaire pour ajuster l'application en fonction des exigences évolutives de l'entreprise. L'implémentation de pipelines CI/CD a aussi contribué à une meilleure fluidité dans les déploiements.

sur une livraison constante et fiable de fonctionnalités nouvelles

Sur le plan individuel. Cette expérience m'a offert l'opportunité d'acquérir des compétences diversifiées dans différents secteurs. particulièrement dans le domaine de l'ingénierie logicielle. dans la sécurisation des environnements informatiques. Elle m'a aussi éveillé à l'importance de la coopération entre les équipes techniques, un aspect fondamental des approches DevOps et DevSecOps.

Perspectives

Dans l'optique d'une amélioration constante, on peut considérer divers axes pour approfondir et renforcer la sécurité de la solution :

Renforcement de la sécurité : intégrer des solutions comme **Wazuh** pour une **détection proactive des menaces**, couplée à une surveillance en temps réel du comportement des applications et des ressources

Intégration de l'IA dans les pratiques DevOps : exploiter des modèles d'IA pour la **détection automatique d'anomalies**, la **prédiction de défaillances**, ou encore l'**optimisation dynamique des ressources cloud**.

Scalabilité avancée : adapter dynamiquement les ressources en fonction de la charge, en intégrant des mécanismes d'**autoscaling** basés sur des métriques prédictives.

Amélioration de l'expérience utilisateur : affiner les outils de visualisation (Grafana, dashboards personnalisés) pour une meilleure compréhension des performances et de l'état global du système.

Bibliographie

Amazon Web Services (AWS) Documentation officielle AWS
<https://docs.aws.amazon.com>

Kubernetes The Kubernetes Documentation <https://kubernetes.io/doc>

Helm Helm Charts Documentation <https://helm.sh/docs>

Terraform by HashiCorp Terraform Documentation
<https://developer.hashicorp.com/terraform>

GitHub Actions Documentation GitHub Actions <https://docs.github.com/actions>

Prometheus Prometheus Monitoring Documentation <https://prometheus.io/docs>

Grafana Labs Grafana Documentation <https://grafana.com/docs>

Trivy by Aqua Security Trivy - Vulnerability Scanner
<https://aquasecurity.github.io/trivy>

OWASP OWASP ZAP & Dependency-Check Documentation <https://owasp.org>

ArgoCD GitOps Continuous Delivery <https://argo-cd.readthedocs.io>



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454